

Lectures on Intelligent Weapon Systems

A systems-engineering textbook through history, models, and responsible decisions

Jiageng Liu

Associate Professor

Information Countermeasure Teaching and Research Office

Shenyang Ligong University

Academic year 2026–2027

Intelligent Weapon Systems

An English textbook for a 24-hour course

IWS

About this book

This textbook supports a twelve-lecture course on intelligent weapon systems. It treats a safety-critical system as an engineering system with a platform, sensing, information processing, decision and execution functions, communication, safety constraints, and assigned human responsibilities.

The book uses historical narratives as short introductions to technical questions. A historical text is not a laboratory log, and an analogy is not a proof. Each chapter therefore separates four things: what the source says, what the narrative reconstructs, what the engineering model assumes, and what the student is asked to decide. The technical treatment remains at the level of system architecture, estimation, planning, control, risk, verification, and responsibility. It does not provide operational procedures for real weapons, target selection, deployment, or evasion.

The chapters build on one another. Chapters 1–4 establish system boundaries, platforms, state transitions, and motion planning. Chapters 5–7 develop uncertainty, thresholds, lifecycle risk, and time-critical decisions. Chapters 8–9 address restricted communication and evidence management. Chapters 10–12 examine information fusion, navigation, integrated systems, and responsibility under irreversible decisions.

How to use these notes

Each chapter contains a narrative prelude, a source note, a foundational-knowledge module, formal definitions, modeling assumptions, mathematical models, worked examples, algorithmic patterns, design checks, historical or methodological notes, review questions, and exercises. The foundational module develops the chapter's prerequisite concepts through definitions, minimum mathematics, a worked calculation, failure boundaries, and an explicit bridge to the application-specific discussion. It also points to public resources for a deeper review. Values marked [Illustrative teaching value] are fictional classroom values; they are not performance data for any real system. Exercises are written for paper analysis, simulation, or safe toy systems.

The companion English hand-drawn slides follow the same chapter order. The Chinese teaching scripts contain the complete spoken narrative and discussion prompts; this book supplies the stable technical vocabulary, models, derivations, and exercises that a student can revisit after class.

Learning objectives for the course

By the end of the course, a student should be able to:

1. describe an intelligent weapon system as a feedback system with technical functions and human responsibilities, rather than an isolated algorithm;
2. distinguish observation, estimation, classification, planning, control, authorization, and accountability;
3. formulate simple state-space, graph, probabilistic, and cost models for safety-critical autonomous functions;
4. evaluate uncertainty, false alarms, missed states, distribution shift, communication limits, and lifecycle risk;
5. design a verification argument with a documented record for a fictional system scenario;
6. explain why system intelligence increases the need for boundaries, records, human judgment, and responsible risk management.

Foundational knowledge route

The book assumes no specialized knowledge of intelligent weapon systems. The technical sequence is built on a small set of public, reviewable foundations:

Foundation	Public review resource	Used in this book
Systems engineering	NASA Systems Engineering Handbook	boundaries, requirements, verification, life-cycle
Risk and AI risk management	NIST AI Risk Management Framework	context, risk, responsibility, review
Linear algebra	MIT OCW 18.06SC	vectors, matrices, frames, covariance
Probability	MIT OCW 6.041SC	Bayes, base rates, thresholds, uncertainty
Robotics	MIT OCW 2.12	coordinate frames, motion, feedback
Planning	LaValle (2006)	free space, graphs, paths, replanning

Each chapter develops the foundation needed for its definitions and equations, then returns to the engineering problem. Students may use the embedded module for self-contained review or follow the public source for a deeper treatment.

Course map

The twelve lectures move from the system boundary to assigned responsibility. Each narrative opens a concrete question, while the engineering column identifies the formal object that students must be able to model, inspect, or question.

Lecture	Narrative lens	Engineering focus
1	Maling: a trace becomes a decision	Intelligent weapon concepts, architecture, evidence layers, feedback, and human responsibility
2	Tian Ji's three lines	Military robot types, platform architecture, capability vectors, and mission allocation
3	Cao Gui and the third drum	Unmanned-aircraft architecture, flight-state machines, degraded operation, and recovery
4	Cook Ding follows structure	Unmanned ground vehicles, traversability, path planning, motion control, and research reports
5	Relieving Zhao by changing the frame	Intelligent munitions, recognition confidence, relevance, authorization, and expected loss
6	The old man and the lost horse	Intelligent mines, lifecycle states, residual risk, withdrawal, and environmental responsibility
7	Bian Que sees the illness early	Terminally sensitive munitions, detection thresholds, sequential evidence, and time windows
8	Flood control through changing channels	Underwater robots, restricted communication, energy margins, and autonomy within stated limits
9	Three people make a tiger	Literature-review methods, provenance, configuration management, reproducibility, and evidence quality

Lecture	Narrative lens	Engineering focus
10	Borrowed arrows and hidden assumptions	Air-defense missile systems, information fusion, association, correlation, and uncertainty
11	The open plank road and the hidden route	Cruise-missile systems, navigation, guidance, map metadata, and evidence-driven replanning
12	Beacon fires and the loss of trust	Ballistic-missile systems, strategic time windows, safety cases, authorization, and documented records

Twelve chapters, twelve complete learning experiences

This course-aligned textbook uses Chapter k as the lasting reading companion for Lecture k . Each chapter supports one 90-minute class, preparatory reading, a safe studio activity, and a subsequent exercise set. The story presents a concrete human problem before the formal variables appear; the formal model then makes the resulting judgment inspectable.

How to read a chapter

Every chapter begins with four compact elements. The **Narrative Prelude** poses a question worth resolving. The **Source Note** labels the material as historical narrative (H), philosophical text (P), rhetorical account (R), literary narrative (L), classical proposition (T), or a fictional classroom example (C). The **Learning Objectives** state what can be demonstrated after the lecture. The **Foundational Knowledge** box identifies the prerequisite concepts and a public source for review.

The full derivation follows three rules. First, every symbol has a physical, informational, or organizational meaning. Second, every equation states its assumptions and does not silently turn a score into an authorization. Third, every worked value is marked as an example unless it comes from a cited public source. In a safety-critical systems course, mathematical fluency should increase a student’s caution about model scope.

Course-to-chapter map

Ch.	Story and engineering question	Foundational and mathematical spine	Safe case product or simulation
1	Maling: What turns a trace into a system decision?	Systems boundary; state, observation, uncertainty, feedback, evidence completeness.	Evidence-card simulator; optional LED/clock status board for a laboratory inspection scenario.
2	Tian Ji: Why can a new pairing outperform a stronger unit?	Vectors, normalization, binary assignment, feasibility, sensitivity.	Three-platform allocation simulator or a token-and-status teaching board.

Ch.	Story and engineering question	Foundational and mathematical spine	Safe case product or simulation
3	Cao Gui: When does an event justify a state transition?	Logic, finite-state machines, guards, timers, reachability, degradation.	Synthetic aerial-vehicle state-machine simulator; traffic-light-style protected-state board.
4	Cook Ding: Why is the shortest route not always the viable route?	Frames, distance, graphs, costs, feedback, path tracking.	Indoor mobile-robot or grid-world simulation with collision margins and route explanations.
5	Relieving Zhao: Why is a high score not an authorization?	Conditional probability, Bayes' rule, confusion matrices, calibration, expected loss.	Harmless classification-and-refusal demonstrator using labeled classroom objects or synthetic data.
6	The old man and the lost horse: What remains after a task ends?	Lifecycle states, hazard analysis, residual-risk accumulation, FMEA-style reasoning.	Persistent environmental-sensor lifecycle board or a state-and-log simulator.
7	Bian Que: Why must early weak evidence be preserved?	Signal detection, likelihood ratios, sequential updating, time windows, alarm fatigue.	Tabletop sequential-alarm simulator with a protected-state output.
8	Water and the deep: What does autonomy mean when communication is sparse?	Queues, message freshness, age of information, energy budget, partial observability.	Water-tank communication-window demonstration or a delay-and-energy simulation.
9	Three people make a tiger: When does repetition become false confidence?	Source types, provenance graphs, configuration management, reproducibility matrices.	Literature-provenance dashboard for a public technical claim.
10	Borrowed arrows: Are several reports independent evidence?	Measurement models, association, covariance, prediction-update estimation, common-mode error.	Multi-sensor airspace analogue with synthetic tracks and disagreement review.
11	The open route and the hidden route: Which map is current enough to guide a decision?	Reference frames, coordinate transforms, route cost, metadata, versioned replanning.	Two-dimensional navigation and map-version simulator for an inspection route.

Ch.	Story and engineering question	Foundational and mathematical spine	Safe case product or simulation
12	Before the final decision: Which claims must hold before responsibility can move forward?	Reliability chains, safety cases, responsibility matrices, documented decision records.	Strategic authorization and safety-case simulator using fictional warnings and safe states.

Assessment artefacts across the twelve chapters

The chapter studios form a cumulative portfolio. By the end of the course, each student should be able to assemble: (1) a system boundary statement; (2) a constrained allocation model; (3) a verified state machine; (4) a map and path-cost explanation; (5) a calibrated decision card; (6) a lifecycle risk register; (7) a sequential-evidence log; (8) a communication-and-energy budget; (9) a provenance graph; (10) a fusion disagreement report; (11) a versioned replanning record; and (12) a safety case with a responsibility matrix. The portfolio assesses modeling, evidence, explanation, and responsibility. It does not assess the construction, deployment, or optimization of real weapon functions.

Permitted studio formats

Each case studio is deliberately transferable to an ordinary laboratory context. Acceptable formats include a spreadsheet, a Python or MATLAB simulation, a paper card game, a mobile-robot exercise in a marked indoor area, or a harmless teaching interface designed in JLCEDA and fabricated through JLCPCB. A physical board may contain indicators, buttons, timers, displays, harmless sensors, and serial logging, but it must represent only evidence status, state transitions, lifecycle records, or review prompts. It must not sense, select, track, guide, or actuate real-world weapon targets.

Contents

Foundational knowledge route	vi
Course map	vii
Twelve chapters, twelve complete learning experiences	ix
How to read a chapter	ix
Course-to-chapter map	ix
Assessment artefacts across the twelve chapters	xi
Permitted studio formats	xi
 I Systems, Platforms, and Feedback	 1
1 Intelligent Weapon Systems: Concepts, Architecture, and Evidence	2
1.1 Foundations: systems, models, and feedback	3
1.1.1 From a real situation to a model	3
1.1.2 State, input, output, and disturbance	4
1.1.3 Feedback changes the meaning of an action	5
1.1.4 Requirements, measures, and verification	5
1.1.5 Foundation self-check	6
1.1.6 Bridge to the Maling system question	6
1.2 Historical opening and source note	6
1.2.1 Foundational knowledge to master	7
1.2.2 Learning content and mathematical derivation	7
1.2.3 Case studio: the evidence-card simulator	8
1.2.4 Whole-chapter synthesis	8
1.2.5 Practice and extension	8
1.3 The story as a systems problem	9
1.4 Problem setup and modeling assumptions	10
1.4.1 What is inside the boundary?	11
1.5 A functional decomposition	11
1.6 Four evidence layers	12
1.6.1 A minimal decision record	13

1.7	From observation to decision	14
1.7.1	Confidence is conditional	14
1.8	Assumptions, interfaces, and failure modes	14
1.8.1	Failure propagation	15
1.9	A small architecture exercise	16
1.9.1	Pass 1: physical components	16
1.9.2	Pass 2: logical functions	16
1.9.3	Pass 3: authority and recovery	16
1.10	Historical reading and methodological interpretation	17
1.11	Chapter laboratory: the evidence card	17
1.12	Interface contracts and semantic failure	17
1.13	Trace reconstruction exercise	18
1.14	Chapter design assignment	18
1.15	Historical notes and further reading	19
1.16	Exercises	19
2	Intelligent Military Robots: Platforms, Capability, and Mission Allocation	21
2.1	Foundations: vectors, comparison, and constrained allocation	22
2.1.1	Scalars, vectors, and matrices	22
2.1.2	Normalization is a modeling choice	22
2.1.3	From a table to an allocation problem	23
2.1.4	Feasibility precedes optimization	24
2.1.5	A small enumeration example	24
2.1.6	Foundation self-check	25
2.1.7	Bridge to Tian Ji's three lines	25
2.2	Historical opening and source note	25
2.2.1	Foundational knowledge to master	26
2.2.2	Learning content and mathematical derivation	26
2.2.3	Case studio: a safe multi-robot allocation board	26
2.2.4	Whole-chapter synthesis and responsibility discussion	27
2.2.5	Practice and extension	27
2.3	The unit is not the system	27
2.4	Assignment as a matrix problem	28
2.5	Autonomy is a matching variable	28
2.6	Capability vectors and normalization	29
2.7	Allocation with constraints	30

2.8	Trade-offs and sensitivity	31
2.9	Autonomy as a function-to-authority map	31
2.10	A classroom allocation game	32
2.11	Historical and methodological interpretation	32
2.12	Matching, greedy choice, and global objective	32
2.13	Assignment review and reassignment	33
2.14	Historical notes and further reading	33
2.15	Exercises	33
3	Military Unmanned Aircraft: State Machines and Safe Degradation	35
3.1	Foundations: logic, finite states, and guarded transitions	36
3.1.1	Sets and propositions	36
3.1.2	The finite-state machine	36
3.1.3	Time, hysteresis, and chattering	37
3.1.4	Mealy and Moore viewpoints	37
3.1.5	Reachability, dead ends, and test cases	37
3.1.6	Foundation self-check	38
3.1.7	Bridge to the third drum	39
3.2	Historical opening and source note	39
3.2.1	Foundational knowledge to master	39
3.2.2	Learning content and mathematical derivation	39
3.2.3	Case studio: a UAV state-machine simulator	40
3.2.4	Whole-chapter synthesis and responsibility discussion	40
3.2.5	Practice and extension	40
3.3	Events are not states	41
3.4	Time and safe degradation	41
3.5	State-machine vocabulary	42
3.6	Guard conditions and invariants	42
3.7	Timed transitions and hysteresis	43
3.8	Hybrid systems and uncertainty	44
3.9	Failure analysis of a state machine	44
3.10	Historical and methodological interpretation	45
3.11	Reachability and state-machine verification	45
3.12	Timed verification exercise	45
3.13	Historical notes and further reading	46
3.14	Exercises	46

4	Unmanned Ground Vehicles: Perception, Planning, and Feedback	47
4.1	Foundations: geometry, maps, paths, and feedback	48
4.1.1	Points, vectors, frames, and distance	48
4.1.2	Maps are claims, not pictures	48
4.1.3	Graphs and path cost	49
4.1.4	Planning and control are different functions	50
4.1.5	Replanning as evidence revision	50
4.1.6	Foundation self-check	51
4.1.7	Bridge to Cook Ding and structure	51
4.2	Historical opening and source note	51
4.2.1	Foundational knowledge to master	52
4.2.2	Learning content and mathematical derivation	52
4.2.3	Case studio: a safe indoor ground robot	52
4.2.4	Whole-chapter synthesis and research-report connection	53
4.2.5	Practice and extension	53
4.3	From geometry to traversability	53
4.4	Global planning, local action, and control	53
4.5	A physical path cost	54
4.6	Problem setup and sensing assumptions	54
4.7	Semantic maps and layered costs	55
4.8	Global planning and local planning	55
4.9	A simple controller	56
4.10	Cost functions and hard constraints	56
4.11	Perception-planning-control loop	57
4.12	Historical and methodological interpretation	57
4.13	Decomposition, sampling, and collision margins	58
4.14	Historical notes and further reading	58
4.15	Exercises	58
5	Intelligent Munitions: Classification Confidence, Relevance, and Authorization	60
5.1	Foundations: probability, classification, and constrained decisions	61
5.1.1	Events, random variables, and conditional probability	61
5.1.2	Classification and the confusion matrix	62
5.1.3	Scores, calibration, and thresholds	62
5.1.4	Expected utility and admissible action	63

5.1.5	Distribution shift	63
5.1.6	Foundation self-check	64
5.1.7	Bridge to strategic relevance	64
5.2	Historical opening and source note	64
5.2.1	Foundational knowledge to master	64
5.2.2	Learning content and mathematical derivation	65
5.2.3	Case studio: a harmless recognition-and-refusal demonstrator	65
5.2.4	Whole-chapter synthesis	65
5.2.5	Practice and extension	66
5.3	Four layers that must not impersonate one another	66
5.4	What confidence means	67
5.5	Unknown inputs and unequal errors	67
5.6	Hard constraints and admissible actions	67
5.7	Calibration and reliability diagrams	68
5.8	Base rates and conditional reasoning	68
5.9	Thresholds and operating points	69
5.10	Relevance is a separate inference	69
5.11	Admissible actions and authority	70
5.12	Distribution shift and challenge sets	70
5.13	Historical and methodological interpretation	71
5.14	Confusion matrices and selective prediction	71
5.15	Decision card exercise	71
5.16	Historical notes and further reading	71
5.17	Exercises	72
6	Intelligent Mines: Lifecycle Risk and the Ability to End	73
6.1	Foundations: lifecycle, hazards, and verification evidence	74
6.1.1	Lifecycle stages and the V-model intuition	74
6.1.2	Hazard, risk, and control	74
6.1.3	Failure modes and effects analysis	75
6.1.4	Fault trees and causal structure	75
6.1.5	Evidence gates and configuration management	76
6.1.6	Change, regression, and retirement	76
6.1.7	Foundation self-check	77
6.1.8	Bridge to the old man and the lost horse	77
6.2	Historical opening and source note	77

6.2.1	Foundational knowledge to master	77
6.2.2	Learning content and mathematical derivation	78
6.2.3	Case studio: persistent environmental sensor lifecycle board . .	78
6.2.4	Whole-chapter synthesis	79
6.2.5	Practice and extension	79
6.3	A system can finish its task and remain active	79
6.4	Risk accumulates across time	79
6.5	State, policy, and responsibility	80
6.6	Lifecycle stages and evidence gates	80
6.7	Hazards, causes, and controls	81
6.8	Residual risk over time	82
6.9	Change management and regression	82
6.10	Post-event learning	83
6.11	Historical and methodological interpretation	83
6.12	FMEA-style lifecycle analysis	84
6.13	Withdrawal test	84
6.14	Historical notes and further reading	84
6.15	Exercises	84
II	Uncertainty, Communication, and Integrated Responsibility	86
7	Terminally Sensitive Munitions: Thresholds and Time-Critical Decisions	87
7.1	Foundations: signals, thresholds, and time-critical review	88
7.1.1	Signal plus noise	88
7.1.2	A threshold is a policy parameter	88
7.1.3	Costs, review capacity, and escalation	89
7.1.4	Sequential evidence	89
7.1.5	Time margin and irreversible transitions	89
7.1.6	Human factors and alarm fatigue	90
7.1.7	Foundation self-check	90
7.1.8	Bridge to Bian Que's early diagnosis	91
7.2	Historical opening and source note	91
7.2.1	Foundational knowledge to master	91
7.2.2	Learning content and mathematical derivation	91
7.2.3	Case studio: a tabletop terminal-sensing simulator	92

7.2.4	Whole-chapter synthesis	92
7.2.5	Practice and extension	92
7.3	Signals, decisions, and consequences	93
7.4	The time window	93
7.5	Escalation rather than a single alarm	93
7.6	Detection theory in a classroom model	94
7.7	Receiver-operating characteristics	94
7.8	Sequential evidence and update discipline	95
7.9	Time-to-action and irreversibility	95
7.10	Escalation policy as a state machine	95
7.11	Trust, alarm fatigue, and human factors	96
7.12	Historical and methodological interpretation	96
7.13	Sequential thresholds and competing hypotheses	96
7.14	Alarm fatigue exercise	97
7.15	Historical notes and further reading	97
7.16	Exercises	97
8	Intelligent Underwater Robots: Autonomy Under Restricted Communication	99
8.1	Foundations: communication, queues, freshness, and local policy . . .	100
8.1.1	Message, packet, delivery, and acknowledgment	100
8.1.2	Bandwidth, latency, and loss	100
8.1.3	A simple queue and freshness example	101
8.1.4	Reliability mechanisms and their boundaries	101
8.1.5	Energy is part of the channel model	101
8.1.6	Local policy under intermittent information	102
8.1.7	Foundation self-check	102
8.1.8	Bridge to changing channels	103
8.2	Historical opening and source note	103
8.2.1	Foundational knowledge to master	103
8.2.2	Learning content and mathematical derivation	104
8.2.3	Case studio: water-tank communication-window demonstrator	104
8.2.4	Whole-chapter synthesis	104
8.2.5	Practice and extension	104
8.3	Why underwater systems are different	105
8.4	Communication windows and local policy	105

8.5	Autonomy within stated limits	106
8.6	A communication channel is a state variable	106
8.7	Queues, freshness, and information age	106
8.8	Energy and communication coupling	107
8.9	Local policy under partial observability	108
8.10	Delay, loss, and stale authority	108
8.11	Historical and methodological interpretation	109
8.12	Protocol semantics and delivery claims	109
8.13	Energy-budget exercise	109
8.14	Historical notes and further reading	109
8.15	Exercises	110
9	Intelligent Weapon Literature Reviews: Evidence and Configuration Management	111
9.1	Foundations: claims, sources, provenance, and reproducibility	112
9.1.1	Claim, evidence, source, and inference	112
9.1.2	Evidence status and source diversity	112
9.1.3	Provenance graphs	113
9.1.4	Configuration is part of meaning	113
9.1.5	Reproducibility and repeatability	114
9.1.6	Literature reading as model checking	114
9.1.7	Foundation self-check	115
9.1.8	Bridge to the “three people make a tiger” problem	115
9.2	Historical opening and source note	115
9.2.1	Foundational knowledge to master	116
9.2.2	Learning content and mathematical derivation	116
9.2.3	Case studio: provenance and review dashboard	116
9.2.4	Whole-chapter synthesis	117
9.2.5	Practice and extension	117
9.3	Evidence has a lineage	117
9.4	Configuration is part of the claim	118
9.5	Circular evidence	118
9.6	Claims, sources, and provenance graphs	118
9.7	Configuration as a coordinate of meaning	119
9.8	Literature review as an engineering activity	119
9.9	Reproducibility and audit trails	120

9.10	The repetition trap	120
9.11	Historical and methodological interpretation	121
9.12	Evidence graphs and source diversity	121
9.13	Reproducibility matrix	121
9.14	Historical notes and further reading	121
9.15	Exercises	122
10	Air-Defense Missiles: Information Fusion and Integrated Systems	123
10.1	Foundations: estimation, covariance, and information fusion	124
10.1.1	Mean, variance, and covariance	124
10.1.2	Weighted fusion of two scalar estimates	124
10.1.3	Prediction and measurement update	125
10.1.4	Association and semantic compatibility	125
10.1.5	Common-mode error and conservative fusion	126
10.1.6	Foundation self-check	126
10.1.7	Bridge to borrowed arrows and hidden assumptions	127
10.2	Historical opening and source note	127
10.2.1	Foundational knowledge to master	127
10.2.2	Learning content and mathematical derivation	128
10.2.3	Case studio: multi-sensor airspace analogue	128
10.2.4	Whole-chapter synthesis	128
10.2.5	Practice and extension	128
10.3	A layered fusion problem	129
10.4	A simple estimator	129
10.5	Correlation and common-mode error	129
10.6	Interface semantics	130
10.7	Measurement models and association	130
10.8	Prediction and update	131
10.9	Correlation and covariance	131
10.10	Interface semantics and uncertainty display	131
10.11	Historical and methodological interpretation	132
10.12	Fusion architecture and disagreement paths	132
10.13	Sensor disagreement exercise	133
10.14	Historical notes and further reading	133
10.15	Exercises	133

11 Cruise Missiles: Navigation, Guidance, and Replanning	135
11.1 Foundations: coordinate frames, navigation, guidance, and control . .	136
11.1.1 Coordinate frames and transformations	136
11.1.2 Navigation, guidance, and control	137
11.1.3 A minimal kinematic model	137
11.1.4 Tracking error and bounded correction	137
11.1.5 Map metadata and route validity	138
11.1.6 Error budgets and uncertainty	138
11.1.7 Foundation self-check	139
11.1.8 Bridge to the visible road and hidden route	139
11.2 Historical opening and source note	139
11.2.1 Foundational knowledge to master	139
11.2.2 Learning content and mathematical derivation	140
11.2.3 Case studio: 2-D cruise-route analogue	140
11.2.4 Whole-chapter synthesis	140
11.2.5 Practice and extension	140
11.3 Three functions, one motion	141
11.4 Map version and mission version	141
11.5 Replanning as evidence revision	141
11.6 Reference frames and coordinate contracts	142
11.7 Map metadata and purpose	142
11.8 Tracking error and bounded correction	143
11.9 Replanning as evidence revision	143
11.10 Historical and methodological interpretation	144
11.11 Coordinate transformation exercise	144
11.12 Replanning and rejected alternatives	144
11.13 Historical notes and further reading	145
11.14 Exercises	145
12 Ballistic Missiles: Strategic Systems and Decision Responsibility	146
12.1 Foundations: requirements, safety cases, and assigned decisions	147
12.1.1 Requirements, constraints, and claims	147
12.1.2 Argument structure	147
12.1.3 Responsibility matrices	148
12.1.4 Decision records and auditability	148
12.1.5 Uncertainty, time, and safe states	149

12.1.6 Assurance cases must be challenged	149
12.1.7 Foundation self-check	150
12.1.8 Bridge to the loss of signal meaning	150
12.2 Historical opening and source note	151
12.2.1 Foundational knowledge to master	151
12.2.2 Learning content and mathematical derivation	151
12.2.3 Case studio: strategic authorization and safety-case simulator .	152
12.2.4 Whole-chapter synthesis	152
12.2.5 Practice and extension	152
12.3 A warning is not an order	153
12.4 Responsibility is not a single button	153
12.5 Safety cases	153
12.6 Warning, confirmation, authorization, and response	154
12.7 Safety cases and structured arguments	154
12.8 Responsibility matrix	155
12.9 Decision record and auditability	155
12.10 Trust as a system variable	156
12.11 Course synthesis exercise	156
12.12 Historical and methodological interpretation	156
12.13 Challenges to a safety case	157
12.14 Safety-case exercise	157
12.15 Historical notes and further reading	157
12.16 Exercises	157
A Mathematical Foundations	159
A.1 Vectors, matrices, and state descriptions	159
A.2 Probability and conditional reasoning	159
A.3 Optimization and constraints	159
A.4 A reading rule for equations	159
A.5 Norms, distances, and uncertainty	160
A.6 Linear algebra for interfaces	160
A.7 Conditional probability and likelihood ratios	161
A.8 Optimization and the feasible set	161
A.9 Graphs and search	161
A.10 Difference equations and stability intuition	162
A.11 Practice problems	162

B	Algorithmic Notation and Pseudocode	163
B.1	A minimal pseudocode convention	163
B.2	Completeness, correctness, and explainability	163
B.3	Algorithm contracts	163
B.4	Complexity and resource budgets	164
B.5	Testing levels	164
B.6	Explainability is a record property	165
B.7	Algorithm review questions	165
C	Source Limits and Reading Protocol	166
C.1	Material labels	166
C.2	Source classes and citation discipline	166
C.3	The four-sentence source check	167
C.4	Reading a translation	167
C.5	A student evidence card	167
C.6	The four-sentence source check	167
C.7	A student evidence card	168
C.8	Evidence-card workflow	168
C.9	Review protocol for course projects	168
D	Case Studio: Twelve Observation-to-Decision Exercises	169
D.1	Case 1: The changed trace	169
D.2	Case 2: Three platforms, four tasks	169
D.3	Case 3: Waiting is a state	169
D.4	Case 4: The open corridor	170
D.5	Case 5: The confident label	170
D.6	Case 6: The system that was turned off	170
D.7	Case 7: The shrinking window	170
D.8	Case 8: The unanswered message	171
D.9	Case 9: Three documents, one source	171
D.10	Case 10: Correlated agreement	171
D.11	Case 11: Three maps	171
D.12	Case 12: The warning chain	172
D.13	Instructor synthesis rubric	172
E	Study Guide, Laboratory Templates, and Review Sheets	173

E.1	A twelve-week reading rhythm	173
E.2	Laboratory 1: system boundary map	173
E.3	Laboratory 2: allocation matrix	173
E.4	Laboratory 3: state-machine test	173
E.5	Laboratory 4: map and feedback	174
E.6	Laboratory 5: calibration table	174
E.7	Laboratory 6: lifecycle gate review	174
E.8	Laboratory 7: threshold and time margin	174
E.9	Laboratory 8: message audit	174
E.10	Laboratory 9: provenance graph	174
E.11	Laboratory 10: fusion note	175
E.12	Laboratory 11: navigation metadata	175
E.13	Laboratory 12: safety case	175
E.14	Final examination review sheet	175
E.15	Project report template	176
F	Glossary of Core Terms	177
G	Extended Worked Examples	179
G.1	Boundary completeness	179
G.2	Assignment with an inadmissible pairing	179
G.3	A state-machine invariant	179
G.4	Grid planning and unknown cells	180
G.5	Base-rate calculation	180
G.6	Lifecycle residual risk	180
G.7	Time margin	180
G.8	Queue evolution	180
G.9	Provenance graph counting	181
G.10	Correlated sensor fusion	181
G.11	Tracking error	181
G.12	Safety-case decomposition	181
G.13	Example interpretation sheet	181
H	Annotated Reading Dossier	183
H.1	How to read the historical sources	183
H.1.1	Sima Qian and the Maling narrative	183

H.1.2	Tian Ji’s horse race	183
H.1.3	Cao Gui and the drumbeats	183
H.1.4	Cook Ding	183
H.1.5	The rescue of Zhao	184
H.1.6	The old man and the horse	184
H.1.7	Bian Que and Duke Huan	184
H.1.8	Flood-control traditions	184
H.1.9	Three people make a tiger	184
H.1.10	Borrowed arrows	184
H.1.11	The plank road and the hidden route	184
H.1.12	Beacon fires	185
H.2	Technical reading route	185
H.3	Reading notes template	185
H.4	Citation and adaptation note	185
Bibliography		186

Part I

Systems, Platforms, and Feedback

Chapter 1

Intelligent Weapon Systems: Concepts, Architecture, and Evidence

Narrative Prelude — The Torch at Maling. The historical narrative of Maling begins with a strategic deception rather than with a machine. Sun Bin changes the visible traces of the Qi army; Pang Juan follows the evidence he believes he understands; at the narrow road, a small sign becomes the gateway to a much larger decision. The story is used here to ask a systems question: before a system can be called intelligent, what exactly belongs inside its boundary?

Source note. The core narrative is found in Sima Qian's *Shiji*, "Sunzi Wuqi Biography" (H). Later details about the two men as schoolmates, their private conversations, weather, and inner thoughts are narrative reconstruction (R). The system diagrams and data values in this chapter are classroom examples (C).

Learning objectives

1. define a safety-critical intelligent system in terms of platform, sensing, information, decision, action, communication, and assurance;
2. distinguish an observation from an interpretation, a command from an authorization, and an action from its recorded outcome;
3. draw a documented functional architecture and identify missing interfaces;
4. formulate a simple feedback model with explicit uncertainty and constraints;
5. explain why a system boundary is an engineering decision with ethical consequences.

Before reading this chapter, review four elementary ideas. A **system** is a set of interacting parts considered together with its purpose and environment. A **model** is a simplified representation used to answer a defined question. A **state** records the variables needed for the next decision, while an **input** changes the state and an **output** reports what the system knows or does. In a feedback sys-

tem, outputs are observed again and used to update later inputs. NASA’s public systems-engineering handbook presents the same basic discipline through stakeholder needs, requirements, architecture, verification, and lifecycle thinking [16]. NIST’s public AI Risk Management Framework adds a complementary point: risk management includes the technical model and the wider human context [14].

1.1 Foundations: systems, models, and feedback

This chapter uses the word “system” in its engineering sense. A system is a selected collection of components, relations, interfaces, and constraints considered for a stated purpose. The selection matters: a sensor may be inside the technical product boundary, while its calibration procedure, an operator’s interpretation, a communications service, and a rule set may be outside the product but inside the operational system. NASA describes systems engineering as an interdisciplinary approach to realizing successful systems across their life cycle [16]. The first engineering action is to state the question that the boundary must help answer, then draw the boundary around the relevant elements.

1.1.1 From a real situation to a model

Any model is deliberately incomplete. A useful model retains the variables that can change the decision being studied and omits variables whose influence is negligible for that question. Let the real situation be $\mathcal{R}$ and the model be $\mathcal{M}$. A modeling map is written informally as

$$\Phi : \mathcal{R} \longrightarrow \mathcal{M}.$$

The map Φ is not neutral. It chooses a time scale, a coordinate system, a set of allowed actions, and an error description. A model of a laboratory mobile platform may keep position, energy reserve, and communication state; a model of a report-review workflow may keep source, timestamp, version, and reviewer. Neither model is “the system.” Each is a statement about which distinctions matter.

A **boundary** separates the elements a model treats as internal from those it treats as environmental. An **interface** is a declared relation through which information, energy, material, authority, or responsibility crosses that boundary. An interface contract should name at least the data or resource, unit, frame, time, version, validity condition, and owner.

The elementary context diagram has a system S , an environment E , and interfaces $I_1, \dots, I_r$. It is often more useful than a long component list because it makes a missing relationship visible. For example, a detector can report a numerical score, but unless the interface specifies the input population, timestamp, confidence semantics, and user of the score, the receiving function cannot know what decision the number may support.

1.1.2 State, input, output, and disturbance

For a discrete-time system, a compact starting point is

$$x_{k+1} = f(x_k, u_k, d_k), \quad (1.1)$$

$$y_k = h(x_k, u_k, n_k), \quad (1.2)$$

where x_k is the **state**, u_k is an input, d_k is a disturbance, y_k is an output, and n_k represents measurement noise. The state is not every fact about the world; it is the information the model needs to predict the next relevant state. An output is what the system exposes or measures, not necessarily the state itself.

For a linear approximation near a chosen operating condition, the notation becomes

$$x_{k+1} = Ax_k + Bu_k + Gw_k, \quad z_k = Hx_k + v_k.$$

The matrices express dependencies. A describes nominal state evolution, B describes the effect of an input, G maps process uncertainty w_k , and H maps state to an observation z_k . The equation is valuable even before values are known because it forces four questions: What must be estimated? What can be observed? What can be changed? What uncertainty remains?

Worked example: a simple evidence loop

Consider a fictional indoor inspection platform. Let $x_k = [b_k \ q_k]^T$, where b_k is remaining battery fraction and q_k is link-quality index. A coarse update model is

$$x_{k+1} = \begin{bmatrix} 1 & 0 \\ 0 & 0.9 \end{bmatrix} x_k + \begin{bmatrix} -0.04 \\ 0 \end{bmatrix} r_k + \begin{bmatrix} w_k^b \\ w_k^q \end{bmatrix},$$

where $r_k = 1$ means one nominal work interval. If $b_k = 0.60$ and $q_k = 0.80$, the nominal next state is $[0.56 \ 0.72]^T$ before uncertainty. The calculation does *not* decide what the platform should do. It only makes a premise visible: if a task is continued, energy decreases and link quality may decay. A policy must separately specify reserve limits, observation quality, and a safe state. [Illustrative teaching value]

1.1.3 Feedback changes the meaning of an action

An open-loop plan applies inputs without using later observations. A feedback loop observes the result, estimates the new state, and revises the next input:

$$z_k \rightarrow \hat{x}_k \rightarrow \pi(\hat{x}_k, \mathcal{C}) \rightarrow u_k \rightarrow x_{k+1} \rightarrow z_{k+1}.$$

Here $\hat{x}_k$ is an estimate, π is a decision or control policy, and $\mathcal{C}$ is the declared constraint set. Feedback is not automatically wise. If the observation is stale, biased, misframed, or interpreted outside its scope, fast feedback can repeat a mistake faster. A responsible loop therefore carries uncertainty, validity conditions, and the right to decline a transition.

Object	Question it answers	A common mistake
State x_k	What must be known for the next update?	storing only what is easy to measure
Observation z_k	What evidence is currently available?	treating the observation as the world state
Estimate $\hat{x}_k$	What does the model infer from evidence?	hiding its uncertainty or age
Policy π	Which option is considered under constraints?	treating a recommendation as permission
Record ρ_k	How can the transition be re-constructed?	retaining only the final output

1.1.4 Requirements, measures, and verification

A requirement should be testable. “The system shall be intelligent” cannot be verified because it has no observable criterion. In contrast, a fictional requirement such as “The system shall enter REVIEW when input age exceeds the declared validity window” identifies a condition, an action, and a test. Requirements are often organized from stakeholder need to system requirement, subsystem requirement, interface requirement, and verification evidence. This structure prevents a performance number from silently becoming a mission permission.

Verification asks whether a stated requirement was met. Validation asks whether the requirement addressed the intended need in its context. A sensor may be verified to report values within its laboratory specification, while the system remains invalid for a poorly defined task. NIST’s AI Risk Management Framework similarly separates the context of use, measurement, management, and responsibility for risks [14].

- A boundary that excludes the human authority or rule source makes a consequential decision appear to have no owner.
- A model that contains a score but not its uncertainty, source, or time turns evidence into an unsupported fact.
- An interface that names a field but not its unit, frame, version, and expiry condition is not yet an engineering contract.
- A requirement that cannot be verified is usually a hope, a slogan, or a missing design decision.

1.1.5 Foundation self-check

1. Give one example of an element outside a product boundary but inside the operational system.
2. In $x_{k+1} = Ax_k + Bu_k + Gw_k$, which term represents model uncertainty and which term represents a deliberate input?
3. Explain why an observation can be accurate while an interpretation of it is wrong.
4. Write one verifiable requirement for a fictional status-monitoring function.

The expected reasoning is: name the boundary, preserve the distinction between observation and estimate, and state the evidence needed to check the requirement. If an answer only says that the system should “make a good decision,” the model is still underspecified.

1.1.6 Bridge to the Maling system question

The Maling narrative begins with an observable trace and ends with a consequential organizational decision. The technical reading begins in between: define the relevant environment, record the observation and its source, distinguish it from the state estimate, identify the policy and constraints that use the estimate, and preserve the record of the resulting transition. The rest of this chapter applies that basic system vocabulary to an evidence-to-decision architecture.

1.2 Historical opening and source note

The first lecture opens with the Maling narrative and the question hidden inside the phrase “know the other and know oneself.” The instructor should pause before explaining the historical outcome and ask: if three observers report three different situ-

ations, what exactly does “our side knows” mean? The story introduces the course’s central distinction: an observation is not an interpretation, an interpretation is not an authorization, and an authorization is not an outcome.

1.2.1 Foundational knowledge to master

Students first need the difference among automation, autonomy, and intelligence. **Automation** executes a predefined relation between input and output. **Autonomy** selects or adapts behavior within stated limits when all information is not centrally supplied. **Intelligence** is used here as a functional description of sensing, representation, inference, adaptation, and decision aid; it is not a claim that a machine possesses human judgment.

The minimum system vocabulary is

$$\mathcal{S} = (E, x, z, u, a, \kappa, \mathcal{C}, \rho),$$

where E is the environment, x the state, z an observation, u a recommendation or control input, a an authorization state, κ a configuration, $\mathcal{C}$ constraints, and ρ the record. The point of the tuple is pedagogical: removing a variable removes a question. If a is omitted, the model cannot distinguish a recommendation from permission. If ρ is omitted, it cannot support later review.

1.2.2 Learning content and mathematical derivation

The basic feedback model is

$$z_k = h(x_k, E, \kappa) + v_k, \quad (1.3)$$

$$\hat{x}_k = \mathcal{E}(z_{0:k}, \kappa), \quad (1.4)$$

$$u_k = \pi(\hat{x}_k, G, \mathcal{C}, \kappa), \quad (1.5)$$

$$x_{k+1} = f(x_k, u_k, E, \kappa) + w_k. \quad (1.6)$$

Students should derive the following error chain in words: measurement error changes the estimate; estimation error changes the recommendation; a constraint or authority check may block the recommendation; the action changes the environment; the next observation tests the previous model. A score without this chain is not a system explanation.

For a small classroom architecture, define the evidence completeness indicator

$$\chi(\rho) = \prod_{j=1}^m I_j,$$

where $I_j = 1$ if a required field is present and valid, and $I_j = 0$ otherwise. The product is intentionally strict: one missing field makes the record incomplete. A softer completeness score $\bar{\chi} = m^{-1} \sum_j I_j$ may rank records for review, but it must not be mistaken for authorization.

1.2.3 Case studio: the evidence-card simulator

Students build a safe evidence-card simulator in a spreadsheet or Python notebook. Each row contains source, timestamp, observation, estimate, confidence, configuration, constraint result, authority state, selected safe state, and owner. The simulator should inject three disturbances: a stale map, a conflicting observation, and an expired authority token. The required output is a state transition such as REQUEST_REVIEW or SAFE_HOLD, never a real-world engagement action.

The assessment has four parts:

1. draw the system boundary and identify what is outside it;
2. write the state update and the missing-field condition;
3. run the three disturbances and record the resulting state;
4. explain which conclusion is supported, which is conditional, and which is rejected.

No physical board is necessary for this first lecture. A 嘉立创 EDA exercise may be introduced later by designing a small “evidence status” board with LEDs, a button, a clock module, and a serial interface, but the board is a teaching interface rather than a weapon component.

1.2.4 Whole-chapter synthesis

The first lecture should end with the following chain on the board:

environment → sensing → estimation → decision → authorization,
limited action → record.

Every later lecture instantiates this chain with a different platform or evidence problem. Students should understand that the platform changes, but the responsibility questions persist.

1.2.5 Practice and extension

1. Draw a complete feedback architecture and label one uncertainty at every arrow.
2. Explain why a correct classifier output can still lead to an incorrect system decision.
3. Construct a seven-field evidence record for a fictional inspection task.
4. Compare “human in the loop” with “human able to understand and override the loop.”
5. Write a one-paragraph boundary statement for a fictional classroom system.

Basic idea	Symbol in this chapter	Why it matters here
Environment	E	defines what is outside and what can disturb the system
State	x_k	identifies what must be estimated before a decision
Observation	z_k	separates measured evidence from interpretation
Input	u_k	distinguishes a recommendation or command from an outcome
Uncertainty	v_k, w_k, P_k	prevents a conditional estimate from appearing exact

1.3 The story as a systems problem

The Maling narrative is often told as if the decisive moment were the final ambush. A systems engineer reads the story earlier. Someone changes an observable trace; another person interprets it; an organization changes its movement; the final result emerges from the interaction of sensing, belief, communication, and action. The question is not whether a tree, a fire, or a written sign is “intelligent.” The question is how a chain of functions converts an observation into a decision.

Imagine the report arriving at the commander’s camp. It is not yet a fact about the whole battlefield. It is a piece of evidence with a source, a time, an observation process, and a possible explanation. The report might be accurate as a statement about what was seen and still be misleading as a statement about what the opposing army intended. The gap between those two claims is the gap between measurement and inference.

The story therefore gives us a useful sequence:

1. a physical or social environment produces observable traces;
2. an observer selects some traces and omits others;
3. a receiver assigns meaning to the selected traces;
4. an organization combines the interpretation with a goal, a rule, and a time limit;
5. an action changes the environment and creates new traces.

The sequence creates feedback. An isolated question such as “How accurate is the detector?” is then insufficient. We must also ask what the detector may influence, what uncertainty survives the interface, and who is responsible after the detector has spoken.

For this textbook, an intelligent weapon system is a technical and human system that acquires information about an environment, transforms information into an internal state estimate, selects or recommends actions under declared constraints, and executes or supports execution while remaining connected to human authority, assurance, and assigned responsibility. The definition is deliberately broader than a platform or an algorithm.

The technical discussions in this book are restricted to architecture, estimation, planning, control, verification, risk, communication, and governance. They do not provide real-world target selection, deployment, evasion, or engagement procedures.

1.4 Problem setup and modeling assumptions

Let the environment be E , mission intent be G , system state be x_k , observation be z_k , control or recommendation be u_k , human authorization be a_k , and configuration be κ . A discrete feedback model can be written as

$$z_k = h(x_k, E, \kappa) + v_k, \quad (1.7)$$

$$x_{k+1} = f(x_k, u_k, E, \kappa) + w_k, \quad (1.8)$$

$$u_k = \pi(\hat{x}_k, G, \mathcal{C}, \kappa), \quad (1.9)$$

where v_k and w_k represent measurement and model uncertainty, and $\mathcal{C}$ collects hard and soft constraints. The equation is a bookkeeping device: it forces us to say where the observation enters, where uncertainty is represented, and where the mission objective is used.

The subscript k is a time index, not a claim that all system functions are synchronous. A camera, a communication link, an estimator, and a human supervisor may operate at different rates. If a decision at time k uses a map from time $k - 20$, that age is part of the evidence. If an authorization was issued for configuration κ_0 but the system is running κ_1 , the record is incomplete even if the displayed state looks familiar.

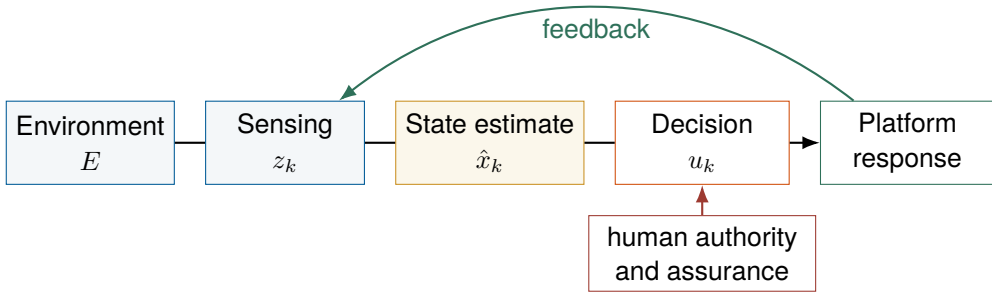


Figure 1.1: A system boundary must show both the technical loop and the authority loop.

1.4.1 What is inside the boundary?

A practical boundary often extends beyond the physical outline of the vehicle. It may include a remote operator, a mission database, a timing service, a maintenance procedure, and an organization that approves changes. A boundary drawn around only the processor can make human approval appear to be external noise, although that approval may be a necessary condition for action.

Use the following boundary questions before drawing an architecture:

1. What can the system observe directly, and what arrives through another system?
2. Which variables are measured, which are estimated, and which are merely assumed?
3. Which decisions are recommended, which are authorized, and which are executed?
4. Which failure modes are handled by the system, and which are handed to a human?
5. Which records must survive a restart, a link outage, a software update, or a dispute?

A system boundary is a declared set of components, interfaces, authorities, assumptions, and records included in the analysis of a function. It is valid only when the analyst also states what has been excluded and how excluded conditions can affect the result.

1.5 A functional decomposition

An architecture becomes inspectable when vague verbs are replaced by functions. “The system understands the scene” can be decomposed into sensing, calibration,

preprocessing, detection, association, classification, state estimation, planning, authorization, control, actuation, and logging. Each function has inputs, outputs, assumptions, timing, and failure behavior.

Function	Typical output	Question that must remain visible
Sensing	timestamped measurement z_k	What was measured, with what quality and age?
Preprocessing	calibrated signal	Which transformations changed the data?
Estimation	state and covariance $(\hat{x}_k, P_k)$	What uncertainty remains?
Classification	label and confidence	What population and threshold define confidence?
Planning	candidate options	Which constraints make an option feasible?
Authorization	bounded permission	Who has authority, and for which configuration?
Control	command or reference	What feedback and saturation limits apply?
Logging	evidence record	Can an independent reviewer reconstruct the decision?

The table repeats questions on purpose. Under pressure, a designer should not have to rely on intuition to remember that a confidence score is not an authorization, and that a command is not proof of execution.

1.6 Four evidence layers

The same event can appear in at least four forms: raw observation, processed information, an interpreted state, and an authorized action. A screen label may be correct while the action inferred from it is not authorized. Conversely, a human may have a valid authorization while the system state is too stale to support action. A documented design stores the relation between layers rather than retaining only the final label.

Layer	Example statement	Typical error
Observation	"A return was recorded at 10:31:04."	timestamp, calibration, or sensor artifact
Information	"The return is associated with track 7."	incorrect association or duplicate counting
Interpretation	"Track 7 may be a moving vehicle."	model mismatch, base-rate neglect, or stale data
Authorization	"Request a second observation."	authority, scope, version, or interface ambiguity
Outcome	"The request was sent and acknowledged."	confusing intent with execution

Decision-record rule

For every consequential decision, record what was observed, how it was transformed, what assumptions were used, which constraints were checked, who authorized the next step, and what actually happened. A record of only the final decision is an outcome log, not an evidence chain.

1.6.1 A minimal decision record

Let a decision record be

$$r_k = (t_k, z_{k_0:k}, \hat{x}_k, P_k, \kappa_k, \mathcal{C}_k, a_k, u_k, y_k),$$

where $z_{k_0:k}$ is the relevant observation window, P_k describes uncertainty, κ_k is the configuration, $\mathcal{C}_k$ is the constraint result, a_k is the authority state, u_k is the selected command or recommendation, and y_k is the observed outcome. The exact fields depend on the application, but their relationships should remain visible.

Worked example: a fictional inspection exercise

Consider a fictional monitoring node in a disaster-response exercise. Its camera reports a moving object; a detector labels it "vehicle-like"; a fusion module reports "route obstruction possible"; the supervisor authorizes "request a second observation." These are four distinct statements. Replacing the last one with "intercept" would be an unjustified change of action class, not a harmless paraphrase.

Suppose the observation arrives at $t = 12$ s, the detector is version 3.2, the map is 18 minutes old, and the confidence score is 0.68. A responsible record does not merely say "confidence 68 percent." It states the reference population, the

threshold used, the map age, the unresolved alternatives, and the fact that the authorized response was information gathering. [Illustrative teaching value]

1.7 From observation to decision

The transformation from z_k to u_k can be represented as a sequence of conditional claims:

$$z_k \Rightarrow \mathcal{I}_k \quad \text{the measurement passed integrity checks,} \quad (1.10)$$

$$\mathcal{I}_k \Rightarrow \hat{x}_k \quad \text{the state estimate is within scope,} \quad (1.11)$$

$$\hat{x}_k \Rightarrow \mathcal{O}_k \quad \text{candidate options satisfy hard constraints,} \quad (1.12)$$

$$\mathcal{O}_k \Rightarrow u_k \quad \text{an option is recommended or authorized.} \quad (1.13)$$

The arrows are not logical certainties. They denote an argument whose assumptions require review. If one arrow lacks support, the downstream output must be marked as conditional. This is the technical meaning of “knowing the system before trusting its decision.”

1.7.1 Confidence is conditional

If H denotes a hypothesis and Z the measurement, then

$$P(H | Z) = \frac{P(Z | H)P(H)}{P(Z)}.$$

The posterior probability depends on the prior $P(H)$ and on the measurement model $P(Z | H)$. It is therefore incomplete to move a number such as 0.90 from one environment to another without checking whether the population, sensor, time scale, and operating conditions changed. A confidence number is a statement about a model, not a property that travels independently of context.

Evidence quality is the degree to which a claim can be traced to a defined source, a known method, a stated condition, an identified uncertainty, and an independently checkable record. Repetition, visual polish, and organizational rank are not substitutes for these properties.

1.8 Assumptions, interfaces, and failure modes

Every model is a compressed view of a larger world. The compression is useful only when the omitted variables are either irrelevant to the current decision or covered by

a conservative boundary. A state model that omits communication quality may work in a laboratory and fail in the field; a classifier that omits time of day may silently inherit a lighting bias; a planner that omits stopping distance may report a path that is geometrically valid but dynamically unavailable.

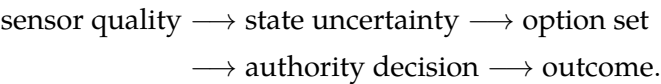
Assumption	What it permits	How to challenge it
Sensor timestamps are aligned	compare measurements in one update	inject a known time offset
Map is current	use map obstacles as constraints	replay with stale and changed maps
Human can intervene	delegate a bounded decision	test workload and communication delay
Classifier confidence is calibrated	compare scores to thresholds	evaluate reliability by score bin
Log is complete	reconstruct a decision	remove one field and attempt an audit

For each interface, write the input type, output type, timestamp rule, version identifier, failure indication, and owner. If an interface has no named owner, it is not yet an engineered interface; it is an assumption waiting to fail.

1.8.1 Failure propagation

Failure does not always travel as a visible error. A missing timestamp may be interpreted as a default value. A stale map may be accepted because no alarm is defined for age. An authorization message may be displayed as sent even though acknowledgment was never received. These are semantic failures: the system continues to operate while the meaning of its records has changed.

One way to inspect propagation is to build a dependency chain:



For each arrow, ask whether uncertainty is propagated, suppressed, or converted into a binary status. Suppression may simplify the interface but increases the risk that a downstream user treats a conditional result as a fact.

1.9 A small architecture exercise

Assume a fictional ground platform has a camera, a range sensor, an inertial unit, a local processor, a remote supervisor, and a record server. The platform is used only to inspect marked locations in a closed teaching environment. Draw the architecture in three passes.

1.9.1 Pass 1: physical components

Place the sensors, processor, communication link, supervisor console, and record server. Use arrows for data flow and annotate each arrow with a rate or event condition. Do not draw a single arrow labelled “information”; distinguish measurements, estimates, recommendations, authorizations, acknowledgments, and outcomes.

1.9.2 Pass 2: logical functions

Place calibration, detection, state estimation, map update, path recommendation, constraint checking, human approval, control, and logging. A physical processor may host several functions, but co-location does not erase their interfaces. The point of the diagram is to make assumptions visible.

1.9.3 Pass 3: authority and recovery

Add the conditions for degraded operation, safe hold, review, and termination. Draw the record generated when a link fails, when the operator rejects a recommendation, and when the platform enters a safe state. A system is not complete when its normal path is drawn; it is complete when its abnormal paths have owners.

[title=Evidence-aware decision loop]

```
while mission_window_is_open:
    observation <- acquire_measurement()
    integrity <- check_time_quality_and_scope(observation)
    estimate <- update_state(observation, integrity)
    if evidence_is_out_of_scope(estimate):
        state <- REQUEST_REVIEW
    elif hard_constraints_fail(estimate):
        state <- SAFE_HOLD
    else:
        options <- evaluate_bounded_options(estimate)
        state <- await_authorization(options)
    record(observation, estimate, state, owner, version)
```

The pseudocode contains no real-world weapon action. It shows that uncertainty, constraints, authorization, and records belong within the feedback process.

1.10 Historical reading and methodological interpretation

Read Sima Qian’s passage before reading a modern retelling. The source provides a narrative of Sun Bin, Pang Juan, the changed traces, and the decisive encounter. It does not provide a sensor specification, a probability of detection, or an algorithmic performance guarantee. Those are modern questions imposed by the class.

The responsible analogy is therefore narrow: the story helps us discuss evidence, inference, and the consequences of a system boundary. It cannot prove that an ancient commander used a modern decision architecture. The distinction matters because a vivid story can create a false sense of evidence. A good lecturer uses the story to open a question and then marks the point at which engineering analysis begins.

Historical and methodological note

The Maling episode is a historical narrative with layers of later retelling. In class, separate the textual core, later literary detail, and classroom reconstruction. The narrative is valuable precisely when students can enjoy its drama without mistaking drama for measurement. The relevant primary reading is listed in the bibliography as *Shiji*, “Sunzi Wuqi Biography.”

1.11 Chapter laboratory: the evidence card

Prepare one evidence card for the sentence “The opposing force has reduced its strength.” The card must contain:

1. the exact observation that supports the sentence;
2. the source, time, and transformation history;
3. at least two alternative explanations;
4. the decision that the evidence may support;
5. the decision that remains outside the limits of the evidence;
6. the next observation that would most reduce uncertainty.

Students should then exchange cards. The reviewer is not allowed to ask whether the conclusion feels plausible. The reviewer may ask only whether the source, transformation, assumptions, and authorized consequence are explicit. This turns the narrative into a practice in disciplined reasoning.

1.12 Interface contracts and semantic failure

An interface contract states what a function promises to preserve. For an observation interface, the contract may include units, timestamp, coordinate frame, quality

flag, source identity, and configuration. For an authorization interface, it may include scope, validity interval, issuer, confirmation requirement, and expiry behavior. A value can be numerically correct and semantically invalid when one of these fields is missing.

Interface	Contract fields	Semantic failure example
Sensor to estimator	units, frame, time, quality	meters interpreted as centimeters
Estimator to planner	state, covariance, age, scope	stale state presented as current
Planner to authority	options, constraints, rejected cases	one option displayed as a command
Authority to controller	scope, expiry, version, issuer	old permission accepted after update
Controller to log	command, acknowledgment, outcome	intent recorded as execution

1.13 Trace reconstruction exercise

Assume an audit receives only the final statement “the system entered a safe state at 14:06.” Reconstructing the decision requires the preceding observation window, estimator version, map age, guard values, authority state, and the actual state transition event. If one field cannot be recovered, the review should mark the conclusion conditional rather than fill the gap with a plausible story.

The exercise can be formalized as an inverse problem. Given outcome y_k , find a set of records R satisfying

$$\mathcal{A}(R) \Rightarrow y_k,$$

where $\mathcal{A}$ is the documented decision architecture. If many incompatible records satisfy the same outcome, the record does not permit the decision to be reconstructed. A good record makes the inverse problem narrow enough for an independent reviewer to reproduce the argument.

1.14 Chapter design assignment

Write a one-page system boundary statement for a fictional inspection platform. It must include purpose, environment, excluded conditions, sensing, estimation, decision, authority, communication, recovery, end state, and records. Then write a second statement that is deliberately too narrow by omitting the human and lifecycle layers. Compare the conclusions. The assignment demonstrates that a boundary is not neutral: it changes what can be claimed.

1.15 Historical notes and further reading

For the historical opening, begin with Sima Qian and compare a later translation rather than relying on a quotation detached from context. For system architecture and probabilistic state estimation, see [7, 8, 13]. For planning and decision models, see [6, 9]. Appendix C gives the source-limits protocol used throughout the book.

Review questions

1. Why is a sensor measurement not the same thing as a state estimate?
2. Which parts of the system boundary should include human authority even when the physical platform is autonomous?
3. What does the covariance P_k add to the state estimate $\hat{x}_k$?
4. Why can a correct observation still support an incorrect decision?
5. What is the difference between a recommendation, an authorization, and an outcome record?

1.16 Exercises

- 1.1 Draw a seven-layer architecture for an intelligent system: platform, sensing, information processing, decision, control, communication, and assurance. Add one human role and one log record to every layer.
- 1.2 Give one example in which a correct sensor measurement produces an incorrect decision because the task definition is incomplete.
- 1.3 For the model above, identify E , G , z_k , $\hat{x}_k$, u_k , and a_k in a fictional ground-vehicle inspection task.
- 1.4 Write a short evidence card for the Maling narrative. Separate historical core, narrative reconstruction, and engineering interpretation.
- 1.5 Why should a system store the rejected recommendation as well as the selected action? Answer in terms of auditability and later diagnosis.
- 1.6 Construct two system boundaries for the same inspection exercise: one that includes only the platform and one that includes the operator, record server, and maintenance organization. What conclusions change?
- 1.7 Write three assumptions required by a feedback model and propose one test for each assumption.
- 1.8 Design a record schema for a link-loss event. Include time, state, last valid observation, authority status, and recovery action.

- 1.9** Explain how repetition of a claim can increase social confidence without increasing evidential independence.
- 1.10** Use the four evidence layers to analyze one everyday automated decision, such as a spam filter or a navigation recommendation.

Chapter summary

- Intelligence is a property of a system with feedback, defined functions, and human responsibility, not of a single classifier.
- Observation, interpretation, recommendation, authorization, and outcome are different evidence layers.
- A state estimate should carry uncertainty, time, provenance, and configuration.
- Interfaces are part of the system boundary because they can suppress or preserve uncertainty.
- A system boundary determines what the system can know, what it can do, and who must remain responsible.

Chapter 2

Intelligent Military Robots: Platforms, Capability, and Mission Allocation

Narrative Prelude — Tian Ji’s Three Lines. The historical account of Tian Ji’s horse race is short. Sun Bin changes the pairing of three resources and three opponents; the weakest race is accepted as a local loss so that the overall objective becomes attainable. The later dramatic dialogue is reconstruction. The engineering question is precise: when individual platforms do not improve, how can a different assignment improve the performance of the whole system?

Source note. The core is in *Shiji*, with later versions differing in horse names and race details (H plus R). The assignment matrices, capability values, and autonomy labels below are example models (C).

Learning objectives

1. represent a robot or platform by a multidimensional capability vector;
2. formulate a mission-allocation problem using a cost or utility matrix;
3. explain why autonomous level is a task-dependent design variable rather than a ranking;
4. analyze reassignment when communication, energy, or human takeover conditions change.

The mathematics needed here is elementary linear algebra and discrete optimization. A scalar is one number; a vector is an ordered list of numbers; a matrix is a rectangular arrangement that can represent data, a transformation, or a cost table. If $\mathbf{c}_i$ is a capability vector, its coordinates must have defined units or normalized meanings before they are compared. MIT OpenCourseWare’s public Linear Algebra course provides review material on vectors, matrices, linear combinations, and linear maps [17]. The assignment problem then adds a discrete idea: a binary variable records whether a platform–task pairing is selected, while constraints remove impossible pairings.

2.1 Foundations: vectors, comparison, and constrained allocation

Mission allocation uses a simple mathematical idea with an important discipline: no platform is “best” without a task, an environment, a constraint set, and a measure of comparison. This module introduces the vector and matrix notation needed to represent those qualifications. It uses a fictional inspection and logistics setting only; the purpose is to learn how an allocation model for the whole system is constructed.

2.1.1 Scalars, vectors, and matrices

A scalar is a single number, such as an available energy fraction $e = 0.72$. A vector is an ordered set of numbers, such as a capability description

$$\mathbf{c}_i = \begin{bmatrix} m_i \\ s_i \\ e_i \\ ell_i \end{bmatrix},$$

where the entries might refer to mobility suitability, sensing suitability, energy reserve, and link availability. A matrix is a rectangular table. The i th row of a capability matrix can describe platform i ; the j th column can describe task criterion j . MIT’s public linear-algebra materials give the underlying rules for dimensions, matrix products, and linear maps [17].

The order of coordinates is a contract. A vector $[0.8, 0.6, 0.9]^\top$ has no engineering meaning unless the reader knows the coordinate names, units, directions of preference, reference conditions, and time. Two vectors of the same numerical length can be incomparable when one is measured at low temperature and another at nominal temperature, or when one score is a probability and the other is a normalized rating.

A **feature** is a measured or estimated attribute of an object. A **criterion** is a rule for judging an option in relation to a task. Features may become criteria after a direction of preference, reference condition, and weight or threshold are declared. A feature is not a decision by itself.

2.1.2 Normalization is a modeling choice

Different units cannot be added directly. A distance in meters, a delay in seconds, and a confidence fraction need a declared transformation before comparison. For a benefit-type attribute r_i over a reference range $[r_{\min}, r_{\max}]$, min–max normalization is

$$\bar{r}_i = \frac{r_i - r_{\min}}{r_{\max} - r_{\min}}.$$

For a burden-type attribute, where smaller is preferred, one may use

$$\bar{r}_i = \frac{r_{\max} - r_i}{r_{\max} - r_{\min}}.$$

Both formulas yield a dimensionless number only relative to the stated range. The range is not an innocent technicality: an outlier, a changed operating condition, or an obsolete maximum changes the comparison.

If weights $w_j \geq 0$ sum to one, a simple aggregate score is $s_i = \sum_j w_j \bar{r}_{ij}$. This is convenient but conditional. A weighted sum permits compensation: a high score in one coordinate can offset a low score in another. A hard safety or authorization condition should instead be represented as a constraint, not hidden as a small weight.

Worked example: what normalization does and does not say

Two fictional platforms have energy reserves 0.75 and 0.55, and their observed report delays are 8 s and 4 s. Over a declared teaching range of energy [0.40, 0.90] and delay [0, 12] s, their normalized vectors are

$$\bar{\mathbf{c}}_A = \begin{bmatrix} 0.70 \\ 0.33 \end{bmatrix}, \quad \bar{\mathbf{c}}_B = \begin{bmatrix} 0.30 \\ 0.67 \end{bmatrix}.$$

With equal weights both aggregate to 0.515. The correct conclusion is not that the platforms are identical. It is that this particular two-criterion, equal-weight score cannot distinguish them. The task must state whether reserve, delay, observability, or a hard limit is decisive. [Illustrative teaching value]

2.1.3 From a table to an allocation problem

Let R_i denote platform i and T_j denote task j . Define a binary decision variable

$$x_{ij} = \begin{cases} 1, & \text{if } R_i \text{ is assigned to } T_j, \\ 0, & \text{otherwise.} \end{cases}$$

If $C = [c_{ij}]$ is a cost matrix, a basic one-to-one assignment problem is

$$\min_{x_{ij}} \sum_i \sum_j c_{ij} x_{ij}, \tag{2.1}$$

$$\text{subject to } \sum_j x_{ij} \leq 1 \quad \text{for every platform } i, \tag{2.2}$$

$$\sum_i x_{ij} = 1 \quad \text{for every required task } j, \tag{2.3}$$

$$x_{ij} \in \{0, 1\}. \tag{2.4}$$

The first constraint prevents double booking. The second says each required task has one assigned platform. The third expresses the discrete nature of an assignment. More realistic classroom models add task windows, energy reserves, communication availability, operator workload, and a value for deferring a task.

2.1.4 Feasibility precedes optimization

Suppose $a_{ij} \in \{0, 1\}$ records whether a pairing is admissible. Then $x_{ij} \leq a_{ij}$ removes prohibited pairings even if they have low numerical cost. The feasible set is

$$\mathcal{F} = \{X \mid x_{ij} \leq a_{ij}, \text{ resource and authority constraints hold}\}.$$

Optimization answers “Which option is preferred among feasible options?” It does not answer “Which prohibited option is least bad?” When $\mathcal{F}$ is empty, the model’s truthful output is infeasible. That output should prompt a revised task, added resources, or a safe state, not a silent violation of constraints.

[title=Human-readable allocation review]

```
define each task, completion condition, time window, and owner
record platform capability data with units, age, and uncertainty
remove pairings that violate hard constraints or task authority
compute cost or utility only for the remaining pairings
compare feasible allocations and inspect sensitivity to weights
record the selected allocation, unassigned tasks, assumptions, and review time
```

2.1.5 A small enumeration example

For three platforms and three tasks, there are $3! = 6$ complete one-to-one assignments. A classroom solution may enumerate all six rather than invoke an opaque solver. Suppose a cost matrix is

$$C = \begin{bmatrix} 4 & 8 & 5 \\ 7 & 3 & 6 \\ 6 & 5 & 4 \end{bmatrix}.$$

The diagonal allocation has cost $4 + 3 + 4 = 11$. But if pairing (R_2, T_2) becomes unavailable because its required link is not present, the diagonal is infeasible, regardless of its small sum. Students should then compare only the remaining permutations and state whether a task may be deferred. Larger instances can be solved by specialized methods such as the Hungarian algorithm, but the modeling contract remains more important than the solver name.

- Averaging attributes without stating units, normalization range, and task conditions.
- Treating a capability score as an authorization to perform a task.
- Choosing weights after seeing the desired allocation and calling the result objective.
- Ignoring uncertainty and time: an old, precise-looking number may be less useful than a current bounded estimate.
- Optimizing a complete assignment when the honest answer is that one task must remain unassigned.

2.1.6 Foundation self-check

1. State the units and preference direction of each coordinate in a capability vector.
2. Normalize a benefit value 7 over $[5, 9]$ and a burden value 3 over $[0, 6]$.
3. Why should an unavailable platform–task pairing be removed before computing a minimum-cost assignment?
4. Give one reason a high average capability score may be irrelevant to a particular task.

The key is to distinguish a comparison score from a feasibility rule. A weighted sum ranks feasible options; it does not authorize an infeasible one.

2.1.7 Bridge to Tian Ji's three lines

The lesson taken from the Tian Ji narrative is not an operational maxim. It is a systems observation: aggregate performance depends on the matching of heterogeneous resources to heterogeneous tasks. The chapter now uses capability vectors, feasibility constraints, and transparent allocation logic to show why a new arrangement can change system performance without claiming that any single unit has changed.

2.2 Historical opening and source note

The lecture uses Tian Ji's three lines to show that system performance can change when heterogeneous resources are assigned differently. The story is not presented as a battlefield recipe. It is a compact introduction to matching: a platform is evaluated relative to a task, an environment, an information condition, and an authority boundary.

2.2.1 Foundational knowledge to master

A military robot is treated here as a platform that senses, computes, communicates, and acts in a declared environment under human-set objectives. The main categories used for undergraduate comparison are ground robots, aerial robots, underwater robots, stationary robotic subsystems, and multi-robot teams. The category is not the learning endpoint. Students must identify the platform, mobility, sensor suite, mission payload, power source, communication method, human relationship, and recovery state.

The generic architecture is

platform $\rightarrow$ sensors $\rightarrow$ state estimate $\rightarrow$ task allocation $\rightarrow$ controller $\rightarrow$ actuators, with communication, energy, safety, and human supervision crossing every layer. A robot with more sensors is not automatically more autonomous; it may simply produce more unresolved data.

2.2.2 Learning content and mathematical derivation

Let platform i have capability vector

$$\mathbf{c}_i = (m_i, s_i, e_i, \ell_i, r_i)^\top,$$

for mobility suitability, sensing suitability, energy margin, link availability, and recoverability. Let task j require vector $\mathbf{q}_j$. A simple mismatch cost is

$$d_{ij} = (\mathbf{c}_i - \mathbf{q}_j)^\top W (\mathbf{c}_i - \mathbf{q}_j),$$

where $W \succeq 0$ sets the relative importance of dimensions. The derivation is a weighted quadratic form: normalize each coordinate, compute the error, weight it, and sum the squared error. The matrix should be explained, not hidden.

Introduce $x_{ij} \in \{0, 1\}$ for assignment. The constrained allocation problem is

$$\min_X \sum_{i,j} d_{ij} x_{ij}, \quad \sum_j x_{ij} \leq 1, \quad \sum_i x_{ij} \leq 1, \quad x_{ij} \leq a_{ij},$$

where a_{ij} is the admissibility matrix. The constraints are checked before the score. A high capability score cannot make an unauthorized pairing valid.

2.2.3 Case studio: a safe multi-robot allocation board

Students create a three-platform, four-task allocation simulator for indoor inspection, package delivery, communication relay, and environmental sampling. The scenario is fictional and safe for classroom work. The interface can be implemented in a spreadsheet, Python, or a small 嘉立创 EDA board whose buttons select tasks and LEDs show feasible assignments.

The experiment has three rounds. In Round 1, communication is reliable. In Round 2, one platform loses link availability. In Round 3, operator workload becomes the binding constraint. Students must recompute the feasible set, not merely lower a score. They submit the capability matrix, admissibility matrix, selected assignment, unassigned task, and sensitivity to two changed weights.

2.2.4 Whole-chapter synthesis and responsibility discussion

The development sequence should be explained as a change in the relationship among platform, operator, and task: teleoperation, remote supervision, local adaptation within stated limits, and coordinated multi-platform behavior. Trends should be discussed through perception, onboard computation, energy management, reliable communication, and verification. Avoid treating “higher autonomy” as a universal ranking.

2.2.5 Practice and extension

1. Compare ground, aerial, and underwater robots using six common architecture fields.
2. Normalize one benefit and one burden attribute and explain the reference ranges.
3. Enumerate all feasible assignments for a 3×3 classroom matrix.
4. Give an example where a lower-capability robot is the better assignment.
5. Define the human authority and recovery condition for one delegated function.

Normalization changes quantities with different units onto a common scale. It does not create information. If a score is normalized from a narrow or biased range, the resulting comparison remains limited by that range. Always record the reference minimum, maximum, direction of preference, and task condition.

2.3 The unit is not the system

An individual platform may be fast, robust, quiet, well-sensed, energy-limited, or easy to supervise. No single scalar “strength” captures these properties. Let platform i be described by

$$\mathbf{c}_i = (m_i, s_i, e_i, \ell_i, h_i),$$

where m_i is mobility adaptability, s_i sensing suitability, e_i energy endurance, ℓ_i communication availability, and h_i the reachability of human supervision. These are not

real performance specifications. They are dimensions that prevent a vague claim such as “Platform A is best” from hiding its conditions.

2.4 Assignment as a matrix problem

Let tasks be $T_1, \dots, T_n$ and platforms be $R_1, \dots, R_m$. A cost matrix $C = [c_{ij}]$ describes the expected burden of assigning platform R_i to task T_j . A binary assignment variable x_{ij} equals one when that pairing is chosen. A simple formulation is

$$\min_{x_{ij}} \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

subject to

$$\sum_j x_{ij} \leq 1, \quad \sum_i x_{ij} = 1, \quad x_{ij} \in \{0, 1\}.$$

The constraints can be expanded to include energy reserve, communication, operator workload, and task-specific admissibility. A low cost cannot make an inadmissible assignment valid.

Worked example

For a fictional disaster-response exercise, three platforms are assigned to observation, supply transport, and communications relay. The utility matrix is

$$U = \begin{bmatrix} 8 & 4 & 6 \\ 5 & 9 & 3 \\ 7 & 6 & 8 \end{bmatrix}.$$

The best diagonal assignment may still be suboptimal. Students should compare all feasible permutations and then state the constraints they assumed. [Illustrative teaching value]

2.5 Autonomy is a matching variable

This book uses four descriptive levels: direct teleoperation, supervised autonomy, autonomy within stated limits, and coordinated autonomy. The levels do not form a universal ladder. A task with intermittent communication may require more local autonomy but stricter action limits. A task with serious consequences and reliable communication may require frequent human confirmation. The useful question is: “Which decision is delegated, under which conditions, and with which recovery path?”

Mode		Delegated function	Required human relation
Teleoperation		low-level motion	continuous command and observation
Supervised autonomy	auton-	routine control inside a defined envelope	human monitors and can intervene
Autonomy within stated limits	within	local adaptation under hard constraints	human approves the mission and exceptions
Coordinated autonomy	auton-	multiple agents share task state	human owns goals, boundaries, and reassignment

2.6 Capability vectors and normalization

The vector $\mathbf{c}_i$ is useful only when its coordinates have declared meanings. A score of 8 for sensing suitability might mean spatial resolution, coverage, stability, or operator interpretability. These are not interchangeable. Before comparing platforms, define a scale, a reference task, and a direction of preference.

Let the raw measurement for platform i and capability r be q_{ir} . For a benefit attribute, a simple classroom normalization is

$$\bar{q}_{ir} = \frac{q_{ir} - q_r^{\min}}{q_r^{\max} - q_r^{\min}},$$

whereas for a burden attribute the order is reversed. A weighted score

$$S_i = \sum_{r=1}^p w_r \bar{q}_{ir}, \quad w_r \geq 0, \quad \sum_r w_r = 1$$

can support discussion, but it should never conceal an inadmissible condition. A platform with an excellent score can still fail a hard communication, energy, or supervision requirement.

A capability profile is a task- and condition-specific description of what a platform can sense, compute, communicate, move, and report, together with the uncertainty and limitations of those claims. It is not a permanent scalar rank.

Dimension	Example variable	Interpretation question
Mobility	speed, turning margin, terrain tolerance	Can the platform remain inside the allowed envelope?
Sensing	coverage, resolution, latency	Which states are observable, and when?
Computation	update rate, memory, model scope	Can the estimate be produced before it becomes stale?
Link quality	availability, delay, acknowledgment	What can be supervised or confirmed?
Energy	reserve, recharge, recovery margin	What safe state remains feasible?
Human interface	workload, clarity, intervention	Can a human understand and act in time?

2.7 Allocation with constraints

The basic assignment formulation can be extended with admissibility indicators $b_{ij} \in \{0, 1\}$, energy consumption e_{ij} , and operator workload h_{ij} . A classroom model is

$$\min_x \sum_{i,j} c_{ij} x_{ij}, \quad (2.5)$$

$$\text{s.t. } x_{ij} \leq b_{ij}, \quad (2.6)$$

$$\sum_j e_{ij} x_{ij} \leq E_i^{\max}, \quad (2.7)$$

$$\sum_{i,j} h_{ij} x_{ij} \leq H^{\max}, \quad (2.8)$$

$$x_{ij} \in \{0, 1\}. \quad (2.9)$$

The first constraint prevents a numerical optimizer from selecting a pairing that has already been declared invalid. The second and third constraints make resource limits explicit. If no feasible solution exists, the system must report infeasibility and request a mission revision; it must not silently relax a safety boundary.

Worked example: assignment under a changed premise

Consider three fictional platforms R_1, R_2, R_3 and three classroom tasks: map a marked room, carry a sensor package, and relay a message. The utility matrix is

$$U = \begin{bmatrix} 8 & 4 & 6 \\ 5 & 9 & 3 \\ 7 & 6 & 8 \end{bmatrix}.$$

The permutation $(R_1, T_1), (R_2, T_2), (R_3, T_3)$ has utility 25. Now suppose R_3 loses its relay interface. The old solution is no longer feasible even though its utility remains 25 in the old table. Reassignment is not a failure of optimization; it is a response to a changed feasible set. [Illustrative teaching value]

2.8 Trade-offs and sensitivity

Assignments are often sensitive to weights. Let $J(x; w)$ denote the selected objective under weights w . A small change in w can change the winning assignment when two options are close. Students should report the margin between the best and second-best feasible options as well as the winning label.

For two options x^A and x^B , the boundary at which they tie satisfies

$$\sum_r w_r (S_r(x^A) - S_r(x^B)) = 0.$$

This equation is a transparency tool. It shows which capability dimensions actually drive the allocation and gives a supervisor a chance to disagree with the priorities before a platform is committed.

Before accepting an assignment, verify: task definitions are compatible; capabilities are measured on a declared scale; hard constraints are separate from preferences; operator workload is counted; energy and communication reserves are recorded; unassigned tasks have an owner; and the reason for rejecting the next-best option is logged.

2.9 Autonomy as a function-to-authority map

The four descriptive levels in this book become more precise when written as a map from functions to authorities. A platform may autonomously stabilize its motion, recommend a route, and request a second observation, while a human retains authority

over the mission boundary and any irreversible consequence. This is function-level delegation, not a single ladder.

Function	Local autonomy may provide	Human or organizational responsibility
State estimation	filtering and consistency checks	define validity limits and review uncertainty
Path generation	candidate paths inside an envelope	approve objective and hard constraints
Local recovery	hold or safe-state logic	define recovery hierarchy and resource margin
Task allocation	matching under declared costs	accept priorities and resolve conflicts
Consequential action	no automatic delegation in this textbook	explicit authority, review, and documented record

2.10 A classroom allocation game

Divide the class into a platform team, a mission team, and an assurance team. The platform team receives three capability cards, each with one strong and one weak dimension. The mission team receives four task cards with different requirements. The assurance team receives two hidden changes, such as a link outage or a reduced energy reserve.

Round one asks for an allocation using only the visible cards. Round two reveals the hidden changes and allows one reassignment. The instructor then asks which decision was wrong: the original assignment, the missing assumption, or the failure to define a recovery rule. A robust system accounts for its first allocation and for the point at which its premises change.

2.11 Historical and methodological interpretation

The historical account of Tian Ji’s race is brief and has been retold in different forms. Its value here is structural: an overall objective can be changed by pairing resources differently. It is not evidence that the ancient actors solved a modern integer program, and the numbers in the worked examples are fictional.

2.12 Matching, greedy choice, and global objective

The assignment problem can be represented as a bipartite graph whose left nodes are platforms and right nodes are tasks. An edge exists only when the pairing is admis-

sible. A greedy method that selects the locally highest utility edge can fail because it consumes a platform needed by a later task. The horse-race story is a compact reminder that local comparison and global allocation are different operations.

For an assignment x , define total utility $U(x) = \sum_{i,j} u_{ij}x_{ij}$. A review should compare the selected solution with the best feasible alternative and report the utility margin ΔU . A small margin means the allocation is sensitive to measurement error or priority changes and should be treated as conditional.

Greedy failure

Let $u_{11} = 10$, $u_{12} = 9$, $u_{21} = 9$, and $u_{22} = 1$. Greedy choice takes (R_1, T_1) and leaves utility 1, for total 11. The alternative $(R_1, T_2), (R_2, T_1)$ gives 18. The lesson is not that greedy algorithms are always wrong; it is that the task structure must be stated before a local score is treated as a system decision. [Illustrative teaching value]

2.13 Assignment review and reassignment

At every reassignment, preserve the original allocation, the changed premise, the new feasible set, and the authority that accepted the trade-off. A system that changes assignments without retaining the discarded plan cannot later distinguish changed conditions from changed preferences.

2.14 Historical notes and further reading

The historical story illustrates a reassignment logic, not a theorem about all competitions. For formal assignment and matching methods, consult standard operations-research texts. For human-robot interaction and autonomy boundaries, compare the system view in [7, 13, 15].

2.15 Exercises

- 2.1 Construct a 4×3 utility matrix for four fictional platforms and three classroom tasks. State the assumptions behind each entry.
- 2.2 Give a case where the platform with the highest sensing score should not receive the sensing task.
- 2.3 A communication outage removes one assignment from the feasible set. Reformulate the constraint and explain the reassignment.
- 2.4 Compare supervised autonomy and autonomy within stated limits using the dimensions of authority, observability, intervention, and recovery.

2.5 Write the historical evidence card for Tian Ji's race. Which details belong to later versions rather than the minimal historical account?

Chapter summary

- System capability is multidimensional and task-dependent.
- Allocation is a constrained matching problem, not a ranking of platforms.
- Autonomy should be delegated by function and boundary, not described as a single score.

Chapter 3

Military Unmanned Aircraft: State Machines and Safe Degradation

Narrative Prelude — The Third Drum. In *Zuo Zhuan*, Cao Gui waits through the first and second drumbeats and recommends action after the third. After the encounter he refuses to pursue immediately; he first inspects the tracks and looks from the carriage. The story supplies a compact lesson in temporal evidence: an event is not yet a state transition, and a state transition requires an exit condition.

Source note. The drumbeats, the statement “one effort, then decline,” and the inspection of tracks are part of the historical narrative (H). The aircraft state machine, timer values, and loss functions are classroom models (C); they are not ancient descriptions of autonomous flight.

Learning objectives

1. define states, events, guards, actions, and transitions;
2. design a finite-state model for communication loss, degraded operation, recovery, and termination;
3. distinguish a fault event from a safe response to that event;
4. analyze why timers and exit conditions prevent ambiguous behavior.

A finite-state machine is built from elementary set and logic concepts. Let Q be a finite set of states and Σ a set of events. A transition function maps a current state and an event to a next state. A guard is a logical condition: it is either true or false for the current evidence. An invariant is a condition that must remain true while a state is active. These ideas are closely related to the familiar logic operations AND, OR, and NOT. The important beginner’s habit is to write the condition in full rather than hide it in a vague label such as “abnormal.”

3.1 Foundations: logic, finite states, and guarded transitions

Many failures arise because a design uses a word such as “normal,” “fault,” or “return” without defining its entry condition, permitted behavior, and exit condition. Finite-state models provide a small but rigorous language for this problem. They are useful whenever a system has a limited number of operational modes and the meaning of an event depends on the current mode.

3.1.1 Sets and propositions

A set is a collection of distinct objects. Let $Q = \{\text{NORMAL}, \text{VERIFY}, \text{PROTECTED}\}$ be a set of modes. A proposition is a statement that is true or false, such as $p : \ell_k < \ell_{\min}$, where ℓ_k is a link-quality estimate. The basic operators are conjunction $p \wedge q$ (AND), disjunction $p \vee q$ (OR), and negation $\neg p$ (NOT).

Truth tables prevent ambiguous prose. For example, a verification transition might require a low link estimate *and* an elapsed persistence time:

$$g_{\text{verify}} = (\ell_k < \ell_{\min}) \wedge (\tau_k > \tau_{\min}).$$

With AND, a short dip alone does not trigger the transition. Replacing AND with OR would create a different policy. The formula makes that design choice inspectable.

An **event** is an observed occurrence, such as a timeout or a sensor disagreement. A **state** is a persistent mode with declared permissible behavior. A **guard** is a Boolean condition that must be true for a transition to occur. An **invariant** is a condition that must remain true while a state is active.

3.1.2 The finite-state machine

A deterministic finite-state machine can be described by the tuple

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F),$$

where Q is a finite state set, Σ is an input-event alphabet, $\delta : Q \times \Sigma \rightarrow Q$ is a transition function, q_0 is an initial state, and F is a designated set of terminal or accepting states. An engineering state machine normally adds guards, entry actions, exit actions, timers, records, and owners. A useful extended transition notation is

$$(q_k, \sigma_k, g_k) \mapsto (q_{k+1}, \alpha_k, \rho_k),$$

where α_k is a bounded state action and ρ_k is the decision record.

A state diagram should answer five questions for every arrow: What event was seen? Which guard became true? What action is taken on entry? What evidence

is recorded? Under what condition can the state be exited? An arrow labelled only “error” answers none of them.

3.1.3 Time, hysteresis, and chattering

Physical and digital signals are often noisy. If a state transition occurs exactly when a measured value crosses one threshold, small fluctuations can cause repeated transitions, called chattering. Hysteresis uses distinct enter and exit thresholds. For a signal r_k , define

enter DEGRADED when $r_k < r_{\text{enter}}$,
 exit DEGRADED when $r_k > r_{\text{exit}}$, $r_{\text{exit}} > r_{\text{enter}}$.

A persistence timer adds a second safeguard. If au_k counts consecutive low-quality samples, a transition occurs only when both the threshold and duration conditions are met. The timer must itself have a reset rule, a clock source, and a treatment for missing samples.

Worked example: a two-threshold monitor

Let a fictional quality score be sampled once per second. Enter VERIFY when $r_k < 0.35$ for at least 3 consecutive seconds; return to NORMAL only when $r_k > 0.55$ for 5 consecutive seconds. The sample sequence 0.42, 0.31, 0.33, 0.34 enters VERIFY at the fourth sample. The later sequence 0.50, 0.58, 0.56, 0.57, 0.59, 0.60 returns to NORMAL only after five consecutive samples above 0.55. The gap between 0.35 and 0.55 prevents the state from changing on every small fluctuation.

[Illustrative teaching value]

3.1.4 Mealy and Moore viewpoints

In a Moore-style machine, outputs depend primarily on the current state: the state itself makes permitted actions clear. In a Mealy-style machine, outputs depend on the current state and input event. Neither style is universally better. A safe state is often clearer in Moore form because the allowed behavior is visible from the state label. A short notification may be convenient in Mealy form because it responds directly to an event. The important lesson is to avoid hidden actions whose occurrence cannot be reconstructed from a state or event record.

3.1.5 Reachability, dead ends, and test cases

Before implementation, draw the directed graph whose nodes are states and whose edges are possible transitions. Basic review questions are mathematical as well as practical:

- Is every intended state reachable from the initial state?
- Can a prohibited state be reached through an unintended sequence?
- Does every nonterminal state have a defined response to missing or invalid input?
- Is there a deadlock state with no legal exit and no declared terminal meaning?
- Does every state have a record and a named owner?

For a small machine, these questions can be answered by manual traces. For example, start in NORMAL, inject a low-quality event, let the timer expire, and inspect whether the design reaches VERIFY, PROTECTED, or an undefined condition. This exercise exposes incomplete transition logic before it becomes code; it does not claim to prove an entire real-world system safe.

[title=State-machine trace table]

```
write the initial state and all declared invariants
for each test event, record its time, quality, and source
evaluate every guard explicitly as true or false
apply one permitted transition or record why none is enabled
record entry action, owner, timer, and evidence used
check that the next state has an exit or terminal condition
```

- Calling an event a state: “link loss” does not say what behavior is permitted after the observation.
- Using a single threshold with no duration rule for a noisy quantity.
- Providing a nominal transition but no transition for contradictory, stale, or absent evidence.
- Allowing a recovery transition without checking that the condition which caused degradation has actually been resolved.
- Treating an emergency state as self-explanatory rather than giving it a bounded action, recovery condition, and record.

3.1.6 Foundation self-check

1. Write a guard that requires low link quality for at least five samples.
2. Explain the difference between the event “link loss” and the state “VERIFY.”
3. What problem does hysteresis solve, and what problem does it not solve?
4. Trace a state machine through one missing-input event and identify its record.

An acceptable answer identifies a Boolean condition, a time rule, an action, an exit condition, and an owner. Merely renaming the event as “emergency” is not a state design.

3.1.7 Bridge to the third drum

The historical narrative emphasizes waiting for evidence and checking before pursuit. Its engineering translation is limited but useful: an observation should not be mistaken for an automatic mode change. The chapter uses state machines to make timing, guards, hysteresis, safe behavior, and exit conditions explicit in a fictional safety-critical system.

3.2 Historical opening and source note

The third drum in Cao Gui’s story provides a way to teach timing without turning the lesson into a slogan about courage. The system hears an event, evaluates evidence, waits when a condition is unresolved, and changes state only when a guard is satisfied. An unmanned aircraft is therefore introduced through its state transitions, not through the claim that it can simply “fly by itself.”

3.2.1 Foundational knowledge to master

An unmanned aircraft system contains an air vehicle, propulsion and energy subsystem, flight-control computer, navigation and positioning functions, mission payload, data link, ground station, launch/recovery support, and maintenance or configuration records. Aircraft may be compared by fixed-wing, rotary-wing, and hybrid configurations; by endurance and mission payload; and by the degree of human supervision. These are descriptive categories, not performance rankings.

The flight-control loop uses a reference r_k , estimated state $\hat{x}_k$, control input u_k , and measured output y_k :

$$e_k = r_k - y_k, \quad u_k = K_p e_k + K_i \sum_{j=0}^k e_j + K_d (e_k - e_{k-1}).$$

The proportional, integral, and derivative terms should be explained as present error, accumulated error, and change of error. The classroom purpose is to show why delay, saturation, noisy measurements, and state validity matter; it is not a flight-tuning procedure.

3.2.2 Learning content and mathematical derivation

Let

$$Q = \{N, W, D, R, S, T\},$$

represent NORMAL, WAIT, DEGRADED, RETURN, SAFE HOLD, and TERMINATE. A transition is enabled by an event σ_k and guard $g(q_k, \sigma_k)$. For example,

$$g_{\text{return}} = (\ell_k < \ell_{\min} \wedge \tau_k > \tau_{\max}) \vee (r_k < r_{\min}),$$

where ℓ_k is link quality, τ_k a persistence timer, and r_k navigation confidence. The derivation demonstrates why one event can lead to different responses under different state conditions.

Students should also analyze hysteresis:

$$r_{\text{enter}} < r_{\text{exit}},$$

so that a system does not oscillate between degraded and normal modes when a noisy signal hovers near one threshold.

3.2.3 Case studio: a UAV state-machine simulator

The safe laboratory case uses a simulated inspection aircraft in a two-dimensional environment. Students implement six states and inject link loss, navigation uncertainty, low energy, and restored communication. The simulator records event, guard value, transition, timer, state owner, and recovery condition. A physical 嘉立创 EDA board may later provide buttons for events and LEDs for state display; it must remain a tabletop state-machine demonstrator with no flight hardware.

The test matrix includes nominal operation, short link loss, persistent link loss, simultaneous navigation degradation, recovery after a timeout, and contradictory sensor input. A passing result requires the expected state, a complete record, and a declared protected response.

3.2.4 Whole-chapter synthesis and responsibility discussion

Discuss the evolution from remotely piloted aircraft to increasingly automated flight management, emphasizing sensing, navigation, onboard computation, data links, and verification. Current trends should be framed as technical questions: how are unknown inputs handled, how are updates verified, how does the aircraft behave when communication disappears, and how can an operator reconstruct a decision after the event?

3.2.5 Practice and extension

1. Draw the architecture of an unmanned aircraft system and assign one failure mode to each block.
2. Design a six-state machine with entry action, invariant, exit guard, and owner for each state.

3. Explain why “link restored” alone may not justify returning to normal.
4. Compute the time margin for a fictional verification and recovery chain.
5. Compare a local log and a ground log when their timestamps disagree.

Notation	Plain-language meaning	Example
$q \in Q$	current state	$q = \text{VERIFY}$
$\sigma \in \Sigma$	event or input	link quality falls
$g(q, \sigma)$	guard condition	energy is above reserve
$\delta(q, \sigma)$	state transition	VERIFY to PROTECTED
$\alpha(q, \sigma)$	bounded action	record event and stop new task

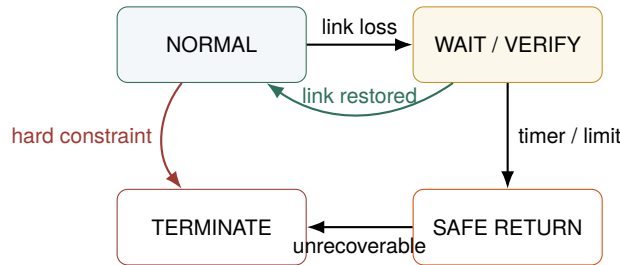
3.3 Events are not states

An event is something that occurs: communication quality falls, energy crosses a threshold, or a sensor disagrees with another sensor. A state is the condition in which a system remains until a transition is enabled. Confusing the two creates brittle logic. “Lost link” is an event; “degraded local operation” is a state with entry actions, permitted behaviors, and a recovery or termination condition.

Let Q be a finite set of states, Σ a set of events, and $\delta : Q \times \Sigma \rightarrow Q$ a transition function. A practical state machine also needs guards $g(q, \sigma)$ and actions $\alpha(q, \sigma)$. The state update is

$$q_{k+1} = \delta(q_k, \sigma_k; g_k),$$

with a record of the event, guard result, action, and responsible authority.



3.4 Time and safe degradation

Suppose communication quality is ℓ_k , energy reserve is e_k , and navigation confidence is r_k . A simple guard for a transition to verification might be

$$g_{\text{verify}} = (\ell_k < \ell_{\min}) \vee (r_k < r_{\min}) \vee (e_k < e_{\min}).$$

The expression is intentionally conservative: it states which conditions are sufficient to change state, not how to tune a real platform. A timer τ prevents a system from remaining indefinitely in a state that was intended to be temporary. Every timer must specify what happens at expiry.

Worked example

In a fictional inspection mission, an aerial platform loses the data link for 40 seconds. The event alone does not determine the response. If energy is ample and navigation is trusted, the system may enter a bounded wait state. If navigation confidence also falls, the same link event may transition directly to a safe-return state. The state machine makes the difference explicit and auditable.

3.5 State-machine vocabulary

A usable state machine distinguishes five objects:

1. a state is a condition that persists long enough to have permitted and forbidden behaviors;
2. an event is an occurrence that may enable a transition;
3. a guard is a predicate that must be true before the transition is allowed;
4. an action is performed on entry, during residence, or on exit;
5. an invariant is a condition that must remain true while the state is active.

The distinction prevents a common design error: writing a list of alarms without specifying what the system does after an alarm. A state called “communication lost” is not enough. Does it wait, hold, request a relay, return, or terminate? Under what evidence? With what timer?

A safe state is a state whose permitted behavior remains inside a declared risk and authority envelope when the system cannot support its normal mission logic. It is not necessarily motionless; it is a state with bounded consequences and an explicit owner.

3.6 Guard conditions and invariants

Suppose q_k is the current state and σ_k the event. A guarded transition can be written

$$q_{k+1} = \delta(q_k, \sigma_k) \quad \text{only if} \quad g(q_k, \sigma_k) = 1.$$

The guard should be evaluated from timestamped variables. For a safe state, an invariant might be

$$I_{\text{safe}} : e_k \geq e_{\text{return}} + e_{\text{margin}}, \quad r_k \geq r_{\text{min}},$$

or, when those conditions cannot be maintained, the state may require a different recovery action. A safety argument is incomplete if the invariant is written but no transition exists for its violation.

Design object	Example	Review question
Entry guard	$\ell_k < \ell_{\text{min}}$	Is the measurement fresh and independently checked?
State invariant	energy reserve remains above margin	What if the invariant becomes false?
Residence action	reduce task scope and record event	Is the action itself bounded?
Exit guard	link restored and verified	Does restored link mean available, trusted, and authorized?
Expiry action	enter safe state	Who owns the expiry decision?

3.7 Timed transitions and hysteresis

Binary thresholds can cause oscillation. If a link is declared healthy whenever $\ell_k \geq 0.50$ and unhealthy otherwise, noise near 0.50 can alternate the state every update. Hysteresis uses distinct entry and exit thresholds:

$$\ell_k < \ell_{\text{loss}} \Rightarrow \text{degraded}, \quad \ell_k > \ell_{\text{restore}} \Rightarrow \text{candidate for recovery},$$

with $\ell_{\text{restore}} > \ell_{\text{loss}}$. A timer can add persistence, requiring the condition to last for Δt before the transition is enabled. The exact values are classroom parameters, not specifications.

Worked example: three drumbeats as temporal evidence

Suppose a fictional system receives three low-quality reports at times t_1, t_2, t_3 . A naive design transitions to “respond” after the first report. A more disciplined design enters “observe” after the first, “verify” after the second, and permits a bounded recommendation only after the third if the sources are sufficiently independent and the guard remains true. The analogy to Cao Gui is about the changing meaning of repeated events, not about copying a historical decision into software. [Illustrative teaching value]

3.8 Hybrid systems and uncertainty

Many intelligent systems combine continuous variables with discrete states. Position, energy, and confidence evolve continuously, while “normal”, “verify”, and “safe hold” are discrete. A hybrid model can be written

$$\dot{x} = f_q(x, u, t), \quad q \in Q, \quad (3.1)$$

$$q^+ = \delta(q, \sigma) \quad \text{when a guard is satisfied.} \quad (3.2)$$

The model separates what changes gradually from what changes by policy. It also exposes a boundary: a continuous estimate may be uncertain, while a discrete state label can look exact. The transition logic must retain the uncertainty that produced the label.

[title=State-transition review procedure]

```
for each candidate transition:
    identify triggering event and timestamp
    evaluate guard using valid, current variables
    check state invariants and authority scope
    if guard and authority are valid:
        execute bounded entry action
        start or update the state timer
    else:
        preserve current state or request review
        record event, guard values, decision, owner, and version
```

3.9 Failure analysis of a state machine

Read a state machine in three directions. Forward reading asks what happens after an event. Backward reading asks which evidence could have produced the current state. Cross-state reading asks whether two states permit contradictory actions. Common failure patterns include:

- a transition has an entry condition but no exit condition;
- a timer expires without a defined action;
- a recovery transition assumes the same sensor that caused the failure;
- two transitions are simultaneously enabled and have no priority rule;
- the interface displays a new state before the record server acknowledges it;
- the state label is preserved after the configuration changes.

For every state, write a one-sentence answer to: What may the system do? What must it never do? What evidence allows it to leave? What happens if no evidence arrives? Who receives the record?

3.10 Historical and methodological interpretation

The passage in *Zuo Zhuan* gives a narrative of timing, morale, observation, and restraint. It does not prescribe a formal machine. The engineering interpretation asks students to encode action together with waiting, verification, inspection, and stopping. In a safety-critical system, the ability to remain in a safe state is part of intelligence.

3.11 Reachability and state-machine verification

For a finite state machine, define the reachable set $\mathcal{R}(q_0)$ as the states that can be reached from initial state q_0 under allowed events and guards. A simple verification question is whether every reachable state has an owner, a bounded behavior, and a path to recovery or termination. An unreachable safety state is decorative; a reachable state with no exit is a latent deadlock.

Property	Test	Failure meaning
Reachability	enumerate event sequences	state cannot be entered or is unexpectedly reachable
Determinism	test simultaneous guards	two actions compete without priority
Progress	inspect timers and exits	system can remain indefinitely in a temporary state
Safety invariant	inject fault values	protected condition is not preserved
Record reconstruction	replay a transition	guard or owner cannot be reconstructed

3.12 Timed verification exercise

Create an event sequence containing link loss, low energy, restored link, and timer expiry. Run it through the state machine twice: once with events arriving in order and once with a delayed event. Compare the results. If a delayed event can reopen

an expired authority or bypass a safe state, the machine needs a validity interval or a versioned event record.

3.13 Historical notes and further reading

The narrative in *Zuo Zhuan* is a historical account, not a formal finite-state machine. Its value is that the character of the decision changes with time and evidence. For state estimation, filtering, and hybrid systems, see [9, 8, 7].

3.14 Exercises

- 3.1 Design a six-state machine for a fictional unmanned inspection platform: normal, waiting, degraded, recovery, safe return, and terminate.
- 3.2 For each state, list one entry condition, one exit condition, one forbidden action, and one required log field.
- 3.3 Explain why “communication lost” should not be used as both an event and a permanent state.
- 3.4 Add a timer to your state machine. What is the failure mode if the timer has no expiry action?
- 3.5 Separate the historical evidence in the Cao Gui narrative from the modern state-machine interpretation.

Chapter summary

- Events trigger transitions; states define what the system may do while conditions persist.
- Safe degradation requires guards, timers, recovery paths, and termination paths.
- A state machine is also a responsibility map: each transition needs evidence and an owner.

Chapter 4

Unmanned Ground Vehicles: Perception, Planning, and Feedback

Narrative Prelude — A Blade with Room to Move. Zhuangzi’s Cook Ding does not force the blade through dense material. He follows structure, and at a difficult joint he becomes alert, stops, looks, and slows down. The parable is not an engineering record. It is a disciplined metaphor for a ground platform that must treat a map as an estimate of traversability rather than as a promise of passage.

Source note. The textual core is “Nourishing Life” in *Zhuangzi* (P). The workshop, blade movement, and detailed scene are literary reconstruction. The ground-vehicle architecture, cost functions, and route values are classroom examples (C).

Learning objectives

1. distinguish measurements, semantic terrain estimates, global paths, local trajectories, and feedback control;
2. define a traversability representation with physical meaning;
3. formulate a weighted path cost and explain its trade-offs;
4. identify when local detouring is a successful response rather than a planning failure.

Three basic ideas support this chapter. First, a coordinate is meaningful only relative to a reference frame. Second, a map is a representation of space, not space itself; a grid cell can be known, occupied, free, or unknown. Third, feedback control compares a desired value with a measured value and acts on the error. A simple error is $e_k = x_k - x_k^r$, where x_k^r is a reference. LaValle’s public online text introduces planning through configuration, obstacles, free space, graphs, and search [20]; MIT’s public Introduction to Robotics course provides a complementary route into frames, motion, and control [19].

4.1 Foundations: geometry, maps, paths, and feedback

Path planning begins long before an algorithm runs. A system must decide what a location means, which regions are traversable, which constraints are hard, what uncertainty is represented, and how an intended path is translated into bounded motion. This module introduces the geometric and control vocabulary needed for that chain. The public robotics and planning resources listed in the bibliography provide a broader mathematical treatment [19, 20].

4.1.1 Points, vectors, frames, and distance

In a planar teaching model, a point is represented by coordinates $p = [x \ y]^\top$ in a named reference frame. A displacement vector is the difference between two points: $\Delta p = p_2 - p_1$. The Euclidean distance is

$$d(p_1, p_2) = \|p_2 - p_1\|_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

The formula is only meaningful when both points use the same frame, unit, datum, and time convention. A coordinate pair without a frame label is incomplete engineering data. The frame transformations used in navigation are developed further in Chapter 11.

The **workspace** is the physical region in which a platform moves. A **configuration** describes the platform's position and orientation. The **configuration space** is the set of possible configurations; its free subset $\mathcal{C}_{\text{free}}$ contains configurations that satisfy declared geometric and safety constraints. A path is acceptable only when it stays in the relevant free set.

Representing a finite-size platform as a point can be convenient, but it hides clearance. One common teaching construction expands obstacles by a safety margin and plans the point path outside the expanded obstacles. The margin should reflect localization uncertainty, stopping distance, map age, and the cost of being wrong. It is not a decorative buffer.

4.1.2 Maps are claims, not pictures

An occupancy grid partitions space into cells. A cell may contain a probability or category such as free, occupied, unknown, or restricted. A semantic map adds meaning: corridor, stairs, soft ground, work zone, or protected area. A route planner should use the semantics and confidence assigned by the map, rather than treating all empty-looking geometry as equally traversable.

Let $m(v)$ denote the metadata of a map cell or region v . A useful record is

$$m(v) = (\text{value}, \text{frame}, \text{resolution}, \text{time}, \text{source}, \text{uncertainty}, \text{purpose}).$$

The last coordinate matters: a map generated for rapid evacuation and a map generated for routine inspection may assign different costs to the same corridor. A map is therefore a versioned claim about a place under a particular objective.

4.1.3 Graphs and path cost

A grid or road network can be represented by a graph $G = (V, E)$. Nodes V represent cells or waypoints; edges E represent allowed local transitions. A path $P = (v_0, v_1, \dots, v_N)$ has additive cost

$$J(P) = \sum_{k=0}^{N-1} c(v_k, v_{k+1}).$$

The edge cost can combine distance, estimated traversal burden, uncertainty, energy, and deviation from a preferred corridor. A simple example is

$$c(e) = \alpha d(e) + \beta r(e) + \gamma u(e),$$

where d is distance, r is a terrain or rule burden, u is an uncertainty burden, and the weights are documented teaching choices. Hard restrictions should remove an edge from E rather than give it a merely large cost.

Breadth-first search works on equal-cost edges. Dijkstra's algorithm finds minimum cost for nonnegative edge weights. A* search uses a heuristic estimate of remaining cost to prioritize promising nodes. These algorithms can be mathematically correct while returning an unacceptable path when the map omits a restriction, the cost hides a hard boundary, or the current state differs from the assumed state.

Worked example: distance is not the whole path cost

Two fictional routes connect a laboratory entrance to a charging station. Route A has distance 90 m, uncertainty burden 0.6, and rule burden 0.1; Route B has distance 110 m, uncertainty burden 0.1, and rule burden 0.1. With $\alpha = 1$, $\beta = 30$, and $\gamma = 0$ after combining rule burden into a separate hard check, the comparable costs are 108 and 113. If uncertainty weight rises to 50, Route A costs 120 while Route B costs 115, so the longer route becomes preferable under that stated model. The arithmetic does not choose the weights; it reveals the consequence of choosing them. [Illustrative teaching value]

4.1.4 Planning and control are different functions

A planner chooses a reference path $p^*(s)$ or sequence of waypoints. A controller generates bounded inputs that reduce tracking error. For a scalar teaching model, let $e_k = p_k - p_k^*$ be position error and choose

$$u_k = -K e_k, \quad K > 0.$$

If the nominal update is $p_{k+1} = p_k + u_k$, then $e_{k+1} = (1 - K)e_k$. In this simplified model, $0 < K < 2$ makes the nominal error decay, while large gains can create oscillation. Real platforms have dynamics, delay, saturation, disturbances, and estimation error; this equation gives intuition, not a certification.

A local controller cannot repair a globally invalid plan. Conversely, a perfect map plan cannot move a platform when its local sensing detects an unmodeled obstruction. A sound architecture keeps the functions separate and uses feedback to trigger an explained replan.

Layer	Main object	Failure question
Perception	observations and map claims	What is unknown or stale?
Planning	feasible path and cost model	Which boundary or objective does it assume?
Guidance	desired local path or waypoint	When does the reference expire?
Control	bounded corrective input	What happens under delay or saturation?
Monitoring	residual, margin, and record	What observation justifies re-planning?

4.1.5 Replanning as evidence revision

A replan should have a declared trigger. Let $J(P; \mu_k, G)$ be the cost of path P under map claim μ_k and goal G . One abstract trigger is

$$|J(P; \mu_{k+1}, G) - J(P; \mu_k, G)| > \eta,$$

or a change in the feasible set itself. The system record should identify the new observation, map version, rejected path, new cost or constraint, time, and owner. Changing routes merely because an output “looks better” is not explainable planning.

- Subtracting coordinates expressed in different frames or units.
- Treating an unknown map cell as free simply because it is not marked occupied.
- Encoding a prohibited region as a low-priority cost rather than a removed transition.
- Confusing a globally short path with a locally trackable or recoverable path.
- Replanning without recording the changed evidence or the previous plan's assumptions.

4.1.6 Foundation self-check

1. List the frame and unit required before subtracting two position vectors.
2. Explain why an unknown map cell should not automatically be treated as free.
3. For a graph with equal edge costs, which basic search method is sufficient and why?
4. Give one changed premise that would justify replanning a previously valid route.

The expected answer keeps geometry, map semantics, planning, and control separate. A shorter path is not automatically safer, more current, or recoverable.

4.1.7 Bridge to Cook Ding and structure

The story of Cook Ding is used as a philosophical opening about respecting structure, not as a technical source for planning. The chapter's engineering question is concrete: how does a system transform geometric observations into traversability claims, choose a constrained path, control bounded motion, and revise the plan when the evidence changes? The following sections develop that perception–planning–control loop.

4.2 Historical opening and source note

Cook Ding's blade follows structure rather than forcing a straight line. The story gives students an intuitive entry to traversability: an open-looking route can contain unknown ground, poor clearance, high slope, weak localization, or a lost communication margin. A ground vehicle must understand the route as a structured estimate, not as a line drawn on a screen.

4.2.1 Foundational knowledge to master

An unmanned ground vehicle combines a chassis and drive train, energy source, on-board computer, perception sensors, localization, communication, mission payload, actuator interfaces, and operator or supervisory station. Typical categories include wheeled, tracked, legged, and hybrid platforms. The course should compare them by terrain adaptability, stability, energy consumption, payload, sensing geometry, communication dependence, and recovery options.

The working chain is

observe terrain $\rightarrow$ estimate traversability $\rightarrow$ plan path $\rightarrow$ track path $\rightarrow$ update map.

This chain makes clear why a camera image, a map cell, a route, and a steering command are different objects.

4.2.2 Learning content and mathematical derivation

For graph $G = (V, E)$ and path P , define

$$J(P) = \sum_{e \in P} (\alpha d_e + \beta r_e + \gamma u_e + \delta q_e),$$

where d_e is distance, r_e terrain burden, u_e uncertainty, and q_e recovery or communication burden. Derive the cost by first defining a hard feasible set, then normalizing each soft criterion, then selecting weights. A forbidden edge must be removed from E rather than assigned a merely large cost.

For tracking error $e_k = p_k - p_k^*$, a bounded controller is

$$u_k = \text{sat}(-K e_k, u_{\min}, u_{\max}).$$

Students should explain the role of saturation and why a plan that is globally short may be locally untrackable.

4.2.3 Case studio: a safe indoor ground robot

The recommended product is a small indoor line-following and obstacle-avoidance vehicle using a 嘉立创 EDA-designed sensor and status board. The board may contain reflectance sensors, a distance sensor, motor-driver interface, battery monitoring, and an emergency stop input. The exercise is limited to a marked indoor track and carries no weapon payload.

Students compare three routes on a printed map, then let the vehicle encounter an unknown region. The report must contain the original map, observed change, rejected route, new route, cost calculation, tracking error, energy use, and recovery behavior. The physical product is assessed as an engineering artifact; the planning model is assessed separately from the hardware.

4.2.4 Whole-chapter synthesis and research-report connection

This lecture also introduces the report format required later: problem statement, system boundary, related work, assumptions, method, experiment or simulation, results, limitations, safety and environmental considerations, and conclusion. Students must distinguish what the vehicle observed from what the authors infer about the algorithm.

4.2.5 Practice and extension

1. Compare wheeled and tracked platforms using a common capability matrix.
2. Construct a three-route cost table and perform a sensitivity analysis on γ .
3. Derive the position-error update for the unsaturated proportional controller.
4. Identify one case where local detouring is a successful response, not a planning error.
5. Draft the methods and limitations sections for the indoor experiment.

A map is an estimate with structure. It should carry at least geometry, source, time, resolution, and uncertainty. A cell marked “free” means free under the map’s assumptions and validity interval; it does not mean that every future observation must agree.

4.3 From geometry to traversability

A camera, lidar, radar, inertial unit, and odometry produce different measurements. Let a semantic map at location q be

$$\mathbf{m}(q) = (o(q), s(q), \tau(q), u(q), \ell(q)),$$

where o is obstacle risk, s is slope or attitude risk, τ is surface traversability, u is localization uncertainty, and ℓ is communication availability. The vector is not a sensor output; it is a model assembled from multiple sources.

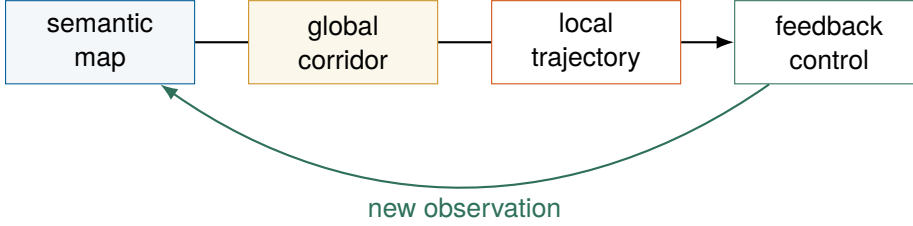
4.4 Global planning, local action, and control

Global planning proposes a corridor through a known or estimated map. Local planning generates a short-horizon trajectory under current observations. Feedback control reduces the error between desired and measured motion. If the state is $\mathbf{x}_k$ and

the control is $\mathbf{u}_k$,

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k.$$

The disturbance $\mathbf{w}_k$ is not a nuisance to be deleted from the model; it is the reason feedback is needed.



4.5 A physical path cost

For a candidate path P , use the illustrative cost

$$J(P) = \int_P (w_d + w_e c_e(q) + w_r r_t(q) + w_u u(q) + w_\ell r_\ell(q)) \, ds.$$

Distance, energy, terrain risk, localization uncertainty, and link risk are different quantities. The weights make priorities explicit; they are not objective truths. A hard constraint, such as a maximum slope or a minimum recoverability margin, should be represented in the feasible set rather than hidden inside a soft weight.

Worked example

Three routes in a fictional ground-logistics exercise have the following normalized scores:

	d	e	r_t	u	r_ℓ
A	4	7	8	6	5
B	6	5	3	3	4
C	8	4	2	2	2

Route A is shortest but riskier; Route C preserves more margin. No route is “best” until the mission objective and hard constraints are stated. [Illustrative teaching value]

4.6 Problem setup and sensing assumptions

Let the workspace be $W \subset \mathbb{R}^2$, obstacles be $O_1, \dots, O_n$, and free space be

$$W_{\text{free}} = W \setminus \bigcup_{j=1}^n O_j.$$

This geometric definition is a starting point rather than a complete traversability model. A platform has size, turning limits, stopping distance, sensor blind spots, and uncertain localization. The free set for a point is not the free set for a body.

Let a measurement from sensor i be

$$z_k^{(i)} = h_i(x_k, E) + v_k^{(i)}.$$

The observation becomes useful only after calibration, timestamp alignment, and a quality declaration. A map cell should therefore carry both a semantic value and a confidence or age field. A binary obstacle map may be convenient for a diagram, but it should not be mistaken for a complete account of the environment.

Traversability is a model-based estimate of whether a platform can move through a region while satisfying geometric, dynamic, energy, communication, and recovery constraints. It is not identical to visual openness.

4.7 Semantic maps and layered costs

A useful local map can be written as a stack of layers:

Layer	Variable	Meaning
Geometry	obstacle occupancy $o(q)$	Is the region occupied or unknown?
Surface	slope and roughness $s(q)$	What physical burden is expected?
Localization	uncertainty $u(q)$	How much can position error change the conclusion?
Communication	link quality $\ell(q)$	Can a human or peer receive evidence?
Recovery	reversibility $r(q)$	Can the system stop, wait, or leave safely?

These layers should not be collapsed too early. An area can be geometrically open, physically difficult, poorly localized, and easy to recover from. A cost function can combine them for a particular task, but the original layers should remain available for review.

4.8 Global planning and local planning

Global planning uses a longer horizon and a more stable map to propose a corridor. Local planning uses recent observations and platform dynamics to generate a short-

horizon trajectory. The two planners can disagree without either being defective. Their objectives and time scales differ.

Let P be a global path and $\tau_{k:k+H}$ a local trajectory. A local trajectory is admissible when it remains near the corridor, satisfies dynamics, and preserves a recovery margin:

$$\tau_{k:k+H} \in \mathcal{F}(P, \hat{E}_k, \hat{x}_k, \mathcal{C}).$$

The set $\mathcal{F}$ changes when the environment estimate or constraints change. Re-planning is therefore not a sign that the original plan was foolish; it is the expected behavior of a feedback system.

4.9 A simple controller

For a teaching model with position error e_p and heading error e_ψ , a proportional controller can be written

$$u_k = K_p e_{p,k} + K_\psi e_{\psi,k}.$$

The expression is intentionally simple. A real implementation would include actuator saturation, delay, dynamics, filtering, and stability analysis. The educational point is that a path is a reference, while feedback is the mechanism that responds to deviation.

Worked example: detour as a successful update

Suppose a platform follows a corridor and observes that the next cell is blocked. The global map had marked the cell as open, so the new observation contradicts the map. The system should mark the cell as uncertain or occupied, generate locally feasible alternatives, evaluate whether the corridor remains valid, and record the map correction. A detour is a sound planning response when it follows the evidence, respects constraints, and remains reversible. [Illustrative teaching value]

4.10 Cost functions and hard constraints

For a candidate path P , use the illustrative cost

$$J(P) = \int_P (w_d + w_e c_e(q) + w_r r_t(q) + w_u u(q) + w_\ell r_\ell(q)) \, ds.$$

The weights make priorities explicit; they are not objective truths. A hard constraint, such as a maximum slope or a minimum recoverability margin, should be represented in the feasible set rather than hidden inside a soft weight. If a path violates a hard constraint, a low numerical cost cannot make it admissible.

A robustness margin is the remaining distance between a predicted condition and a declared failure or constraint boundary. A plan with zero margin may be nominally feasible but is not robust to measurement noise, delay, or model error.

4.11 Perception-planning-control loop

The loop should be read as a repeated argument: the map proposes a world model, the planner proposes a motion, the controller produces a response, and the new measurement challenges the model. If the loop is drawn without feedback, it represents an open-loop script rather than an intelligent system.

Stage	Input and output	Failure to look for
Perception	signals to local map	blind spot, false obstacle, stale timestamp
Planning	map and goal to path	infeasible corridor, hidden hard constraint
Trajectory generation	path to short-horizon reference	ignores dynamics or stopping margin
Control	reference to bounded input	saturation, delay, oscillation
Feedback	outcome to model update	fails to recognize model mismatch

[title=Evidence-aware local replanning]

```
observe local region and validate timestamp
update geometry, terrain, uncertainty, and recovery layers
if a hard constraint is violated:
    enter safe state and request a new corridor
else:
    generate candidate short-horizon trajectories
    reject candidates outside dynamic or recovery bounds
    rank remaining candidates by declared cost
    execute only the bounded reference permitted by authority
record map revision, candidate set, rejection reasons, and outcome
```

4.12 Historical and methodological interpretation

Cook Ding’s language of finding room between structures is a philosophical image of skilled perception. It does not describe lidar, path planning, or control. The useful

analogy is limited: expertise is not blind force, and a system should respond to structure and uncertainty rather than to a geometric line alone. The engineering model begins when we define the map, dynamics, constraints, and feedback signal.

4.13 Decomposition, sampling, and collision margins

For a grid workspace, a cell decomposition converts a geometric problem into a graph problem. Each cell is a node, and an edge exists only if the platform can traverse the shared boundary while preserving a body-clearance and stopping margin. If the platform has radius r and the map uncertainty is δ , an obstacle should be inflated by at least $r + \delta$ in a conservative teaching model.

Sampling a few free points does not prove that the entire region is free. It can miss a narrow obstacle or a narrow corridor. A test should therefore include boundary cases, unknown cells, and points near the minimum clearance. Report the path length together with the collision margin and evidence that the map was current.

Clearance comparison

Two routes have equal length. Route A has minimum clearance 0.4 units and route B has minimum clearance 1.1 units. If localization uncertainty is 0.5 units, A has no remaining geometric margin while B retains a positive margin. A length-only planner treats them as tied; an evidence-aware planner does not.

[Illustrative teaching value]

4.14 Historical notes and further reading

The Cook Ding passage is a philosophical parable, not a record of vehicle navigation. Its useful question is structural: what does the system know about the space through which it intends to move? For planning and mobile robotics, see [6, 7, 8]. The reference lecture notes by Bullo and Smith provide a model for theorem-and-exercise organization, but the present chapter has a different scope and application.

4.15 Exercises

- 4.1 Define five map variables for a fictional ground platform and give each a measurement source and uncertainty statement.
- 4.2 Choose weights for the three routes in the worked example. Compute and interpret the resulting costs.
- 4.3 Give one condition that should be hard, not soft. Explain what would go wrong if it were only a penalty.

- 4.4 Write a local re-planning rule for a newly observed obstacle. Include observation, model update, action, and record.
- 4.5 Explain the difference between a global path, a local trajectory, and a control input.

Chapter summary

- A geometric line is not automatically a traversable route.
- Planning and control operate at different time scales and require feedback.
- A cost function exposes priorities; a constraint defines what is admissible.

Chapter 5

Intelligent Munitions: Classification Confidence, Relevance, and Authorization

Narrative Prelude — Ninety-Nine Percent Is Not an Answer. When Zhao was besieged, the most visible fact was the city in danger. Tian Ji's first impulse was to move directly toward the visible crisis. Sun Bin asked a different question: what must change for the siege to end? The historical lesson is not a reusable military trick. It is a warning against treating a strong signal as a complete definition of the task.

Source note. The account of Wei attacking Zhao, Zhao asking Qi for aid, and Sun Bin's argument to pressure Wei's vulnerable interior comes from *Shiji* (H). "En-circling Wei to rescue Zhao" is a later summary. Classification scores, decision cards, and loss values are fictional teaching constructs (C).

Learning objectives

1. distinguish classification, task relevance, recommendation, and authorization;
2. interpret a confidence score as conditional model output rather than universal certainty;
3. explain calibration, distribution shift, false alarms, and missed states;
4. formulate an admissible-action set with environment, rule, authority, and review constraints.

The central prerequisite is conditional probability. For events A and B , $P(A | B)$ means the probability of A after conditioning on B . Bayes' rule combines a prior belief with the likelihood of the observation. A classifier's score should be separated from the decision threshold that uses it. The basic confusion matrix distinguishes true positives, false positives, true negatives, and false negatives. MIT's public probability course provides review material on conditional probability,

random variables, and statistical inference [18]. NIST emphasizes that an AI output must be interpreted in relation to its context, intended use, and risk controls [14].

5.1 Foundations: probability, classification, and constrained decisions

This chapter needs a careful distinction between three questions: What was observed? What state is likely? What response is permitted? Probability can support the second question, but it cannot by itself answer the third. The mathematical tools below make that separation explicit. Public review material on conditional probability and random variables is available through MIT OpenCourseWare [18].

5.1.1 Events, random variables, and conditional probability

An event is a set of outcomes, such as $A =$ “a monitored condition is present.” A random variable maps an outcome to a numerical value, such as a sensor score S . The probability of A is $P(A)$; conditional probability asks about A after observing another event B :

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

The condition after the vertical bar must be stated. “Ninety percent confidence” is incomplete unless it says confidence in what proposition, under which population, given what information, and how it was estimated.

Bayes’ rule connects a prior belief to evidence:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B | A)P(A) + P(B | \bar{A})P(\bar{A})}.$$

The prior $P(A)$ is often called a base rate. It matters strongly when the event is rare. The likelihood terms $P(B | A)$ and $P(B | \bar{A})$ are properties of the evidence mechanism under declared conditions; they are not universal constants.

Worked example: the base-rate check

In a fictional quality-monitoring population, 1 in 1,000 cases has a condition A . A test is positive in 99% of affected cases and in 1% of unaffected cases. For 100,000 cases, the expected counts are about 99 true positives and 999 false positives. Thus

$$P(A | +) \approx \frac{99}{99 + 999} \approx 0.09.$$

The positive signal raises concern and justifies further evidence, but it does not make the condition almost certain. The calculation depends on the stated population and test behavior; it should not be transferred without checking both.

[Illustrative teaching value]

5.1.2 Classification and the confusion matrix

A binary classifier maps an input x to a label $\hat{y} \in \{0, 1\}$, perhaps through a score $s(x)$ and threshold θ . Four outcomes are possible:

	Actual positive	Actual negative
Predicted positive	true positive (TP)	false positive (FP)
Predicted negative	false negative (FN)	true negative (TN)

Common measures are sensitivity $TP/(TP + FN)$, specificity $TN/(TN + FP)$, precision $TP/(TP + FP)$, and false-positive rate $FP/(FP + TN)$. These measures answer different questions. High sensitivity can coexist with poor precision for rare events. A single accuracy number hides the distribution of errors and the consequence of each type.

5.1.3 Scores, calibration, and thresholds

If a system emits $s(x) = 0.8$, a calibrated interpretation would be that cases receiving similar scores occur with the relevant outcome about 80% of the time in the declared reference population. Calibration must be checked empirically by grouping scores and comparing predicted with observed frequency. It can change when the population, sensor, environment, annotation rule, or model version changes.

A threshold creates a decision rule

$$\hat{y} = \begin{cases} 1, & s(x) \geq \theta, \\ 0, & s(x) < \theta. \end{cases}$$

Moving θ trades false positives against false negatives. A simplified expected-loss model is

$$L(\theta) = C_{FP}P(FP; \theta) + C_{FN}P(FN; \theta).$$

The model becomes more responsible when it includes review burden, delay, uncertainty, recovery, and the distinction between a technical alert and an authorized response. A cost function cannot erase a hard rule.

Selective prediction permits a classifier to refuse a label when input is ambiguous, out of distribution, incomplete, or outside its declared task. The result is a safe state or request for review, not an arbitrary forced label. Coverage is the fraction of inputs accepted; selective risk is the error among accepted inputs.

5.1.4 Expected utility and admissible action

Let A be a set of candidate responses and let Y be an uncertain state. A decision model may compare expected loss

$$\mathbb{E}[L(a, Y) \mid z] = \sum_y L(a, y)P(y \mid z).$$

This comparison is meaningful only over actions that are already admissible. Write

$$A_{\text{adm}} = \{a \in A \mid C_{\text{safety}} \wedge C_{\text{scope}} \wedge C_{\text{authority}} \wedge C_{\text{review}}\}.$$

The rule says that probability selects among permitted options; it cannot generate permission. This distinction is central to safety-critical system design and aligns with lifecycle risk-management practice in the NIST framework [14].

[title=Evidence-to-decision separation]

```
acquire input and record source, time, version, and missing fields
estimate label or state with confidence and calibration context
detect unknown, out-of-scope, or contradictory input conditions
assess task relevance separately from the classification label
remove responses that violate hard constraints or authority
compare remaining bounded responses and request review when needed
record evidence, uncertainty, alternatives, selected state, and owner
```

5.1.5 Distribution shift

An evaluation dataset is a sample from a population. If the deployment population changes because of sensor replacement, lighting, geography, weather, data processing, or user behavior, a score may no longer have the interpretation it had during evaluation. This is called distribution shift. A responsible design declares an operating envelope, monitors changes, tests challenge cases, and provides an abstention or protected behavior outside the envelope.

- Reading a classifier score as a direct probability without checking calibration and base rate.
- Treating correct classification as proof of task relevance.
- Choosing a threshold without assigning costs, review capacity, and failure consequence.
- Assuming two agreeing outputs are independent evidence without inspecting shared data, models, or clocks.
- Forcing unfamiliar inputs into a known label set instead of exposing uncer-

tainty.

5.1.6 Foundation self-check

1. Compute the posterior in a rare-event test when the base rate changes by one order of magnitude.
2. Identify TP, FP, TN, and FN in a four-case table.
3. Why can a calibrated score still fail to justify a response?
4. Give one condition under which a selective classifier should refuse a label.

The reasoning should separate posterior belief, task relevance, admissible action, and authorization. Probability informs a decision; it does not create permission.

5.1.7 Bridge to strategic relevance

The historical opening for this chapter asks whether the most visible event is necessarily the decisive system concern. The engineering lesson is not a tactical rule. It is the layered discipline of separating classification, confidence, relevance, hard constraints, authority, and bounded response. The chapter applies the probability tools above to that sequence.

5.2 Historical opening and source note

The story of relieving Zhao by changing the frame introduces a question about task relevance. A recognition system may answer one question correctly while the decision maker is asking another. This lecture therefore begins with the distinction between “what does the object resemble?” and “what decision is admissible under the current mission, environment, and authority?”

5.2.1 Foundational knowledge to master

An intelligent munition is treated at a public conceptual level as a munition or delivery system that uses onboard sensing, computation, guidance, or decision logic to adapt its behavior to a declared task. Its generic functional chain is launch or release interface, stabilization, sensing or seeker, signal processing, classification or tracking, guidance and control, terminal action logic, safety and authorization constraints, and post-event record. The chain is intentionally abstract and excludes construction details, operating parameters, targeting procedures, and deployment instructions.

Students compare guidance, navigation, and control roles; distinguish a sensor observation from a seeker estimate; and explain why a terminal decision should have

an explicit reject or protected path. Development trends are discussed through miniaturized processors, multimode sensing, onboard learning, networked updates, and increased need for verification.

5.2.2 Learning content and mathematical derivation

For hypothesis H and observation z ,

$$P(H | z) = \frac{P(z | H)P(H)}{P(z | H)P(H) + P(z | \overline{H})P(\overline{H})}.$$

Students calculate how the same classifier performance gives different posterior values under different base rates. A score $s(x)$ becomes a label only after a threshold θ is selected. The loss of a threshold is

$$L(\theta) = C_{\text{FA}}P_{\text{FA}}(\theta) + C_{\text{M}}P_{\text{M}}(\theta),$$

but the action set must first be filtered by hard constraints:

$$A_{\text{adm}} = \{a \in A \mid C_{\text{scope}} \wedge C_{\text{authority}} \wedge C_{\text{safety}}\}.$$

The derivation teaches three layers: evidence updates belief; loss compares permitted options; authority decides whether a transition may occur.

5.2.3 Case studio: a harmless recognition-and-refusal demonstrator

Students build a tabletop or simulated classification system for colored cards, recyclable objects, or laboratory markers. A small 嘉立创 EDA board can provide an RGB sensor, status LEDs, a push button for human confirmation, and a serial log. The device has no weapon interface and no physical actuator beyond an indicator.

The test set includes normal samples, unfamiliar samples, changed lighting, and contradictory labels. The system must display label, calibrated confidence range, sample condition, task scope, and one of ACCEPT_FOR_REVIEW, REQUEST_MORE_EVIDENCE, or REJECT_OUT_OF_SCOPE. Students compare accuracy, precision, recall, calibration, and refusal coverage.

5.2.4 Whole-chapter synthesis

The chapter's central separation is fourfold: the observation is not the estimate; the estimate is not the classification; the classification is not the relevance judgment; and relevance is not authorization. A technically successful classroom classifier remains incomplete until it can reveal its uncertainty, refuse an out-of-scope input, preserve its configuration, and request review.

5.2.5 Practice and extension

- 1. Compute a posterior probability for two different base rates.
- 2. Draw a four-layer chain from classification to authorization.
- 3. Construct a confusion matrix and calculate precision, recall, and false-alarm rate.
- 4. Explain why an out-of-distribution input should not be forced into a known class.
- 5. Write a decision card that records confidence, relevance, admissibility, and owner.

Basic term	Meaning	Question for this course
Prior	belief before the current ob- servation	How common is the state?
Likelihood	how compatible the observa- tion is with a state	Which measurement model is used?
Posterior	updated belief after the obser- vation	What uncertainty remains?
Threshold	boundary that changes a sys- tem state	What consequence follows?
Calibration	relation between score and ob- served frequency	Under which population?

5.3 Four layers that must not impersonate one another

An observation can support a classification. A classification can inform a relevance assessment. A relevance assessment can support a recommendation. A recommen-
dation can be authorized only by a valid authority under stated conditions. These
four layers answer different questions:

Layer	Question
Classification	Which known class best matches the input?
Task relevance	Does the state matter for this mission, place, time, and rule set?
Recommendation	Which bounded response best manages expected conse- quences?
Authorization	Who confirms conditions and accepts responsibility for the next step?

The lesson of the historical opening is exactly this separation. “Zhao is under pres-
sure” is a strong state description. “Move directly to Zhao” is a proposed path. “Re-

lieve the siege” is the mission objective. The statements are related, but none is identical to another.

5.4 What confidence means

For model M , label set K , and input x , a classifier may output

$$\hat{y} = \arg \max_{k \in K} p(y = k \mid x, M), \quad c(x) = \max_{k \in K} p(y = k \mid x, M).$$

If $c(x) = 0.99$, the strict reading is limited: within the model, the trained label set, and the input representation, one label has high relative support. The score does not contain a mission objective, a legal or organizational rule, an environment condition, or an authorization signature.

5.5 Unknown inputs and unequal errors

If $K = \{A, B, C\}$ but an input belongs to an unfamiliar state D , a closed-set classifier can still choose A, B, or C with high confidence. This is a distribution-shift problem. A practical system must retain an explicit “unknown / request review” path rather than forcing every input into a familiar label.

Accuracy alone is also insufficient because error consequences differ. Let actions be $A = \{\text{observe, verify, abort}\}$. A classroom risk model is

$$\mathcal{R}(a \mid x) = \sum_y p(y \mid x) L(a, y) + \lambda D(a),$$

where $L(a, y)$ is the consequence of action a when the true state is y , and $D(a)$ represents delay or resource cost. The model asks students to state consequences; it does not give a real system a permission to act.

5.6 Hard constraints and admissible actions

Some conditions cannot be outweighed by a high score. Define

$$A_{\text{adm}} = \{a \in A \mid C_{\text{env}} \wedge C_{\text{rule}} \wedge C_{\text{authority}} \wedge C_{\text{review}}\}.$$

Only after the admissible set is identified may a system compare the risks of its permitted responses. This is the decisive difference between an optimization result and a decision with a named responsible party.

Worked example

Three fictional safety-monitoring scenes all give $c(x) = 0.99$ for a restricted-area anomaly. In Scene A, an independent source is pending; in Scene B, location uncertainty is high; in Scene C, the rule window is closed and no authority is available. The confidence value is identical, but the admissible response may differ. A correct answer names the missing condition and the record owner, not merely a preferred action. [Illustrative teaching value]

5.7 Calibration and reliability diagrams

A classifier score becomes useful when it is calibrated. Let $s(x)$ be a score divided into bins $B_1, \dots, B_m$. A reliability table compares the average score in a bin with the observed frequency of the stated class:

$$\text{Rel}(B_j) = \left(\frac{1}{|B_j|} \sum_{x \in B_j} s(x), \frac{1}{|B_j|} \sum_{x \in B_j} \mathbf{1}\{y = \hat{y}\} \right).$$

If a group of cases receives scores near 0.8 but is correct only 0.55 of the time, the score is overconfident for that group. Calibration does not make a model correct; it makes the relationship between score and observed frequency more honest under the evaluated conditions.

A score is calibrated on a population when cases assigned score p satisfy the stated event at approximately frequency p , subject to sampling variation and the declared evaluation conditions. Calibration is population- and time-dependent.

5.8 Base rates and conditional reasoning

Suppose a fictional anomaly detector has sensitivity $P(+ | A) = 0.95$ and false-positive rate $P(+ | \neg A) = 0.05$. If the anomaly base rate is $P(A) = 0.01$, then

$$P(A | +) = \frac{0.95 \cdot 0.01}{0.95 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.161.$$

The values are illustrative. A strong detector can still produce a majority of false alarms when the event is rare. Interpret a high score with reference to its population and consequence model.

5.9 Thresholds and operating points

Let $s(x)$ be a score and θ a threshold. The decision rule

$$\hat{d}(x) = \begin{cases} 1, & s(x) \geq \theta, \\ 0, & s(x) < \theta \end{cases}$$

creates a trade-off between missed states and false alarms. Changing θ changes the operating point; it does not reveal a universally correct threshold. If the cost of a false alarm is C_{FA} and the cost of a missed state is C_M , a simplified expected loss is

$$L(\theta) = C_{FA}P(\hat{d} = 1, A = 0) + C_MP(\hat{d} = 0, A = 1).$$

The loss is incomplete until delay, review workload, uncertainty, and authorization are included.

Score state	What it can support	What it cannot support alone
Low score	routine monitoring or no alert	proof that no relevant state exists
Intermediate score	request for more evidence	automatic escalation
High score	a bounded recommendation, if calibrated	authorization or consequence by itself
Unknown / out of scope	safe state or human review	forced classification into a known class

5.10 Relevance is a separate inference

Even a correct label may be irrelevant to the current task. Let Y be a classified state and G the mission objective. Relevance asks about $P(\text{matters for } G \mid Y, \text{context})$, whereas classification asks about $P(Y \mid x)$. A system that skips this layer tends to convert detection into action.

Worked example: the same score, three decisions

Three fictional safety-monitoring scenes all give $c(x) = 0.99$ for a restricted-area anomaly. In Scene A, an independent source is pending, so the admissible response is verification. In Scene B, the location uncertainty is high, so the system marks the result as unresolved. In Scene C, the event is outside the declared task scope, so it is recorded and referred rather than acted upon. The classification score is identical; the decision is different because relevance, constraints, and

authority differ. [Illustrative teaching value]

5.11 Admissible actions and authority

Define

$$A_{\text{adm}} = \{a \in A \mid C_{\text{env}} \wedge C_{\text{rule}} \wedge C_{\text{authority}} \wedge C_{\text{review}}\}.$$

Only after the admissible set is identified may a system compare expected risks. This abstract formulation keeps one distinction visible: a high confidence value can select among permitted options but cannot create permission.

[title=Confidence-to-decision separation]

```
score <- classify(input)
calibration <- check_population_and_version(score)
if input_is_unknown_or_out_of_scope(input):
    action <- REQUEST_REVIEW
else:
    relevance <- assess_task_context(label, mission, rules)
    admissible <- filter_by_hard_constraints(actions, authority, review)
    action <- compare_expected_loss(admissible, relevance, calibration)
record(score, label, calibration, relevance, admissible, action, owner)
```

5.12 Distribution shift and challenge sets

A model can pass a test set and fail after a change in lighting, geography, sensor mounting, weather, population, or interface. A challenge set is a deliberately selected collection of cases that probes known weaknesses: near-threshold examples, unfamiliar conditions, ambiguous labels, conflicting sensors, and stale metadata.

For a score bin B , compare performance across conditions $c \in \mathcal{C}$:

$$\Delta_B = \max_{c \in \mathcal{C}} \text{err}(B, c) - \min_{c \in \mathcal{C}} \text{err}(B, c).$$

A large Δ_B means that one global confidence interpretation is unsafe. The remedy may be recalibration, a narrower operating envelope, more evidence, or a safe state.

Before accepting a classification output, ask: What is the label set? What is the reference population? How was the score calibrated? What happens to unknown inputs? Which task rule makes the output relevant? Which actions are admissible? Who authorizes them?

5.13 Historical and methodological interpretation

The account of relieving Zhao raises a question about objectives and relevance: the visible crisis need not define the only meaningful intervention. This chapter uses the analogy only until “what matters for the task” has been translated into a formal relevance layer and an admissible-action set.

5.14 Confusion matrices and selective prediction

For a binary teaching task, the confusion matrix records true positives, false positives, true negatives, and false negatives. A selective classifier adds a reject option r for inputs that are unknown, ambiguous, or outside scope. Its coverage is the fraction of inputs accepted for classification; its selective risk is the error rate among accepted inputs. Lower coverage can be responsible when it removes cases for which the model has no defensible basis.

Output	Meaning	Required next question
Class A	model selects a known label	Is the input in the evaluated population?
Class B	model selects another known label	Is the distinction relevant to the task?
Reject	model refuses or requests re-view	What evidence can resolve the uncertainty?
Out of scope	task or authority does not apply	Who owns the referral?

5.15 Decision card exercise

For one fictional case, write four cards: classification, relevance, recommendation, and authorization. A reviewer must be able to remove any one card and explain what downstream claim can no longer be made. This exercise makes the separation of layers visible and prevents a high score from becoming a hidden command.

5.16 Historical notes and further reading

Read the relevant *Shiji* passage before using the later idiom. For calibration and out-of-distribution detection, see [10, 11, 12]. For risk management and documentation, see [13, 15].

5.17 Exercises

- 5.1 Write a four-layer explanation of a fictional model output with confidence 0.95. Do not use a real weapon scenario.
- 5.2 Give two distinct reasons why a calibrated confidence value may still be insufficient for authorization.
- 5.3 Construct a 3×3 loss table for observe, verify, and abort under three fictional states. Explain each relative cost.
- 5.4 Define C_{env} , C_{rule} , $C_{\text{authority}}$, and C_{review} for a laboratory access-control example.
- 5.5 Prepare an evidence card that distinguishes the *Shiji* passage, your narrative reconstruction, and your engineering interpretation.

Chapter summary

- Confidence describes conditional support for a label; it does not define mission relevance or authorization.
- Unknown inputs and unequal error costs require explicit review and safe-response paths.
- Hard constraints delimit admissible actions before optimization begins.

Chapter 6

Intelligent Mines: Lifecycle Risk and the Ability to End

Narrative Prelude — The Horse That Did Not Settle the Question. The frontier tale of the old man whose horse was lost, returned, caused injury, and later prevented conscription is often reduced to a slogan about fortune. Its deeper educational use is narrower: a local outcome does not settle a longer causal chain. A system that is beneficial at one moment may remain a risk because it cannot be located, withdrawn, disabled, or explained later.

Source note. The story is a philosophical parable in *Huainanzi*, “Renjian Xun” (P), not a verified biography. The lifecycle diagrams and examples are classroom examples (C). This chapter addresses safety, responsibility, and risk management; it does not teach deployment or evasion of any weapon.

Learning objectives

1. model a system over design, integration, deployment, operation, withdrawal, and residual-risk stages;
2. distinguish mission completion, platform shutdown, policy revocation, and verified end-of-life;
3. formulate a time-dependent residual-risk representation;
4. explain why record reconstruction and disablement are lifecycle requirements.

Lifecycle analysis begins with a simple distinction between a stage and a state. A stage is an organizational period such as design, integration, operation, or withdrawal. A state is a technical condition such as active, degraded, revoked, or verified end state. Risk is commonly discussed as a combination of likelihood and consequence, but the exact scale is a modeling convention. Failure Mode and Effects Analysis (FMEA) asks what can fail, what effect follows, how likely it is, how detectable it is, and which control reduces the risk. NASA’s public systems-engineering material connects requirements, verification, configuration,

and lifecycle reviews [16]; NIST provides a public risk-management vocabulary for identifying, measuring, and treating AI-related risks [14].

6.1 Foundations: lifecycle, hazards, and verification evidence

A system can appear to work at a demonstration and still be unready for responsible use. It may have untested interfaces, unclear withdrawal behavior, an unverified update path, or no record of who accepts residual risk. Systems engineering responds by treating development, operation, maintenance, retirement, and learning as a connected lifecycle rather than a single performance test [16].

6.1.1 Lifecycle stages and the V-model intuition

A simplified lifecycle starts with stakeholder needs, translates them into requirements and architecture, develops components and interfaces, then checks the work through integration, verification, validation, operation, maintenance, and retirement. The V-model is a useful picture: decomposition on the left is paired with progressively broader integration and evidence on the right. The picture is not a command to use one project-management method; it is a reminder that every design statement must eventually meet an evidence question at the corresponding level.

Design question	Matching evidence question	Example artifact
What need is being addressed?	Did the system address the intended need?	validation scenario
What shall the system do?	Was each requirement met?	verification matrix
How are functions allocated?	Do interfaces preserve intended meaning?	integration test
How is a component built?	Does it meet its local contract?	unit test
How is it changed or retired?	Is the final state bounded and recorded?	release or withdrawal record

6.1.2 Hazard, risk, and control

A hazard is a potential source of harm. Risk is commonly treated as a combination of consequence and likelihood, but the simple product

$$R = S \times P$$

is only an early screening device. Severity S , likelihood P , exposure duration, detectability, reversibility, and uncertainty may all matter. A risk estimate must identify its scale and evidence; multiplying arbitrary ordinal labels does not make a precise probability.

A control reduces risk by preventing a hazardous condition, detecting it, limiting its consequence, or preserving recovery. Controls can be technical, procedural, organizational, or evidentiary. A warning display may detect a condition; a hard state constraint may prevent a transition; an independent review may prevent an unsupported authorization. Mature designs do not rely on one control when a common failure can disable it.

Verification asks whether a specified requirement has been satisfied: “Did we build the system right?” **Validation** asks whether the requirement and resulting system serve the intended need in the intended context: “Did we build the right system?” A test can verify a component and still fail to validate the combined technical and human process.

6.1.3 Failure modes and effects analysis

Failure modes and effects analysis (FMEA) begins with a function or component and asks how it can fail, what effect follows, how the failure can be detected, and which control reduces the risk. A simple FMEA row has columns for item, function, failure mode, local effect, system effect, cause, detection, control, evidence, and owner. The method is useful not because a score produces truth but because it prevents the analysis from stopping at “component failed.”

Worked example: a fictional status-message path

For a laboratory monitoring system, consider the function “report current status.” One failure mode is a timestamp that is present but stale. The local effect is that a message looks complete. The system effect is that a reviewer may interpret an old state as current. Possible causes include a clock mismatch or queued delivery. Detection can compare message age with a declared validity window. Controls include time synchronization checks, age display, and a rule that stale status cannot authorize a transition. The analysis shows why a perfectly formatted message can still be hazardous. [Illustrative teaching value]

6.1.4 Fault trees and causal structure

Where FMEA starts from a component failure, fault-tree analysis starts from an undesired top event and decomposes possible causal combinations. An OR gate says any

listed cause can produce the event; an AND gate says several causes must occur together. If independent basic-event probabilities are p_1 and p_2 , an AND approximation is $p_1 p_2$, while an OR approximation is $p_1 + p_2 - p_1 p_2$. The independence assumption must be defended. Shared power, shared software, shared map, or shared organizational pressure can create common-mode failures that defeat the arithmetic.

6.1.5 Evidence gates and configuration management

Every lifecycle transition should have an evidence gate. Before moving from development to an evaluation scenario, for example, a team can require a declared configuration, test procedure, expected result, observed result, anomaly record, and reviewer. A configuration is more than software version. It can include hardware, data, model weights, thresholds, maps, rules, calibration records, and interface definitions. If any of these changes, previous evidence may need re-evaluation.

Let κ denote configuration and $E(\kappa)$ the evidence valid for it. The statement “the system passed” should be replaced by “the stated requirement was evaluated under configuration κ using evidence $E(\kappa)$.” This wording is longer but makes change impact assessable.

[title=Lifecycle evidence gate]

```
state the need, requirement, scope, and non-goals
identify hazards, assumptions, owners, and recovery conditions
freeze or record the configuration to be evaluated
run unit, interface, integration, and scenario tests at stated levels
record deviations, unresolved claims, and residual risk
accept, restrict, revise, or withdraw the configuration with a named sign-off
```

6.1.6 Change, regression, and retirement

An update that fixes one component can alter the behavior of an interface, estimator, threshold, or user workflow. Regression testing checks previously supported claims after a change. The appropriate scope depends on the changed assumption, not merely on the number of changed code lines. Retirement also requires evidence: credentials revoked, stored policy disabled, physical state verified, records preserved, and unresolved hazards transferred to an owner.

- Treating a prototype demonstration as lifecycle validation.
- Using a risk score without the assumptions behind severity and likelihood.
- Recording a test outcome without the exact configuration and test conditions.

- Closing a task when visible motion stops, while policies, credentials, records, or maintenance obligations remain active.
- Calling an updated model “the same system” without examining which evidence is still applicable.

6.1.7 Foundation self-check

1. Give one example of verification and one example of validation for the same fictional requirement.
2. Build one FMEA row for a stale status message.
3. Why can an update to a threshold require regression testing outside the threshold module?
4. Name two pieces of evidence needed before declaring a verified end state.

The expected answer treats lifecycle evidence as configuration-specific. A finished visible task is not necessarily a closed system obligation.

6.1.8 Bridge to the old man and the lost horse

The philosophical story is an invitation to avoid judging a system at one moment only. Its technical translation is lifecycle thinking: a local success can create later obligations, and a local failure can reveal an untested assumption. The chapter applies hazard analysis, evidence gates, configuration management, and verified withdrawal to a fictional safety-critical system.

6.2 Historical opening and source note

The story of the old man and the lost horse introduces time-dependent risk. A local benefit cannot settle the value of a system whose state may persist after the mission, whose location may be uncertain, or whose future interaction with civilians and the environment cannot be reconstructed. This lecture treats lifecycle completion as an engineering function.

6.2.1 Foundational knowledge to master

At a conceptual level, an intelligent mine is discussed as a persistent or semi-persistent hazard system with sensing, decision, activation or inhibition logic, power management, communication or marking, and some declared means of status change or removal. The important question is not how to make such a system effective. It is how a

designer can represent safe states, known location, identity, expiry, self-disablement, recovery, and environmental impact.

The lifecycle is

design $\rightarrow$ manufacture $\rightarrow$ deployment $\rightarrow$ active state
 $\rightarrow$ maintenance $\rightarrow$ withdrawal $\rightarrow$ environmental recovery.

Students should distinguish active, dormant, inhibited, expired, failed, recovered, and unknown states. “Task complete” is not equivalent to “risk removed.”

6.2.2 Learning content and mathematical derivation

Let hazard severity be $S(t)$, exposure probability $P(t)$, detectability factor $D(t)$, and recovery factor $R(t)$. A classroom residual-risk index is

$$\mathcal{R}(t) = S(t)P(t)D(t)(1 - R(t)).$$

This is not a field risk standard. It is a transparent teaching model that shows why risk can remain after an apparent shutdown. Students must state which factors are measured, which are estimated, and which are ordinal.

For lifecycle states q_k , write

$$q_{k+1} = \delta(q_k, \sigma_k, \kappa_k),$$

and require that an end-of-life transition has evidence for identity, location, authority revocation, energy state, policy state, and record integrity. If any premise is missing, the system enters UNKNOWN or REVIEW rather than silently claiming safe withdrawal.

6.2.3 Case studio: persistent environmental sensor lifecycle board

For a safe laboratory exercise, students implement an environmental-monitoring analogue. A 嘉立创 EDA board contains a temperature or soil-moisture sensor, a low-power microcontroller, a real-time clock, a status LED, a tamper or removal input, and a data record interface. The board must demonstrate active, dormant, expired, unknown, and recovered states. It must not contain a trigger, hazardous payload, or deployment mechanism.

The lifecycle experiment compares energy use, status visibility, expiry behavior, and recovery evidence. Students write an FMEA row for stale identity, lost location record, clock failure, and failed disablement. They then propose a control and a verification test for each hazard.

6.2.4 Whole-chapter synthesis

A mature system preserves the conditions under which it should reduce function, suspend operation, withdraw, record residual hazards, and hand responsibility to the next lifecycle owner. In this chapter, the essential intelligent behavior is the ability to end in a known and reviewable condition.

6.2.5 Practice and extension

1. Draw a lifecycle state diagram and define the evidence required at each transition.
2. Calculate how residual risk changes when exposure or detectability changes.
3. Construct an FMEA table with cause, effect, detection, control, and owner.
4. Explain why configuration management continues after a device is physically inactive.
5. Write a verified withdrawal checklist that includes environmental recovery.

Verification asks whether the system was built according to its specified requirements. Validation asks whether the specified system addresses the intended need in its intended context. A platform can pass verification and still fail validation if the requirement was incomplete or the mission context changed.

6.3 A system can finish its task and remain active

“Mission complete” is not the same as “system inactive.” A platform may be powered down while data, policy, stored state, or an external effect remains active. A lifecycle model should therefore record at least the following states:

design → integration → operation → withdrawal → verified end state.

Transitions must include evidence: a checklist, a location confirmation, a version record, a physical recovery record, or a cryptographically signed revocation, depending on the system boundary.

6.4 Risk accumulates across time

Let $B(t)$ represent a time-dependent benefit and $R(t)$ residual risk. An illustrative whole-life criterion is

$$J = \int_{t_0}^{t_f} (\alpha R(t) - \beta B(t)) dt + \gamma R_{\text{post}},$$

where R_{post} is the residual risk after the nominal mission. The equation discourages a narrow comparison of only immediate benefit. Its variables and weights are design choices, not physical constants.

6.5 State, policy, and responsibility

Consider a fictional safety-critical device with three independent records: platform power state p , policy state ρ , and verification state v . A safe end condition might require

$$p = \text{off} \wedge \rho = \text{revoked} \wedge v = \text{verified}.$$

Turning off a visible interface may change neither ρ nor v . This is why responsible systems need an explicit end-of-life argument rather than a mere “off” button.

Worked example

An old facility access rule remains active after the associated project ends. The interface is removed, but the underlying identity policy still accepts a dormant credential. The example is fictional, yet it reveals a lifecycle failure: the organization closed an interface, not a state. [Illustrative teaching value]

6.6 Lifecycle stages and evidence gates

The lifecycle is not a straight line in which design ends when operation begins. It is a sequence of evidence gates. At each gate, the organization asks whether the system remains within its declared purpose and whether the next stage has an owner.

Stage	Main question	Minimum evidence artifact
Design	What is the intended boundary?	architecture, assumptions, hazard list
Integration	Do components pre-serve the contract?	interface tests, configuration record
Validation	Does the model support the declared task?	test protocol, limitations, challenge set
Deployment	Are conditions and authority present?	readiness review, operator assignment
Operation	Is the current state inside the envelope?	logs, alerts, intervention records
Withdrawal	Has permission and access ended?	revocation, recovery, inventory record
End state	Can the claim be independently checked?	verification report and sign-off

A lifecycle gate is a decision point at which evidence is reviewed before a system changes from one organizational or technical stage to another. A gate is meaningful only if the reviewer can reject, defer, or require remediation.

6.7 Hazards, causes, and controls

A hazard is a condition with potential for harm; a failure mode is a way a component or process can fail; a control is a measure that reduces likelihood, consequence, or detectability. These terms should not be collapsed. “Bad outcome” is too vague to guide design.

For a fictional inspection system, a hazard might be “the platform continues an outdated task after the mission has been withdrawn.” Possible causes include a stale policy cache, a missed revocation message, an ambiguous state label, or a missing end-of-life check. Controls include signed policy expiry, an explicit safe state, independent verification, and a record that can be audited after a link outage.

Hazard		Cause	Preventive control		Detective control	
Stale authority		policy not synchro- nized	bounded	validity	periodic	authority
			interval		check	
Unknown	loca- tion	navigation uncer- tainty grows	recovery margin		independent	loca- tion record
Unverified	up- date	version changes during operation	configuration lock		post-change audit	
Incomplete	with- drawal	interface removed only	end-state checklist		independent	sign- off

6.8 Residual risk over time

Let $R(t)$ represent residual risk and $B(t)$ time-dependent benefit. An illustrative whole-life criterion is

$$J = \int_{t_0}^{t_f} (\alpha R(t) - \beta B(t)) \, dt + \gamma R_{\text{post}}.$$

The post-mission term prevents the analysis from stopping at the moment of visible success. A decision that looks attractive during a short window may be poor when storage, maintenance, policy, data, and residual access are included.

The model also teaches an important limitation: risk is not a single natural number waiting to be discovered. The values of α , β , and γ express priorities. A responsible analysis therefore reports the assumptions and shows how the conclusion changes when the weights change.

Worked example: three meanings of “end”

An instructional platform completes its route at 14:00. At 14:01, its motors are stopped. At 14:02, the local policy is still valid. At 14:03, the record server has not confirmed withdrawal. The platform has reached a physical stop, but the organization has not yet established a verified end state. Students should write three separate claims: motion ended, authority ended, and lifecycle ended. Each claim needs different evidence. [Illustrative teaching value]

6.9 Change management and regression

A lifecycle is also a history of changes. Let configuration κ_j be replaced by κ_{j+1} . The change impact is not determined only by the edited file. A minor interface change can affect timestamps, logs, thresholds, or human interpretation. A practical change record includes:

1. the reason for the change;
2. components and assumptions that may be affected;
3. tests selected and tests explicitly omitted;
4. the authority that approved the change;
5. the rollback or protected-state plan;
6. the evidence that the new configuration is the one actually running.

[title=Withdrawal and end-state verification]

```
freeze current configuration and record final state
stop the active task through an authorized transition
revoke or expire the relevant policy
verify location, stored data, credentials, and external interfaces
obtain an independent confirmation of each required end condition
mark unresolved conditions as residual risk with an owner
publish the end-state record and archive the evidence
```

6.10 Post-event learning

An incident review should not ask only “Who made the last mistake?” It should ask which model, interface, training assumption, organizational pressure, or missing record made the mistake likely. The old-man parable is useful here because a local event does not settle the whole causal chain. A favorable immediate outcome can hide a bad process; an unfavorable outcome can reveal a previously invisible control.

A lifecycle review is incomplete when it has no answer to: Who can stop the system? Who can revoke its authority? Who can verify its location and state? Which records remain after shutdown? How are unresolved residual risks transferred to a named owner?

6.11 Historical and methodological interpretation

The parable in *Huainanzi* should not be read as a claim that harm inevitably becomes benefit, or that uncertainty excuses inaction. Its instructional value is the widening of the time horizon. Engineering analysis begins when the class defines states, transitions, evidence gates, and residual risk.

6.12 FMEA-style lifecycle analysis

A failure-mode table can connect lifecycle stages to causes, effects, controls, and evidence. A classroom priority score may be written as $P = S \cdot O \cdot D$, where S is consequence severity, O occurrence likelihood, and D difficulty of detection. The score is a screening aid, not a universal risk truth. It should never replace a hard prohibition or a safety case.

Failure mode	Effect	Preventive control	Evidence
Revocation not received	stale policy remains	bounded validity and retry	acknowledgment or expiry log
End location uncertain	residual state unknown	recovery and independent check	location record
Version mismatch	review applies to wrong system	configuration lock	signed version report
Record incomplete	causal chain cannot be reconstructed	mandatory fields	audit replay

6.13 Withdrawal test

Design a test that begins with a nominal mission-complete state and checks whether platform state, policy state, credential state, stored data, location, and record state all reach their declared end conditions. The test should include a missed revocation message and a stale interface. A system passes only if it enters a safe state or explicitly transfers the unresolved risk to an owner.

6.14 Historical notes and further reading

The parable should not be read as an argument that harm inevitably becomes benefit, or that uncertainty excuses inaction. It invites a wider time horizon. Use [13, 15] for lifecycle risk management and configuration-management literature for version and withdrawal control.

6.15 Exercises

- 6.1 Draw a lifecycle diagram for a fictional autonomous inspection system. Add one evidence artifact to every transition.
- 6.2 Explain the difference between platform shutdown, policy revocation, and verified end state.

- 6.3 Sketch two curves $B(t)$ and $R(t)$ for a system with short-term utility and long-term residual risk.
- 6.4 Design a checklist for a safe withdrawal process. Include at least six items and an owner for each.
- 6.5 Use the evidence-boundary protocol to explain why the frontier story is a parable rather than a lifecycle data source.

Chapter summary

- A system's lifecycle continues after its visible mission ends.
- Residual risk, state revocation, location, version, and verification must be designed from the start.
- "Off" is not a safety claim unless the relevant states and records have actually changed.

Part II

Uncertainty, Communication, and Integrated Responsibility

Terminally Sensitive Munitions: Thresholds and Time-Critical Decisions

Narrative Prelude — Bian Que’s Repeated Warning. The story of Bian Que and Duke Huan of Cai is a sequence of weak signals rejected until the condition becomes irreversible. It is a moral and rhetorical story, not a clinical record. Its engineering value is temporal: the most consequential moment may be the last moment at which a system can still change state safely.

Source note. The story appears in *Han Feizi*, “Explaining Lao” (P/R). It is not independent medical evidence. Threshold curves, detection values, and terminal-decision cases below are classroom constructs (C).

Learning objectives

1. define detection, false alarm, missed state, and decision threshold;
2. explain the trade-off between early intervention and unnecessary interruption;
3. represent time-to-action and irreversibility in a decision model;
4. design an escalation policy with observe, verify, degrade, and terminate states.

A threshold converts a continuous or noisy measurement into a discrete decision. The beginner’s model is a number line: values below a boundary and values above a boundary receive different treatment. Noise means that a measurement can move across the boundary even when the underlying condition has not changed. This motivates hysteresis, persistence timers, and multi-level escalation. The probability of a false alarm and the probability of a missed state are different quantities, and neither can be interpreted without a population, a base rate, and a consequence model.

7.1 Foundations: signals, thresholds, and time-critical review

An alarm is not a fact; it is the output of a detection rule under noise, uncertainty, and a time budget. This module introduces signal-detection vocabulary so that later discussions of warning states do not collapse score, evidence, urgency, and authority into one red indicator.

7.1.1 Signal plus noise

Let an observed scalar be $z = s + n$, where s is a signal component and n is noise. The decomposition is a model, not a literal separation available to a sensor. Noise can arise from measurement error, environment variation, timing jitter, preprocessing, or a mismatch between the model and the situation. A signal-to-noise ratio is often written as

$$\text{SNR} = \frac{\text{signal power}}{\text{noise power}},$$

or in decibels as $10 \log_{10}(\text{SNR})$. It summarizes separability under stated conditions but does not decide the cost of an error.

Sampling converts a changing quantity into observations at times t_k . If sampling is too slow relative to the phenomenon, rapid changes can be missed or misrepresented. A timestamp is therefore part of a sample, as are the sensor configuration and quality indicators. A later chapter treats information age in more detail.

7.1.2 A threshold is a policy parameter

Suppose a detector produces score $s(z)$ and triggers an alert when $s(z) \geq \theta$. Two distributions describe the score under hypotheses H_0 (condition absent) and H_1 (condition present). The threshold produces a false-alarm probability

$$P_{\text{FA}} = P(s(z) \geq \theta \mid H_0)$$

and a detection probability

$$P_{\text{D}} = P(s(z) \geq \theta \mid H_1).$$

Increasing the threshold usually lowers both false alarms and detection probability. The receiver-operating characteristic (ROC) curve plots P_{D} against P_{FA} as the threshold changes. It is a description of a test under an evaluation population, not a command to select one operating point.

A **false alarm** occurs when a detector signals a condition that is absent under the declared reference label. A **miss** occurs when the condition is present but

the detector does not signal. An **operating point** is the selected threshold and resulting trade-off under a specific population, cost model, and review process.

7.1.3 Costs, review capacity, and escalation

A simple decision model compares expected cost:

$$R(\theta) = C_{\text{FA}} P_{\text{FA}}(\theta) + C_{\text{M}} P_{\text{M}}(\theta),$$

where $P_{\text{M}} = 1 - P_{\text{D}}$. But a real review chain has more terms: delayed evidence, reviewer workload, alert fatigue, communication delay, reversibility, and uncertainty in the score. A single alert threshold is often too crude. A more transparent policy contains stages such as MONITOR, VERIFY, DEGRADED, and PROTECTED, each with a separate evidence requirement and action boundary.

Worked example: why a moderate alert may still require a protected response

In a fictional laboratory, a monitor reports score 0.64 for an abnormal condition. The classifier is only moderately reliable near this score; manual review takes four minutes; and the estimated time until a reversible window closes is three minutes. A sensible response may be to enter a bounded protective state while requesting review. This does not claim the condition is certainly present. It recognizes that waiting for certainty can be a decision with its own cost. [Illustrative teaching value]

7.1.4 Sequential evidence

One observation may be weak; a sequence can be more informative if its dependence is understood. Under a simplified independent-observation model, the likelihood ratio after observations $z_1, \dots, z_k$ is

$$\Lambda_k = \prod_{i=1}^k \frac{P(z_i | H_1)}{P(z_i | H_0)}.$$

In practice it is often easier to add log-likelihood ratios. A sequential test can compare the accumulated evidence with upper and lower thresholds: one supports a declared escalation, one supports a declared de-escalation, and the region between them requests more evidence. Independence must not be assumed when samples share the same sensor fault, preprocessing pipeline, or environmental disturbance.

7.1.5 Time margin and irreversible transitions

Let T_{avail} be the estimated time until a condition becomes irreversible, T_{detect} detection delay, T_{verify} confirmation delay, T_{decide} authorization delay, and T_{act} time to enter a

bounded response. A simple margin is

$$M = T_{\text{avail}} - (T_{\text{detect}} + T_{\text{verify}} + T_{\text{decide}} + T_{\text{act}}).$$

If $M < 0$, a policy that waits for all normal stages cannot meet the stated window. The correct engineering response is not to hide the negative margin; it is to redesign the system, adjust sensing, specify a conservative state, reduce workload, or limit the task scope.

7.1.6 Human factors and alarm fatigue

An alert has a human receiver. High false-alarm volume can reduce attention, slow response, or encourage workarounds. Conversely, very rare alerts may be unfamiliar when they occur. The human interface should display source, time, confidence, uncertainty, consequence, recommended bounded state, and the information needed for confirmation. An unexplained red warning forces the operator to infer the very context the design should preserve.

- Selecting a threshold from accuracy alone while ignoring base rates and unequal error costs.
- Treating a detector score as the same thing as an authorization.
- Counting repeated samples as independent corroboration when they share a fault source.
- Designing only an escalation threshold and no de-escalation, expiry, or recovery rule.
- Measuring detector latency while omitting the verification, authorization, and action delays that consume the actual time window.

7.1.7 Foundation self-check

1. Write the false-alarm probability and detection probability for a threshold θ .
2. Explain why the ROC curve does not itself select an operating point.
3. Compute the sign of a time margin when available time is shorter than the full review chain.
4. Give one reason repeated observations may not be independent evidence.

The desired answer makes delay and consequence part of the policy. A threshold is a declared operating choice, not a natural boundary discovered by the sensor.

7.1.8 Bridge to Bian Que's early diagnosis

The historical and literary image of seeing a problem early is a prompt about time, not proof that an alert should compel action. The engineering translation is a disciplined chain: sample and timestamp evidence, characterize errors, select staged thresholds, preserve a time margin, and provide a bounded response when the evidence or time window is insufficient.

7.2 Historical opening and source note

The story of Bian Que seeing illness early is used to teach the accumulation of weak signals. A terminally sensitive munition is introduced through the tension between late, high-resolution evidence and the shrinking time available for a safe decision. The lecture should not imply that the last observation automatically has the highest authority.

7.2.1 Foundational knowledge to master

At the public conceptual level, a terminally sensitive munition is a system whose terminal-stage sensing and logic are intended to distinguish a relevant object or condition near the end of a trajectory or approach. Its basic functional blocks may be described as release and stabilization, terminal sensing, signal conditioning, object or condition discrimination, tracking or aim-point estimation, guidance and control, terminal logic, safety constraints, and record generation.

The emphasis is on functional relationships and timing. The course does not teach seeker construction, target libraries, fusing parameters, or deployment procedures. Development trends can be discussed through multispectral sensing, smaller sensors, onboard computation, better uncertainty representation, and more explicit abort or reject logic.

7.2.2 Learning content and mathematical derivation

For threshold θ and score $s(z)$,

$$P_{\text{FA}}(\theta) = P(s(z) \geq \theta \mid H_0), \quad P_{\text{D}}(\theta) = P(s(z) \geq \theta \mid H_1).$$

For sequential evidence,

$$\Lambda_k = \sum_{i=1}^k \log \frac{P(z_i \mid H_1)}{P(z_i \mid H_0)}.$$

An escalation policy can use an upper threshold, lower threshold, and an intermediate request-for-review region. The decision-time margin is

$$M = T_{\text{avail}} - (T_{\text{detect}} + T_{\text{verify}} + T_{\text{decide}} + T_{\text{act}}).$$

Students should analyze what happens when $M < 0$: a late certainty claim cannot compensate for an impossible review chain.

7.2.3 Case studio: a tabletop terminal-sensing simulator

Students use a simulated moving marker and a safe optical or ultrasonic sensor to study threshold decisions. The marker may represent an object category in a laboratory track; no weapon function is involved. The simulator varies noise, threshold, sensor delay, sample age, and time-to-window-close. A small 嘉立创 EDA board may provide the sensor interface, timer, LEDs, and emergency stop.

The report must show an ROC curve, a confusion matrix, a sequential evidence trace, and a time-margin calculation. Students must design a policy with `CONTINUE_MONITOR`, `REQUEST_REVIEW`, `SAFE_HOLD`, and `TERMINATE_SIMULATION` states.

7.2.4 Whole-chapter synthesis

The system must not confuse the newest observation with the whole case. Sequential evidence works only when timestamp, source condition, delay, and competing explanations are retained. When the time margin becomes negative, the appropriate technical result is not false certainty; it is an explicit safe state and a documented explanation.

7.2.5 Practice and extension

1. Derive the false-alarm and detection probabilities for a selected threshold model.
2. Compare one high-confidence late observation with several weaker early observations.
3. Calculate the time margin under two different verification delays.
4. Design hysteresis and persistence rules for a noisy terminal score.
5. Explain why a new observation may challenge, rather than automatically overwrite, earlier evidence.

Outcome	Statistical name	Engineering response
Alarm when condition is absent	false positive / false alarm	review burden and trust cost
No alarm when condition is present	false negative / missed state	lost decision time
Alarm when condition is present	true positive / detection	bounded response with evidence
No alarm when condition is absent	true negative	continue monitoring

7.3 Signals, decisions, and consequences

Suppose a scalar evidence statistic is $s(x)$ and a decision threshold is θ . The simple rule

$$s(x) \geq \theta \Rightarrow \text{alarm}$$

is useful only after the meanings of “alarm” and the consequences of error have been stated. Decreasing θ can improve detection sensitivity while increasing false alarms. Increasing θ can reduce false alarms while increasing missed states. The selection is not a purely mathematical preference.

7.4 The time window

Let t_c be the time at which a system enters an irreversible region and t_r be the time required for review, communication, and response. A necessary safety condition is

$$t_{\text{now}} + t_r < t_c.$$

If the condition fails, waiting for stronger evidence may be incompatible with the remaining decision window. A responsible design makes the window and the required response time visible before a crisis occurs.

7.5 Escalation rather than a single alarm

A four-level policy may use thresholds $\theta_1 < \theta_2 < \theta_3$:

$s < \theta_1$	routine observation
$\theta_1 \leq s < \theta_2$	collect corroborating evidence
$\theta_2 \leq s < \theta_3$	degrade or request review
$s \geq \theta_3$	safe termination or safe state

The labels are intentionally abstract. A real system requires domain-specific validation, authority rules, and a human factors analysis.

Worked example

A laboratory coolant-monitoring system shows a slowly rising anomaly score. A single high threshold delays false alarms but leaves too little time for manual review. Design an escalation policy that moves from observation to verification to safe shutdown. State what measurement uncertainty would change your thresholds. [Illustrative teaching value]

7.6 Detection theory in a classroom model

Let H_0 denote a normal condition and H_1 a condition requiring review. An evidence statistic s is compared with threshold θ . The four outcomes are:

Truth and decision	Name	Consequence question
H_0 , no alarm	correct rejection	Is monitoring still adequate?
H_0 , alarm	false alarm	What review burden and trust cost follow?
H_1 , alarm	detection	Is the response bounded and authorized?
H_1 , no alarm	missed state	How much decision time is lost?

The notation is deliberately neutral. The same table can describe a laboratory coolant monitor, a network health monitor, or a fictional inspection task. The meaning of H_1 must be declared locally; a detector cannot define the consequence of its own output.

7.7 Receiver-operating characteristics

For a threshold θ , define

$$\text{TPR}(\theta) = P(s \geq \theta \mid H_1), \quad \text{FPR}(\theta) = P(s \geq \theta \mid H_0).$$

As θ changes, the pair (FPR, TPR) traces an operating curve. A curve can compare thresholds, but it cannot decide which point is acceptable. The acceptable point depends on base rate, review capacity, time window, authority, and consequence.

A decision threshold is a declared boundary in an evidence statistic that changes the system’s permitted state or initiates a review path. It is not merely a number chosen to maximize accuracy; it is part of a policy and assurance argument.

7.8 Sequential evidence and update discipline

A sequence of weak signals should not be treated as either one strong signal or as independent proof without checking their relationship. Let the log-likelihood contribution at time k be λ_k . A sequential score can be represented by

$$S_n = S_0 + \sum_{k=1}^n \lambda_k.$$

The system may use lower and upper boundaries to enter review or safe states, while remaining in an observation state between them. The formula is a teaching model. Its purpose is to make accumulation, persistence, and stopping visible.

Worked example: the last reversible minute

Suppose a fictional coolant monitor needs four minutes for review and safe shut-down. At $t = 6$ minutes, the anomaly score is moderate and the system requests a second measurement. At $t = 8$ minutes, the score is unchanged but the remaining time is now too short for the original review path. The correct state transition may be a protected shutdown, not because confidence suddenly became high, but because the feasible response set changed with time. [Illustrative teaching value]

7.9 Time-to-action and irreversibility

Let t_c be the time at which a system enters an irreversible region and t_r the time required for review, communication, and response. A necessary safety condition is

$$t_{\text{now}} + t_r < t_c.$$

More realistically, t_r is uncertain. If T_r is a random variable, one may require

$$P(t_{\text{now}} + T_r < t_c) \geq 1 - \varepsilon.$$

The parameter ε is a policy choice. A design that reports only the mean response time may hide a long tail that consumes the remaining window.

7.10 Escalation policy as a state machine

Thresholds should map to states, not directly to unbounded actions. A four-level policy may use $\theta_1 < \theta_2 < \theta_3$:

$s < \theta_1$	routine observation
$\theta_1 \leq s < \theta_2$	collect corroborating evidence
$\theta_2 \leq s < \theta_3$	degrade or request review
$s \geq \theta_3$	safe termination or safe state

Each row must include entry evidence, exit evidence, a timer, an owner, and a record. The labels are intentionally abstract. They express a pattern for reversible escalation.

```
[title=Time-aware threshold logic]
```

```
score <- update_evidence()
time_margin <- estimate_time_to_irreversibility() - response_time_margin()
if score_is_out_of_scope(score) or time_margin <= 0:
  state <- PROTECTED
elif score >= upper_threshold:
  state <- REVIEW_OR_SAFE_TERMINATION
elif score >= review_threshold:
  state <- VERIFY
else:
  state <- OBSERVE
record(score, thresholds, time_margin, state, authority, version)
```

7.11 Trust, alarm fatigue, and human factors

Repeated false alarms change the receiver. Operators may delay responses, mute alerts, or rely on a second interface that has its own uncertainty. Alarm performance depends on the detector, frequency, salience, explanation, workload, and recovery.

When setting a threshold, document the false-alarm cost, missed-state cost, base rate, calibration population, review time, authority rule, expiry behavior, and the human action expected at each state. A threshold without an action path is only a number on a graph.

7.12 Historical and methodological interpretation

The literary force of the Bian Que story comes from repeated refusal to update a judgment. Modern detection theory requires data, assumptions, and error models; it does not inherit its threshold from a parable. The story opens the temporal question, while the model makes the time window and the cost of delay inspectable.

7.13 Sequential thresholds and competing hypotheses

Let H_0 and H_1 be two competing hypotheses. A sequential evidence score S_n can be compared with a lower boundary L and upper boundary U . Between the boundaries the system remains in an information-gathering state. The lesson is not to choose a magic boundary; it is to define what evidence permits a state transition and what happens when the window closes first.

Condition	State	Record requirement
$S_n < L$	continue observation	current evidence and next observation
$L \leq S_n \leq U$	verify or request review	alternatives and time margin
$S_n > U$	enter a bounded protected response	threshold version and authority
Time margin exhausted	safe state	unresolved uncertainty and owner

7.14 Alarm fatigue exercise

Construct two policies with the same detector: one produces frequent low-level alerts, the other uses a staged escalation. Compare operator workload, review delay, and the risk of ignoring later alarms. This exercise treats alarm quality as a property of the whole interface and workflow, not of the classifier alone.

7.15 Historical notes and further reading

The literary force of the Bian Que story comes from repeated refusal to update a judgment. Modern detection theory requires data, assumptions, and error models; it does not inherit its threshold from a parable. For probability, filtering, and risk management, see [8, 13, 15].

7.16 Exercises

- 7.1 Define false alarm and missed state for a fictional laboratory monitoring system. Which is more costly, and why?
- 7.2 Draw a qualitative receiver-operating-characteristic curve and mark two candidate thresholds.
- 7.3 Given $t_c = 12$ minutes and $t_r = 5$ minutes, explain why a review request at minute 9 is inadequate even if confidence is increasing.
- 7.4 Create a four-state escalation table with entry evidence, exit evidence, owner, and required log.
- 7.5 Explain the source limits between the Bian Que story and a modern threshold model.

Chapter summary

- Thresholds trade false alarms against missed states and must be tied to consequences.
- Time remaining is part of the state, not a background detail.
- Good escalation policies preserve a reversible path whenever the evidence and time permit it.

Chapter 8

Intelligent Underwater Robots: Autonomy Under Restricted Communication

Narrative Prelude — Water Finds a Boundary. Stories of flood control often contrast forcing a barrier with reading a terrain and guiding flow. The traditional materials are layered myths and political narratives, not technical field reports. In this chapter, the metaphor opens a modern question: when a platform cannot remain in contact with people, what constraints must travel with it?

Source note. The chapter uses classical water and flood-control motifs as conceptual material (T/P), not as a direct history of underwater robots. All underwater mission states, energy values, and communication examples are classroom examples (C).

Learning objectives

1. describe the coupling of sensing, navigation, energy, and communication in underwater autonomy;
2. explain why an untransmitted message is not evidence of a received message;
3. formulate a local policy with stated limits, communication windows, and energy reserve;
4. identify appropriate boundaries for autonomy in partially observed environments.

A communication system moves information through a channel. A message is not the same thing as a packet, and a packet is not the same thing as a confirmed meaning. Three elementary quantities matter: latency is the time between sending and receiving; loss is the probability that a message does not arrive as intended; bandwidth is the amount of information that can be carried per unit time. A queue stores information that has been generated but not yet delivered.

If arrivals exceed service for long enough, the queue grows. These basics explain why local autonomy must include freshness, timeout, and recovery rules.

8.1 Foundations: communication, queues, freshness, and local policy

Communication is a channel with delay, loss, ordering rules, bandwidth, energy cost, and a relation to authority. A message that was sent is not necessarily received, and a received message is not necessarily current. This module gives the vocabulary needed to model those distinctions before the chapter turns to intermittent communication and bounded local autonomy.

8.1.1 Message, packet, delivery, and acknowledgment

A **message** is application-level meaning: for example, a versioned status report. A **packet** is a transport unit used to carry some portion of that message. Transmission success only says a packet left an interface. Delivery success means the intended receiver obtained the declared message. An acknowledgment is evidence that a receiver or intermediary reports receipt; it does not by itself prove the message was correctly interpreted or acted upon.

A minimal message record can be written as

$$m = (\text{id}, \text{source}, \text{destination}, t_{\text{create}}, t_{\text{send}}, \kappa, \text{payload}, \text{status}).$$

The timestamp and configuration κ determine whether a message remains valid. A field called “position” without a frame, age, and source identity is not a complete status claim.

8.1.2 Bandwidth, latency, and loss

Bandwidth is an upper bound on information rate under stated conditions. Latency is elapsed time from a defined starting point to a defined ending point; both endpoints must be named. Packet-loss rate estimates the fraction of packets not successfully delivered according to a protocol’s definition. The variables interact: retransmission can reduce apparent loss while increasing latency and energy use.

For a rough service model, if messages arrive at rate λ and a channel serves them at rate μ , a stable queue requires $\lambda < \mu$. As utilization $\rho = \lambda/\mu$ approaches one, waiting time grows sharply in simple queueing models. The exact formula depends on arrival and service distributions, but the engineering insight is general: a channel that is adequate on average can fail during bursts.

The **age of information** at time t is $\Delta(t) = t - u(t)$, where $u(t)$ is the generation time of the freshest information received by time t . Age measures freshness, not merely transmission delay. A recently delivered message can still be stale if it was generated long before it was sent.

8.1.3 A simple queue and freshness example

Suppose fictional sensor summaries are generated every 10 seconds but the channel can transmit one summary every 15 seconds during an adverse period. The average arrival rate is $\lambda = 0.1$ messages/s and service rate is $\mu \approx 0.0667$ messages/s, so $\lambda > \mu$: backlog must grow. Sending every record may make the receiver less informed because it receives a stream of stale states. A policy may instead coalesce routine updates, preserve anomaly reports, attach age, and declare when fresh synchronization is required.

Worked example: send is not delivery

A fictional platform creates a status message at 10:00:00, sends it at 10:00:05, and receives an acknowledgment at 10:00:20. At 10:01:00, its information age at the receiver is at least 60 seconds, not 40 seconds. If the receiver's policy treats data older than 30 seconds as invalid for a transition, the acknowledgement cannot make the message current. The correct state is request for a newer observation or a protected response. [Illustrative teaching value]

8.1.4 Reliability mechanisms and their boundaries

Sequence numbers help detect missing or reordered packets. Checksums help detect accidental corruption. Timeouts and acknowledgments support retransmission. Redundant paths can improve availability when their failures are not common-mode. None of these mechanisms guarantees semantic correctness. A message with a valid checksum can contain a stale map version; an acknowledged packet can be interpreted under a different rule version. Communication protocols and system interfaces must therefore specify both transport behavior and meaning.

8.1.5 Energy is part of the channel model

Transmission, reception, sensing, computation, and motion draw from a finite energy budget. A simple accounting model is

$$E_{k+1} = E_k - E_{\text{sense},k} - E_{\text{compute},k} - E_{\text{comm},k} - E_{\text{motion},k}.$$

The terms are teaching abstractions, but they demonstrate coupling: repeated retries can improve delivery probability while reducing the reserve needed for recovery. A channel policy should therefore specify a retry limit, priority class, expiration rule, and minimum energy reserve.

8.1.6 Local policy under intermittent information

When communication is unavailable, a system should not invent global knowledge. A local policy operates only inside a declared envelope. One abstract model uses a belief state $b_k(x) = P(x_k = x \mid z_{1:k})$, which represents uncertainty about the current state. The policy chooses a local state within stated limits based on b_k , resource margin, and hard constraints. If evidence becomes stale, contradictory, or outside scope, it should reduce function, request synchronization, or enter a safe state.

[title=Freshness-aware message handling]

```
attach identity, source, timestamp, version, priority, and expiry to each
message
on receipt, verify integrity, sequence, frame, and configuration compatibility
compute information age and reject or downgrade stale messages
preserve high-priority anomaly records while coalescing routine updates
limit retries by energy reserve, queue length, and task time window
record transmission, acknowledgment, delivery evidence, and unresolved state
```

- Treating a send operation as evidence that the receiver knows the message.
- Treating an acknowledgment as evidence that the receiver shares the sender's interpretation.
- Reporting latency without saying whether it begins at measurement, message creation, send, or receipt.
- Ignoring queue growth and retaining obsolete routine data ahead of current exceptions.
- Permitting local behavior after authority, map, or task information has expired.

8.1.7 Foundation self-check

1. Distinguish transmission, delivery, acknowledgment, and semantic interpretation.
2. If arrivals exceed service rate for a sustained interval, what happens to the queue?

3. Compute the age of a message generated at 10:00 and received at 10:02 when the current time is 10:05.
4. State one reason to coalesce routine messages but preserve exception records.

The key answer is that freshness and meaning must be carried with the message. A channel can be available and still deliver information that is too old for the current transition.

8.1.8 Bridge to changing channels

The historical flood-control image is used here only as a question about changing the structure through which a flow moves. The engineering chapter asks how a system communicates under restrictions without confusing transmitted data with current shared understanding. The next sections apply queues, freshness, energy margins, and bounded local policy to that problem.

8.2 Historical opening and source note

The water image from the Daoist tradition introduces a different meaning of autonomy. Underwater systems face limited communication, uncertain localization, energy constraints, changing currents, and delayed human supervision. A capable system is therefore one that stays inside a declared boundary when it cannot ask a person immediately.

8.2.1 Foundational knowledge to master

An intelligent underwater robot may be classified as remotely operated, autonomous, or hybrid; by propulsion and buoyancy management; by tethered or untethered communication; and by scientific, inspection, environmental, or security mission. Its basic structure includes pressure-resistant body, buoyancy and propulsion, energy storage, acoustic or optical sensing, inertial navigation, depth and attitude estimation, onboard computation, acoustic or intermittent communication, payload, and recovery support.

The key technical characteristics are three coupled uncertainties: the vehicle may not know its position accurately, the channel may not support continuous exchange, and the energy required for sensing, computation, movement, and communication competes with recovery reserve.

8.2.2 Learning content and mathematical derivation

If messages arrive at rate λ and the communication service rate is μ , sustained queue stability requires $\lambda < \mu$. Information age is

$$\Delta(t) = t - u(t),$$

where $u(t)$ is the generation time of the freshest received information. Energy evolution is

$$E_{k+1} = E_k - E_{\text{sense},k} - E_{\text{compute},k} - E_{\text{comm},k} - E_{\text{move},k}.$$

Under partial observability, a belief state can be written as

$$b_k(x) = P(x_k = x \mid z_{1:k}).$$

Students should explain why a local policy must use both b_k and resource margins, and why it must lower function or request synchronization when information age exceeds validity.

8.2.3 Case studio: water-tank communication-window demonstrator

The safe product is a small water-tank sensor platform or simulated underwater vehicle. A 嘉立创 EDA board can host a pressure or depth sensor, inertial sensor, temperature sensor, low-power controller, acoustic-communication analogue, and data logger. The board remains tethered or operates in a controlled tank; it does not carry a navigation or weapon payload.

Students compare three communication policies: transmit every sample, prioritize anomalies, and coalesce routine data. They measure queue length, information age, energy use, and recovery margin. The design must specify timeout, retry limit, message expiry, safe state, and post-run record.

8.2.4 Whole-chapter synthesis

Autonomy under restricted communication is not permission to improvise beyond a mission. It is the ability to preserve a valid local state, consume resources honestly, queue or discard information according to policy, and enter recovery before a missing link becomes an excuse for an uncontrolled decision.

8.2.5 Practice and extension

1. Classify underwater robots by control relationship and communication condition.
2. Calculate queue growth when message arrival exceeds service capacity.

3. Compare two message policies using freshness and energy rather than delivery count alone.
4. Define a local policy for stale authority and missing acknowledgment.
5. Explain why local autonomy must not silently expand its task scope during communication loss.

Availability describes whether a channel or service can be used under a declared condition. An acknowledgment is a defined response from a receiver; silence is not automatically a negative acknowledgment. A protocol must specify what was acknowledged, when it expires, and what happens after timeout.

8.3 Why underwater systems are different

An underwater platform often relies on acoustic sensing, inertial navigation, depth measurement, propulsion, stored energy, and intermittent acoustic communication. The communication channel has low rate, variable delay, and uncertain delivery. Therefore a ground-station assumption such as “the supervisor will see the next update” is often invalid.

Let the system state include position estimate $\hat{p}_k$, energy e_k , communication queue q_k , and mission progress m_k . A simplified update is

$$\begin{aligned}\hat{p}_{k+1} &= \Phi(\hat{p}_k, z_k, u_k), \\ e_{k+1} &= e_k - \Delta e(u_k, E_k), \\ q_{k+1} &= q_k + r_k - d_k,\end{aligned}$$

where r_k is generated information and d_k is information successfully delivered during a communication opportunity. A message in q_k has been prepared, not received.

8.4 Communication windows and local policy

Let $\mathcal{W}$ be a set of predicted communication windows. A local policy should not assume they all occur. A conservative rule reserves energy for return and for one or more missed windows:

$$e_k \geq e_{\text{return}} + e_{\text{margin}}.$$

If the inequality fails, mission progress is no longer the only objective. The system should enter a return, hold, or safe-mode state according to its declared rules.

Worked example

In a fictional underwater survey, an anomaly record is placed in an upload queue. A changing current raises the estimated energy cost and the next acoustic window is missed. The correct systems question is not “Did the platform report the anomaly?” but “Which log proves message creation, which log proves delivery, and how does the energy constraint alter the allowed state transitions?”

[Illustrative teaching value]

8.5 Autonomy within stated limits

Restricted communication can justify local decision capability, but it strengthens the need for limits. A platform should know its mission envelope, energy reserve, data-retention policy, uncertainty threshold, and safe-return condition before loss of contact. Autonomy without stated limits is an unexamined transfer of responsibility.

8.6 A communication channel is a state variable

Communication should be modeled as a condition that changes what the system can know and what authority can be exercised. Let ℓ_k denote link quality, d_k delivery status, and q_k the queue of undelivered records. A message moves through distinct states:

generated $\rightarrow$ queued $\rightarrow$ transmitted $\rightarrow$ acknowledged $\rightarrow$ archived.

The arrows are not automatic. A message can be generated but dropped before transmission, transmitted but not acknowledged, or acknowledged but archived under the wrong configuration. Each transition needs an event record and a timeout rule.

A communication window is a time interval in which a message may be transmitted with a declared probability, delay, bandwidth, and acknowledgment condition. A predicted window is not a guaranteed delivery event.

8.7 Queues, freshness, and information age

If r_k messages are generated and d_k are delivered, a simple queue update is

$$q_{k+1} = \max\{0, q_k + r_k - d_k\}.$$

The queue length alone is insufficient. Let a_j be the age of message j . A high-priority message may become less useful as a_j grows, while a raw record may remain valuable

for later audit. Priority should therefore be based on declared semantics, not on the convenience of sending the newest message first.

Message state	Meaning	Evidence required
Generated	information object created locally	payload hash, source, time, configuration
Queued	waiting for a channel opportunity	queue identifier and priority
Transmitted	channel accepted a transmission attempt	attempt time and channel status
Acknowledged	receiver confirmed a defined object	acknowledgment identity and time
Archived	record stored for later reconstruction	storage location and integrity check

8.8 Energy and communication coupling

Let energy evolve as

$$e_{k+1} = e_k - \Delta e_{\text{move}} - \Delta e_{\text{sense}} - \Delta e_{\text{comm}} + \eta_k.$$

The term η_k can represent estimation error or a recovered resource in a teaching model. A communication attempt may consume energy without delivering information. The system must therefore compare the expected value of sending with the energy needed for navigation, recovery, and safety.

An abstract reserve constraint is

$$e_k \geq e_{\text{return}}(\hat{p}_k, \hat{E}_k) + e_{\text{margin}} + e_{\text{uncertainty}}.$$

The return term depends on estimated location and environment. The margin is a design choice that must be tested against model error, not merely added as a comfortable number.

Worked example: a missed window

In a fictional underwater survey, an anomaly record is placed in an upload queue. A changing current raises the estimated energy cost and the next acoustic window is missed. The correct systems question is not “Did the platform report the anomaly?” It is: Which log proves message creation, which log proves delivery, how old is the state estimate, and how does the energy constraint alter the allowed state transitions? [Illustrative teaching value]

8.9 Local policy under partial observability

The platform cannot decide from a complete state. It acts on a belief or estimate $b_k(x)$, which is updated from observations and controls:

$$b_{k+1} = \tau(b_k, z_{k+1}, u_k).$$

The notation reminds us that local autonomy is a policy under uncertainty, not a replacement for uncertainty. A conservative policy may allow information gathering and protected recovery while forbidding decisions that require an unavailable authority or a stale map.

[title=Communication-aware local policy]

```
update navigation, energy, queue, and link estimates
if authority is expired or state is out of scope:
    enter safe state
elif energy reserve is insufficient for recovery:
    stop new task progress and follow recovery rule
elif a communication window is available:
    transmit prioritized records and await defined acknowledgment
else:
    continue only bounded local functions inside the envelope
record every state change, attempt, timeout, and unresolved message
```

8.10 Delay, loss, and stale authority

A command sent at time t_0 may arrive at $t_0 + \Delta t$ when the world and the system state have changed. A safe protocol should attach a validity interval and configuration identity to any consequential message. A message that arrives after expiry may be correctly delivered as a packet and still invalid as an instruction.

Authority is stale when the permission remains syntactically valid but its conditions, time window, state estimate, or configuration no longer match the situation for which it was issued.

For every message, answer: What object is acknowledged? What is its validity interval? Which configuration produced it? What happens after timeout? Can the receiver distinguish silence from a negative instruction?

8.11 Historical and methodological interpretation

Traditional water stories are layered myths and political narratives, not technical field reports. Their conceptual value is the relation between environment, boundary, and flow. The modern analysis begins when we define the communication channel, energy reserve, message states, and local policy.

8.12 Protocol semantics and delivery claims

A protocol should distinguish the message object from the transport event. “Delivered” may mean accepted by a channel, received by a device, parsed by an application, acknowledged by a responsible service, or archived for audit. These are different claims. A protocol document should define the exact meaning of each status and the timeout behavior.

Status	Claim established	Claim not established
Sent	transmission attempt occurred	receiver processed content
Received	endpoint obtained a packet	policy accepted its meaning
Acknowledged	defined receiver confirmed object	command was executed
Executed	local outcome was observed	intended external effect occurred

8.13 Energy-budget exercise

Write an energy budget with sensing, computation, communication, motion, return, and uncertainty margin. Then remove the next communication window and recompute the feasible state set. The correct result may be that the mission objective must be reduced rather than that the platform should continue with a smaller reserve.

8.14 Historical notes and further reading

Classical water metaphors can help students see the relation between environment and action, but technical claims require systems literature and tests. For estimation and mobile autonomy, see [8, 7, 9].

8.15 Exercises

- 8.1 Draw the sensing, navigation, energy, communication, mission-control, and recovery loop for a fictional underwater survey platform.
- 8.2 Distinguish generated, queued, transmitted, acknowledged, and archived messages. Which transitions can fail?
- 8.3 Write an energy-reserve inequality for a mission with return and safety margins. Explain every term.
- 8.4 Design a five-state autonomy policy for a missed communication window.
- 8.5 Why is more communication failure not, by itself, a reason to remove all human authority?

Chapter summary

- Underwater autonomy is shaped by coupled uncertainty in navigation, energy, and communication.
- “Message created” and “message received” are distinct state claims.
- Communication limits require stronger predeclared boundaries and recovery rules.

Intelligent Weapon Literature Reviews: Evidence and Configuration Management

Narrative Prelude — Three People Make a Tiger. In the political story known as “three people make a tiger,” a ruler who initially rejects an implausible report becomes uncertain as the same claim is repeated. The lesson is not that repetition always deceives. It is that repeated transmission is not the same as independent observation. A technical system can make the same mistake when citations, screenshots, reports, and logs all point back to a single unexamined source.

Source note. The story is a rhetorical account in *Strategies of the Warring States* (R), not an independently verified incident. Citation chains, version trees, and technical examples are classroom examples (C).

Learning objectives

1. rank evidence by provenance, method, and reproducibility rather than by repetition;
2. distinguish primary experiment, technical report, review, secondary citation, screenshot, and unverified claim;
3. formulate a configuration record for model, data, software, and rule versions;
4. identify circular citation and incomplete-log failure modes.

A reliable technical claim has a source, a method, a condition, and a way to check the result. A primary source reports an observation or experiment directly. A secondary source summarizes or interprets another source. A review compares several sources. A citation chain can become circular when later documents repeat the same original claim. Reproducibility means that another person can rerun or independently inspect the method under the same declared configuration.

ration. Configuration management records which data, model, software, rules, interface, and hardware produced an output.

9.1 Foundations: claims, sources, provenance, and reproducibility

Engineering knowledge does not consist of documents stacked beside one another. It consists of claims whose sources, assumptions, transformations, and limits can be inspected. This chapter's foundations make evidence lineage explicit. The same discipline applies to historical reading, technical literature, data analysis, simulation, and test reports.

9.1.1 Claim, evidence, source, and inference

A **claim** is a proposition that can be supported, weakened, narrowed, or rejected. **Evidence** is material bearing on a claim: a measurement, an inspection record, a test result, a primary document, or a validated calculation. A **source** is the origin or container from which evidence is obtained. An **inference** is the reasoning that links evidence to a claim.

These objects should not be collapsed. A citation is not automatically evidence for every sentence next to it. A measured quantity is not automatically a claim about cause. A historical text can directly support that a source contains a statement, while a reconstruction of motivation may remain uncertain. Good writing and good system design both state the strength of this bridge.

Provenance is the documented lineage of an artifact or claim: where it originated, how it was obtained, transformed, selected, versioned, and used. A provenance record makes it possible to ask whether two apparently independent pieces of evidence actually descend from the same source.

9.1.2 Evidence status and source diversity

For course work, it is helpful to tag a statement by its epistemic status:

Status	What it supports	Appropriate wording
Primary source	what an original document or measurement records	"The source states..."
Secondary synthesis	an author's interpretation or summary	"A later account argues..."
Reconstruction	a plausible illustrative narrative	"One possible reconstruction is..."
Teaching model	an explicit classroom abstraction	"Assume for this example..."
Test evidence	behavior under stated configuration and conditions	"The test observed..."

Source diversity matters because repetition can create the appearance of corroboration without independent evidence. Three slide decks that cite one preliminary report form one lineage, not three independent confirmations. A provenance graph can expose this structure.

9.1.3 Provenance graphs

Represent evidence as a directed graph $G = (V, E)$. Nodes can be sources, measurements, data sets, scripts, models, claims, or decisions. An edge records a relation such as "derived from," "cites," "processed by," "supports," or "uses configuration." The graph does not determine truth automatically. It helps a reviewer see the path from a conclusion back to its premises.

For example, a claim node C may depend on processed data D' , which was produced by script version s , from raw measurement D , under sensor configuration κ . A reproducible claim must preserve enough of this path for another reviewer to inspect or repeat the transformation. The reviewer should ask two questions: "Can I open the final figure?" and "Can I tell what the figure represents and how it was produced?"

9.1.4 Configuration is part of meaning

A numerical output without configuration is incomplete. Let a configuration tuple be

$$\kappa = (\text{hardware, software, model, data, parameters, map, rule, calibration}).$$

The tuple need not be stored as one literal field; it expresses the principle that any factor capable of changing the claim must be identifiable. A model score under one threshold, data preparation procedure, or map version does not automatically support the same conclusion under another.

Worked example: three documents, one lineage

Suppose a technical claim appears in a report, a presentation, and a student handout. All three cite a preliminary laboratory note. The graph has three publication nodes but one underlying evidence node. The correct summary is “three documents repeat one preliminary source,” not “three independent sources confirm the result.” The appropriate next step is to seek an independent measurement, analysis, or primary record, or to narrow the claim. [Illustrative teaching value]

9.1.5 Reproducibility and repeatability

Repeatability asks whether the same team can obtain consistent results with the same setup. **Reproducibility** asks whether an independent team can obtain consistent results using the documented method and comparable conditions. Exact numerical reproduction may be impossible when data are restricted or random processes are involved, but the design should still provide a documented method, declared uncertainty, and a route for independent review.

An elementary reproducibility record includes: the question, claim, source or data identifier, inclusion and exclusion rules, configuration, transformation steps, code or calculation version, output, uncertainty, reviewer, and date. Each item prevents a different kind of silent drift.

[title=Claim review protocol]

write one precise claim and identify its scope and non-claims
link each supporting artifact to source, time, version, and transformation
separate direct observation from inference and interpretation
trace whether multiple citations share a common upstream source
record assumptions, missing evidence, uncertainty, and challenge cases
state what a reviewer could reproduce and what remains conditional

9.1.6 Literature reading as model checking

When reading a paper or source, extract more than a conclusion. Identify the object studied, population or environment, assumptions, variables, method, configuration, measure, result, limitation, and citation lineage. A result may be methodologically sound within its stated scope and still be irrelevant to a different task. This is not a defect in the source; it is a boundary condition the reader must preserve.

- Treating a source citation as proof without checking what the source actually states.

- Treating repeated secondary summaries as independent evidence.
- Omitting dataset, model, parameter, map, or rule version from a result record.
- Presenting a reconstructed narrative as if it were a direct historical source.
- Giving a conclusion but no way to inspect its assumptions, calculation, or challenge conditions.

9.1.7 Foundation self-check

1. Give one example of a primary source, a secondary synthesis, and a teaching model.
2. Draw a provenance path from a raw measurement to a final claim.
3. Why are three documents with one common upstream source not three independent confirmations?
4. Name two configuration fields that can change the meaning of a numerical result.

A strong answer distinguishes direct observation from inference and records the transformations between them. The aim is not bureaucratic citation; it is preserving the conditions under which a claim can be checked.

9.1.8 Bridge to the “three people make a tiger” problem

The story makes a cognitive point about repetition and belief. Its engineering translation is not that repeated reports are always wrong. It is that the independence and lineage of reports must be examined before repetition is counted as corroboration. The chapter now uses provenance graphs, configuration records, and reproducibility matrices to make that examination systematic.

9.2 Historical opening and source note

The “three people make a tiger” story opens a question about repeated claims. Three documents may look like three sources while all three descend from one preliminary note. The lecture turns the literary image into a practical research task: trace a claim, inspect its source, preserve configuration, and state what remains unverified.

9.2.1 Foundational knowledge to master

A literature review for intelligent weapon systems is not a sequence of abstracts. It is a structured answer to a question. The minimum workflow is define scope, search, screen, extract, compare, synthesize, challenge, and report. Students classify material as primary source, experiment, technical report, standard, review, secondary account, or unverified reproduction. They also record population, method, model, data, configuration, metric, limitation, and relevance.

The evidence chain is

question $\rightarrow$ search strategy $\rightarrow$ source set $\rightarrow$ extraction table
 $\rightarrow$ comparison $\rightarrow$ claim.

Every arrow is a possible loss of meaning. A good review makes those transformations visible.

9.2.2 Learning content and mathematical derivation

For a search result set, precision and recall provide a useful teaching vocabulary:

$$\text{Precision} = \frac{\text{relevant retrieved}}{\text{all retrieved}}, \quad \text{Recall} = \frac{\text{relevant retrieved}}{\text{all relevant items}}.$$

Students should see that a narrow search may have high precision but miss important work, while a broad search may have high recall but create heavy screening burden. A simple evidence-weight model is

$$W_i = Q_i M_i R_i C_i,$$

where Q_i is source quality, M_i methodological strength, R_i relevance, and C_i configuration completeness. The product is a classroom rubric, not a probability of truth. It forces the reviewer to explain why a source is being used.

9.2.3 Case studio: provenance and review dashboard

Students use a spreadsheet, Jupyter notebook, or a small local database to build a provenance graph for a chosen non-sensitive topic such as autonomous navigation, sensor fusion, or robot safety. The dashboard records source identifier, year, type, research question, method, data, configuration, result, limitation, citation lineage, and review status.

The exercise includes one deliberately circular citation chain. Students must locate the common upstream source, mark the repeated claims, and add an independent source or narrow the conclusion. The exercise does not require a 嘉立创 EDA product. A physical “source-status” indicator board may support the presentation, while the core product is the reproducible evidence matrix.

9.2.4 Whole-chapter synthesis

Evidence gains strength from a documented source, an appropriate method, stated configuration, and independent confirmation. Repetition can increase familiarity while adding no new support. The reviewer narrows a conclusion when the path to its source cannot be reconstructed.

9.2.5 Practice and extension

- 1. Write a review question with inclusion and exclusion criteria.
- 2. Calculate precision and recall for a fictional search result set.
- 3. Draw a provenance graph from raw data to a published claim.
- 4. Identify a configuration field that could invalidate an earlier comparison.
- 5. Draft a structured abstract that separates result, limitation, and inference.

Question	Strong evidence	Weak substitute
What was observed?	primary data or log	copied sentence
Under what conditions?	protocol and metadata	general claim
Can it be checked?	reproducible method or audit trail	screenshot alone
Is it independent?	distinct observation path	several documents from one source

9.3 Evidence has a lineage

An engineering claim becomes more useful when a reader can trace it backwards. A literature review should not be a pile of summaries. For each claim, identify a source, method, data, operating conditions, limitations, and relation to other claims. The same rule applies to a system log: a dashboard image is not a substitute for a timestamped raw record with provenance.

Evidence type	Primary use	Common limitation
Primary study or test	establishes a method and observed result	may be narrow in conditions or data
Technical report	documents design and integration	may lack independent review
Review article	compares a field	can inherit a source's error
Secondary citation	points to a source	is not independent evidence
Screenshot or excerpt	preserves a display claim	may omit context, version, or provenance

9.4 Configuration is part of the claim

Let a configuration be a tuple

$$\kappa = (v_{\text{model}}, v_{\text{data}}, v_{\text{software}}, v_{\text{rules}}, v_{\text{interface}}).$$

An output y is meaningful only relative to κ . If a result is repeated after an unrecorded data relabeling or rule change, the organization may falsely believe it has reproduced the same system behavior. Configuration management makes such differences visible.

9.5 Circular evidence

Suppose internal note A cites report B, report B cites public article C, and article C cites A. The cycle may create the visual appearance of three sources but has only one unverified origin. A graph representation helps: sources are nodes and citations or derivations are directed edges. A claim is stronger when its supporting paths include independent primary observations rather than returning to the same source.

Worked example

A team finds a widely repeated statement that a certain model “has been validated.” The original citation is a preliminary internal note; later slides copied the claim without the note’s operating conditions. Construct a provenance graph and write the minimum additional evidence required before the claim can be treated as a design input. [Illustrative teaching value]

9.6 Claims, sources, and provenance graphs

Represent a claim as a node c and its supporting material as a directed provenance graph $G = (V, E)$. An edge may mean “cited by,” “derived from,” “tested against,”

or “transformed into.” These relations should not be mixed. A review article can summarize a primary experiment, but it does not become a second experiment merely by repeating the result.

Two pieces of evidence are independent for a stated claim only to the extent that their observations, data generation, processing, assumptions, and failure modes do not share an unexamined common source. Different documents or interfaces do not guarantee independence.

Source type	What it can establish	What it cannot establish alone
Primary study or test	method and observed result under conditions	general validity beyond tested conditions
Technical report	design decisions and integration context	independent confirmation
Review article	comparison and synthesis	freedom from inherited errors
Secondary citation	location of a claim	the original method or result
Screenshot or excerpt	a display existed at a time	complete provenance or configuration

9.7 Configuration as a coordinate of meaning

Let

$$\kappa = (v_{\text{model}}, v_{\text{data}}, v_{\text{software}}, v_{\text{rules}}, v_{\text{interface}}, v_{\text{hardware}}).$$

An output y is meaningful only relative to κ and the operating condition χ . Repeating an experiment while changing an unrecorded data filter or interface threshold is not a controlled replication. Configuration management is therefore part of scientific and engineering honesty.

9.8 Literature review as an engineering activity

A useful literature matrix contains at least: claim, source, method, population or environment, measured variables, result, limitation, configuration, and relation to other sources. The matrix is more than a reading aid. It prevents a familiar sentence from becoming detached from its conditions as it travels through presentations, reports, and teaching material.

Worked example: the validated model

A team finds a widely repeated statement that a model “has been validated.” The original citation is a preliminary internal note; later slides copied the claim without its operating conditions. The correct next step is not to count the slides. Build a provenance graph, retrieve the original test, record its population and configuration, identify missing independent tests, and replace the broad claim with a conditional one. [Illustrative teaching value]

9.9 Reproducibility and audit trails

For a result $y = f(D, M, \kappa)$, reproducibility requires access to the relevant data D , model M , configuration κ , preprocessing, randomization policy, evaluation protocol, and output record. Some data cannot be shared, but the limitation must be named and compensated by a stronger description or an independent audit.

[title=Evidence-lineage procedure]

```
state the exact claim and its scope
locate the nearest primary source
record data, method, configuration, and conditions
trace every secondary citation back to the primary source
mark shared origins and circular paths
separate observed result from interpretation
write the missing-evidence request before drawing a conclusion
```

9.10 The repetition trap

Repetition can increase social confidence while leaving epistemic confidence unchanged. In a system, the same phenomenon appears when two dashboards read from one cached database, or when three reports copy one unverified test. The review question is: “How many independent observation and reasoning paths support this claim?”

For every important claim, record its source, method, conditions, configuration, limitations, independent support, and permitted use. A claim should become narrower when its evidence is narrower. That is not weak writing; it is accurate writing.

9.11 Historical and methodological interpretation

The three-tiger story is a rhetorical account about how repeated reports can change belief. It is not an independently verified experiment in crowd psychology. Its classroom value is methodological: students learn to separate source count, transmission count, and observation count.

9.12 Evidence graphs and source diversity

Let each source path have a diversity vector $d = (d_1, d_2, d_3, d_4)$. The four components represent data, method, team, and environment. The vector is not a probability of truth. It is a review aid that makes shared origins explicit. Two papers may have different authors but share the same benchmark, preprocessing script, and evaluation protocol.

Source diversity review

A claim is supported by a public article, an internal report, and a presentation. All three use the same dataset and model checkpoint. The apparent document diversity is high, but data and method diversity are low. A reviewer should request either an independent dataset, an independent implementation, or a narrower claim. [Illustrative teaching value]

9.13 Reproducibility matrix

Create rows for data, preprocessing, model, rules, interface, hardware, randomization, and evaluation. For each row mark fixed, recorded, unavailable, or changed. A result with one unavailable row may still be useful, but its scope must be stated. A result with several changed rows should not be presented as a replication without further analysis.

9.14 Historical notes and further reading

The rhetorical story opens a provenance analysis; it is not evidence about crowd psychology. Review the evidence protocol in Appendix C. For risk and documentation practices, see [13, 15]; for research methods, consult the original studies cited in the chapter bibliography as well as summaries.

9.15 Exercises

- 9.1 Build an evidence matrix for five sources related to one intelligent-system topic. Include claim, source type, method, condition, limitation, and reliability question.
- 9.2 Draw a circular-citation graph with four nodes. Explain why the number of documents is not the number of independent observations.
- 9.3 Define a configuration tuple for a fictional classifier. Which change requires retesting?
- 9.4 A screenshot and an original log disagree. What questions must be answered before assigning fault?
- 9.5 Apply the source-limits protocol to the three-tiger story.

Chapter summary

- Repetition is not independence; provenance is part of evidence quality.
- A result must be tied to its model, data, software, rules, and interface configuration.
- A documented evidence path exposes missing links instead of hiding them behind authority or volume.

Chapter 10

Air-Defense Missiles: Information Fusion and Integrated Systems

Narrative Prelude — Borrowed Arrows. The episode of borrowing arrows from the fog is a literary narrative from *Romance of the Three Kingdoms*. It depicts a decision made under obscured visibility and strong expectations. It is not a source for military technology. Here it opens a systems question: when several sensors report related signals, how can a system distinguish corroboration from multiple echoes of the same error?

Source note. The “borrowed arrows” episode is literary fiction (L). Sensor models, track fusion, and classroom scenarios are fictional examples (C). This chapter explains estimation and decision responsibility, not real air-defense deployment or response procedures.

Learning objectives

1. describe the stages of multi-source information fusion;
2. distinguish independent corroboration from correlated measurements;
3. formulate a simple track-state update and association question;
4. explain why color, timing, and interface design can turn uncertainty into false certainty.

The mathematical foundation is the idea of representing an uncertain quantity by a vector and a covariance matrix. A vector collects related variables; a matrix can transform coordinates or describe how errors vary and co-vary. If two errors are correlated, observing one tells us something about the other, so simply adding two confidence values can overstate the amount of independent information. MIT’s public Linear Algebra course reviews vectors, matrices, and linear transformations [17]; its public probability course supports the conditional-probability side of estimation [18].

10.1 Foundations: estimation, covariance, and information fusion

Information fusion combines observations, models, and prior estimates to form a state estimate. The central difficulty is not averaging numbers. It is preserving the assumptions that make an average meaningful: frame, timestamp, sensor model, uncertainty, dependence, and association. This module introduces the minimal estimation mathematics used in the chapter.

10.1.1 Mean, variance, and covariance

For a random variable X , the mean $\mu = \mathbb{E}[X]$ is a central value under a stated distribution. The variance is

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2],$$

and measures spread around the mean. For two variables X and Y , covariance is

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)].$$

Positive covariance means that deviations tend to occur together; negative covariance means they tend to occur in opposite directions. For a vector state x , covariance is a matrix P whose diagonal entries are variances and whose off-diagonal entries represent dependence.

Correlation is normalized covariance. A correlation near zero does not prove independence, and a correlation estimated in one environment may not persist in another. These caveats matter because fusion can become overconfident if correlated measurements are counted as independent votes.

An **estimate** $\hat{x}$ is a model-based statement about an unobserved or imperfectly observed state. Its uncertainty describes the remaining plausible error under stated assumptions. A numerical estimate without a source, frame, time, and uncertainty model is not a complete fusion result.

10.1.2 Weighted fusion of two scalar estimates

Suppose two independent unbiased measurements of the same scalar state are z_1 and z_2 with variances σ_1^2 and σ_2^2 . A minimum-variance weighted estimate is

$$\hat{x} = \frac{z_1/\sigma_1^2 + z_2/\sigma_2^2}{1/\sigma_1^2 + 1/\sigma_2^2}, \quad \sigma_{\hat{x}}^2 = \left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2} \right)^{-1}.$$

The more precise measurement receives more weight. The formula assumes both measurements refer to the same quantity at compatible time and frame, are unbiased,

and have independent errors. Violating any of these assumptions can make the output misleadingly precise.

Worked example: precision is not the whole truth

Two fictional temperature estimates are $z_1 = 20.0$ with standard deviation 1.0 and $z_2 = 22.0$ with standard deviation 2.0. The inverse-variance estimate is $\hat{x} = (20.0 + 22.0/4)/(1 + 1/4) = 20.4$, with variance 0.8. This is reasonable only if both measure the same place and time and their errors are independent. If both derive from the same biased calibration table, the apparent reduction in variance is not justified. [Illustrative teaching value]

10.1.3 Prediction and measurement update

For a linear discrete model,

$$x_{k+1} = Ax_k + Bu_k + w_k, \quad (10.1)$$

$$z_k = Hx_k + v_k, \quad (10.2)$$

where process noise has covariance Q and measurement noise has covariance R . A familiar estimation pattern first predicts

$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_{k-1}, \quad P_k^- = AP_{k-1}A^\top + Q,$$

then compares the prediction with measurement through residual $r_k = z_k - H\hat{x}_k^-$. A Kalman-style gain is

$$K_k = P_k^- H^\top (HP_k^- H^\top + R)^{-1},$$

and the update is $\hat{x}_k = \hat{x}_k^- + K_k r_k$. These equations teach the structure: predict from a model, measure, compare, weight by uncertainty, update, and record disagreement. They are not a license to assume any real sensor process is linear or Gaussian.

10.1.4 Association and semantic compatibility

Before fusing two observations, a system must decide whether they refer to the same entity or event. This is called association. A distance in coordinate space alone may be insufficient when timestamps differ, objects are nearby, labels conflict, or frames are mismatched. An association gate might require compatible frame, time difference below a limit, semantic class compatibility, and a residual within declared uncertainty. If the evidence fails the gate, the correct action may be to preserve competing hypotheses rather than force a merge.

10.1.5 Common-mode error and conservative fusion

Two sources can agree because they share a clock, map, model, calibration, data pipeline, or organizational assumption. That is common-mode dependence. A simple weighted average that treats them as independent can report false confidence. Better strategies include explicitly modeling covariance, using a conservative uncertainty bound, seeking a differently sourced observation, exposing disagreement to review, or declining to reduce uncertainty until dependence is understood.

Check	Question	Failure if omitted
Frame	Do both values use the same coordinate convention?	numerically close but geometrically different values
Time	Do they refer to a compatible time interval?	stale and current state mixed
Association	Do they concern the same entity or event?	two objects merged into one
Uncertainty	What errors and dependence are declared?	false precision from averaging
Semantics	Do labels and rules mean the same thing?	identical field names with different meaning

- Averaging measurements before checking frame, unit, timestamp, and object identity.
- Equating repeated agreement with independent corroboration.
- Displaying a fused point estimate while hiding residuals and source disagreement.
- Treating a covariance matrix as decoration rather than a statement of error assumptions.
- Assuming an estimator’s mathematical optimality transfers to a changed environment or unmodeled interface.

10.1.6 Foundation self-check

1. Identify the diagonal and off-diagonal meaning in a covariance matrix.
2. State the independence assumption behind inverse-variance fusion.
3. Why must association be checked before combining two measurements?

4. Give one conservative response to suspected common-mode error.

The expected answer keeps the estimate, uncertainty, association, and dependence model together. Agreement can be evidence, but shared assumptions can make agreement less informative than it appears.

10.1.7 Bridge to borrowed arrows and hidden assumptions

The narrative opening prompts the question of whether an apparent resource or observation has the meaning assigned to it. The engineering version is precise: a fusion output is only as credible as its sensor models, association logic, dependence assumptions, and records. The rest of the chapter builds an architecture that preserves rather than hides those conditions.

10.2 Historical opening and source note

The borrowed-arrows story begins in fog, where many visible impacts might be mistaken for independent confirmation. The technical entry is an air-defense missile system understood as a networked system: sensors, command and control, tracking, launch or engagement management, interceptor guidance, communication, identification, and human authority. The lecture remains at a public systems level and does not teach deployment or engagement procedures.

10.2.1 Foundational knowledge to master

Students first distinguish a missile round from an air-defense weapon system. A generic architecture contains surveillance or detection sensors, track formation, identification and classification, command and control, engagement management, interceptor or effectors, guidance and control, communication, safety interlocks, and after-action records. Broad categories can be compared by engagement range, sensor relationship, guidance relationship, mobility, and command architecture without listing sensitive performance data.

The information chain is

detection → track → identification → decision aid
→ authorization → bounded response → assessment.

Students should identify where false tracks, delayed data, ambiguous identity, or common-mode errors can enter.

10.2.2 Learning content and mathematical derivation

For a linear measurement model $z_i = H_i x + v_i$, the information form of independent measurements is summarized by precision matrices:

$$P^{-1} = P_0^{-1} + \sum_i H_i^T R_i^{-1} H_i.$$

The derivation shows why a lower measurement variance appears to increase certainty. It also shows the danger: if two sensors share a map, clock, model, or processing chain, the independence assumption fails. A covariance-aware model adds cross-covariances or applies a conservative bound.

Association is separate from fusion. A measurement gate can use residual $r_i = z_i - H_i \hat{x}$ and innovation covariance S_i :

$$\nu_i = r_i^T S_i^{-1} r_i.$$

The scalar is a consistency measure, not an identity proof. Students must preserve unknown and conflicting hypotheses rather than force all signals into one track.

10.2.3 Case studio: multi-sensor airspace analogue

The safe case is a two-dimensional classroom airspace simulator with moving colored markers. It uses synthetic radar-like range data, camera-like bearing data, and a communication report. Students inject a shared stale map, a delayed message, and a false but internally consistent sensor source. They compare naive averaging, covariance-aware fusion, and explicit disagreement.

A 嘉立创 EDA board may serve as a tabletop sensor-fusion interface with three independent input channels, timestamp buttons, and LEDs for agreement, conflict, unknown, and review. The board must not connect to real surveillance, targeting, or weapon hardware.

10.2.4 Whole-chapter synthesis

More signals do not automatically mean more independent evidence. Fusion is defensible only when the observation model, timestamp, association hypothesis, covariance claim, and common-mode risks are visible to review. A principled disagreement state is often safer and more informative than a forced single answer.

10.2.5 Practice and extension

1. Draw the public functional architecture of an air-defense missile system.
2. Derive the scalar weighted-average estimator from inverse variance.

3. Explain why three agreeing sources may still be one common-mode source.
4. Construct an association gate and define the reject case.
5. Design an interface that displays uncertainty and disagreement without compressing them into one color.

Variance measures the spread of one uncertain quantity. Covariance measures how two quantities vary together. A covariance matrix has variances on its diagonal and pairwise covariances off the diagonal. A fused estimate should retain this structure instead of reporting only a single point.

10.3 A layered fusion problem

An integrated sensing system generally includes detection, measurement, association, state estimation, classification, and human or organizational interpretation. Let sensor i produce measurement

$$z_k^{(i)} = H_i x_k + v_k^{(i)}.$$

The matrices H_i map state variables into sensor observations. Combining measurements is not equivalent to averaging their numbers. The fusion process must ask whether two measurements refer to the same object, whether their errors are correlated, whether their timestamps align, and whether their quality declarations are current.

10.4 A simple estimator

For an illustrative linear estimator, the predicted state and covariance are

$$\hat{x}_k^- = F \hat{x}_{k-1}, \quad (10.3)$$

$$P_k^- = F P_{k-1} F^T + Q. \quad (10.4)$$

After a measurement, the update uses a gain K_k based on expected uncertainty. The important engineering point is not memorizing the Kalman filter. It is recognizing that an estimate must retain uncertainty and source lineage; a polished track display should not erase either.

10.5 Correlation and common-mode error

Two sources are not independent merely because they use two screens. They may share a time reference, a map, a calibration source, a classifier, an operator, or a

weather disturbance. Treating correlated observations as independent double-counts confidence. In a high-consequence system, fused confidence should be accompanied by a statement of source diversity and unresolved disagreement.

Worked example

Three fictional sensors produce a similar alert. Two inherit the same map timestamp and one is independently timed but has lower spatial precision. Design a fusion note that reports agreement without pretending the first two measurements are independent. [Illustrative teaching value]

10.6 Interface semantics

A color or icon can silently change an epistemic state. For example, red may mean “high confidence,” “high consequence,” “waiting for confirmation,” or “interface fault.” These are different meanings. Every safety-critical interface should make uncertainty, recency, source count, and authorization state legible without requiring a user to infer them from color alone.

10.7 Measurement models and association

Let sensor i produce

$$z_k^{(i)} = H_i x_k + v_k^{(i)}, \quad v_k^{(i)} \sim (0, R_i).$$

Before combining measurements, the system must decide which state each measurement refers to. This is the association problem. If two observations are incorrectly assigned to the same track, a mathematically stable estimator can produce a physically meaningless result.

For a predicted measurement $\hat{z}_k$ and observed measurement z_k , a simple innovation is

$$\nu_k = z_k - \hat{z}_k, \quad S_k = H P_k^- H^\top + R.$$

The normalized quantity $\nu_k^\top S_k^{-1} \nu_k$ can be used as a consistency check in a classroom model. The threshold is a design choice and must be connected to false association risk.

A common-mode error is an error shared by multiple sources because they depend on a common sensor, map, clock, calibration, model, operator, or environmental condition. It can make correlated evidence look independent.

10.8 Prediction and update

For an illustrative linear estimator,

$$\hat{x}_k^- = F\hat{x}_{k-1} + Bu_{k-1}, \quad (10.5)$$

$$P_k^- = FP_{k-1}F^\top + Q, \quad (10.6)$$

$$K_k = P_k^- H^\top (HP_k^- H^\top + R)^{-1}, \quad (10.7)$$

$$\hat{x}_k = \hat{x}_k^- + K_k(z_k - H\hat{x}_k^-), \quad (10.8)$$

$$P_k = (I - K_k H)P_k^-. \quad (10.9)$$

The equations show where the model, measurement, and uncertainty enter. They do not remove the need to inspect source lineage, timestamp alignment, or model validity. A fused track should retain covariance and a record of which measurements contributed.

Worked example: one-dimensional fusion

Suppose two classroom sensors measure the same scalar state. Sensor A reports 10 with variance 4; Sensor B reports 14 with variance 1. If independence is justified, the precision-weighted estimate is

$$\hat{x} = \frac{10/4 + 14/1}{1/4 + 1} = 13.2, \quad P = \frac{1}{1/4 + 1} = 0.8.$$

If both sensors inherit the same biased map, the numerical variance 0.8 is misleadingly small. The correct report must add a common-mode uncertainty or mark the fusion as conditional. [Illustrative teaching value]

10.9 Correlation and covariance

For two errors with covariance matrix

$$R = \begin{bmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix},$$

the correlation coefficient ρ controls how much independent information the pair provides. Treating $\rho = 0$ when the sensors share a map or clock can double-count evidence. A fusion report should state what independence assumption was used and what observation would challenge it.

10.10 Interface semantics and uncertainty display

A color or icon can silently change an epistemic state. Red may mean high confidence, high consequence, waiting for confirmation, or interface fault. These are different

meanings. A safety-critical interface should make uncertainty, recency, source count, disagreement, and authorization state legible without requiring a user to infer them from color alone.

Displayed item	Should answer	Dangerous ambiguity
Track label	What state is being estimated?	label confused with identity
Confidence	Under which model and population?	number treated as permission
Source count	How many independent paths?	duplicate sources counted twice
Age	When was the evidence valid?	stale result appears current
Authorization	What is permitted now?	warning appears to be an order

[title=Fusion review procedure]

align timestamps and coordinate frames
validate sensor configuration and quality declarations
associate measurements with candidate states
estimate state and covariance using declared correlation assumptions
retain disagreement and rejected associations
present estimate, uncertainty, age, source lineage, and authority separately
record the complete update for later reconstruction

Review a fusion system only when it can explain which measurements contributed, which assumptions were used, which sources are correlated, why alternatives were rejected, and what uncertainty remains. A smoother track alone is insufficient.

10.11 Historical and methodological interpretation

The borrowed-arrows episode is literary fiction. It opens the question of visibility, expectation, and correlated evidence; it is not a source for military technology. The engineering analysis begins with measurement models, association, covariance, and interface semantics.

10.12 Fusion architecture and disagreement paths

A complete fusion architecture includes a fused estimate and a disagreement path. When sensors conflict, the system may maintain multiple hypotheses, lower confi-

dence, request a new observation, or enter a safe state. Forcing a single track label too early makes disagreement disappear from the record.

Fusion output	Meaning	Review question
Agreement	observations are consistent under the model	Are sources independent?
Conflict	observations disagree	Is the disagreement preserved?
Stale	evidence age exceeds validity	Does the display show age?
Unknown	association or class unresolved	What is the protected response?

10.13 Sensor disagreement exercise

Write two fusion reports for the same three measurements. The first assumes independence and reports a precise estimate. The second includes shared bias and reports a wider uncertainty. Compare which report is more useful for a responsible decision and why.

10.14 Historical notes and further reading

The literary opening should be labelled as fiction. The engineering topic is multi-sensor estimation and correlated evidence; see [9, 8, 7]. Human-centered risk controls are discussed in [13].

10.15 Exercises

- 10.1 List the stages of a multi-source fusion pipeline and give one failure mode for each.
- 10.2 Explain why two measurements obtained from the same map version are not automatically independent.
- 10.3 For $z_k = Hx_k + v_k$, state the meanings of z_k , H , x_k , and v_k .
- 10.4 Design an interface legend that distinguishes detection, confirmed track, unresolved conflict, stale information, and authorized response.
- 10.5 Write a two-paragraph source boundary for the literary opening and the engineering analysis.

Chapter summary

- Fusion combines measurements with uncertainty, association, and provenance; it is more than arithmetic on numbers.
- Correlated sources can create false confidence if treated as independent.
- Interfaces should preserve uncertainty and decision status instead of visually compressing them into certainty.

Cruise Missiles: Navigation, Guidance, and Replanning

Narrative Prelude — Three Maps Can All Be True. The phrase “repair the plank road openly while advancing by another route” is a later summary of a historical tradition around Han Xin. The classroom problem is incompatibility: different maps can each be accurate for different dates, goals, scales, or reference frames. A system fails when it silently treats them as one map.

Source note. The historical basis and the later idiom must be separated (H plus later R/T). The three-map scenario, trajectories, and navigation values are classroom examples (C). The chapter discusses planning and control principles, not operational route design for real weapons.

Learning objectives

1. distinguish map correctness, currency, applicability, and goal alignment;
2. define navigation, guidance, and control as separate functions;
3. formulate a simple tracking-error model and replanning trigger;
4. explain why information fusion cannot resolve a conflict in mission objectives by averaging data.

Navigation begins with coordinates. A point can be written as a vector, but the vector has meaning only in a named frame. A rotation changes orientation; a translation changes origin; a rigid transformation combines both. If p^B is a point in frame B , a simple transformation into frame A is $p^A = R_B^A p^B + t_B^A$. Guidance uses a reference path, while control reduces the difference between the current state and the reference. LaValle’s public planning text provides the planning vocabulary of configuration, free space, graphs, and paths [20]; MIT’s public robotics course provides a robotics-oriented foundation for frames and motion [19].

11.1 Foundations: coordinate frames, navigation, guidance, and control

Motion-related system claims become ambiguous when a course uses the word “location” without declaring a reference frame, timestamp, map version, and purpose. This module builds the basic chain from coordinate transformations to navigation estimates, guidance references, and bounded control. It is a conceptual preparation for the chapter’s discussion of route evidence and replanning, not an operational procedure.

11.1.1 Coordinate frames and transformations

Let frame A be a local map frame and frame B a platform body frame. In two dimensions, a rigid transformation from coordinates in B to coordinates in A is

$$p^A = R_B^A p^B + t_B^A,$$

where R_B^A is a rotation matrix and t_B^A is a translation. For heading ϕ ,

$$R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}.$$

The superscript and subscript are semantic labels: T_B^A means “transform a representation from frame B into frame A .” A product of transformations is meaningful only when the adjacent frame labels match.

Homogeneous coordinates combine rotation and translation:

$$\begin{bmatrix} p^A \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R_B^A & t_B^A \\ 0 & 1 \end{bmatrix}}_{T_B^A} \begin{bmatrix} p^B \\ 1 \end{bmatrix}.$$

The inverse $T_A^B = (T_B^A)^{-1}$ changes representation in the opposite direction. The mathematics is straightforward; the difficult part is preserving which physical reference each symbol represents.

Worked example: a named-frame calculation

In a fictional indoor map frame M , a platform frame B is translated by $t_B^M = [10 \ 5]^T$ m and rotated by 90° . A point measured at $p^B = [2 \ 0]^T$ m becomes $p^M = R(90^\circ)p^B + t_B^M = [0 \ 2]^T + [10 \ 5]^T = [10 \ 7]^T$ m. Omitting the frame labels would leave the same arithmetic but remove the check that makes it meaningful.

[Illustrative teaching value]

11.1.2 Navigation, guidance, and control

These are related but distinct functions:

- **Navigation** estimates current state, such as position, heading, velocity, uncertainty, and time in a declared frame.
- **Guidance** selects a desired waypoint, path, or trajectory consistent with a goal and constraints.
- **Control** produces bounded inputs intended to reduce the difference between estimated state and reference.

When they are merged into one verb, errors become difficult to diagnose. A poor route may be a guidance problem; a route followed in the wrong coordinate frame may be a navigation or interface problem; unstable tracking may be a control problem. The responsibility and evidence for each are different.

11.1.3 A minimal kinematic model

For a planar nonholonomic teaching model with position (x, y) , heading ψ , speed v , and turn rate ω ,

$$\begin{aligned}\dot{x} &= v \cos \psi, \\ \dot{y} &= v \sin \psi, \\ \dot{\psi} &= \omega.\end{aligned}$$

This model describes geometry of motion but omits acceleration limits, slipping, delay, actuator saturation, and terrain. A planner that uses it should state those omissions. A controller that tracks a reference must remain inside declared bounds even when the state estimate is uncertain.

11.1.4 Tracking error and bounded correction

Let a reference point be p_k^* and estimated position be $\hat{p}_k$. The tracking error is $e_k = \hat{p}_k - p_k^*$. A proportional teaching controller is

$$u_k = \text{sat}(-K e_k, u_{\min}, u_{\max}),$$

where sat applies input limits. Saturation is essential: a mathematical controller that requests an impossible correction does not produce an acceptable physical behavior. A complete design also specifies what happens when the error, uncertainty, map age, or resource margin exceeds its envelope.

11.1.5 Map metadata and route validity

An intended path is conditional on the map that generated it. Let

$$\mu = (\text{frame, resolution, age, source, uncertainty, purpose, version}).$$

Two map products may overlap geometrically but disagree in purpose or date. A navigation display should preserve this metadata rather than make a path appear timeless. When a new observation changes map claim μ_k to μ_{k+1} , a replan is an evidence revision, not merely a preference change.

11.1.6 Error budgets and uncertainty

Position error can arise from sensor noise, frame transformation, time synchronization, map registration, and model drift. A simple additive bound is often overly conservative, while a covariance model can represent uncertainty statistically. Either way, a system needs a declared margin between estimated path and hard boundaries. The margin should account for both nominal tracking error and uncertainty in the information that produced the path.

[title=Frame-aware route review]

```
record current state, uncertainty, time, and frame
verify map version, resolution, purpose, and validity interval
express start, goal, obstacles, and planned route in compatible frames
check free-space margins under estimated state uncertainty
select a route with stated cost, constraints, and recovery condition
replan only when new evidence changes feasibility or declared cost
record the discarded route, new premise, approval state, and owner
```

- Adding or comparing coordinates from different frames, datums, or timestamps.
- Using a route produced for one map version after the map's validity or purpose has changed.
- Treating navigation, guidance, and control as one opaque "autonomy" block.
- Designing a nominal tracker without saturation, uncertainty, or recovery behavior.
- Calling every change in a route "replanning" without identifying the new evidence or changed constraint.

11.1.7 Foundation self-check

1. Compute a transformed point and label the source and destination frames.
2. Explain the separate roles of navigation, guidance, and control.
3. Why does a controller need saturation or a bounded-input rule?
4. What map metadata must be checked before reusing a route?

The expected answer treats a path as conditional on frames, map claims, objectives, and uncertainty. Replanning should cite the changed premise rather than merely show a different route.

11.1.8 Bridge to the visible road and hidden route

The historical phrase provides a narrative lens on the difference between visible geometry and the information structure behind a route. Its technical counterpart is a chain of contracts: coordinate frame, map metadata, navigation estimate, guidance objective, bounded control, and evidence-driven replanning. The following chapter sections make those contracts concrete.

11.2 Historical opening and source note

The visible and hidden routes associated with Han Xin are used here as a question about maps, purpose, and information age. A cruise missile is introduced as a system that must maintain a route and state estimate over time while its environment, map, communication condition, and task constraints may change. The lecture uses public conceptual knowledge only and avoids route planning for real targets or performance parameters.

11.2.1 Foundational knowledge to master

A cruise missile is described generically as a guided vehicle that uses sustained flight through the atmosphere and a navigation, guidance, and control chain to follow a planned trajectory. Its main functional blocks include airframe and propulsion, energy and environmental management, navigation sensors, mission computer, route or terrain database, guidance law, flight-control system, communication or update interface, safety constraints, and terminal decision logic.

Students distinguish navigation from guidance and control. They also compare mission planning, route following, map matching, re-planning, and terminal uncertainty. Development trends are framed through multi-source navigation, onboard computing, improved map metadata, adaptive guidance, resilient timing, and verification under distribution shift.

11.2.2 Learning content and mathematical derivation

For a planar kinematic model,

$$\dot{x} = v \cos \psi, \quad \dot{y} = v \sin \psi, \quad \dot{\psi} = \omega.$$

A frame transformation is

$$p^A = R_B^A p^B + t_B^A.$$

Students derive why a position without a frame and timestamp is incomplete. For a route P under map claim μ and goal G ,

$$J(P; \mu, G) = \sum_{e \in P} c(e; \mu, G).$$

Replanning is justified when the feasible set changes or $|J(P; \mu_{k+1}, G) - J(P; \mu_k, G)| > \eta$. The threshold is a declared teaching parameter, not a universal operational rule.

11.2.3 Case studio: 2-D cruise-route analogue

Students build a two-dimensional navigation and replanning simulator with a vehicle point, bounded turning rate, map layers, time-stamped obstacles, and three route objectives: distance, uncertainty, and recovery margin. The simulation deliberately uses an abstract vehicle and generic waypoints, not a real missile model or real route.

The experiment introduces a stale map, a changed goal, an unknown region, and a frame mismatch. Students must reject at least one route, explain the changed premise, and show the controller's tracking error. A small 嘉立创 EDA board can display route state, map version, and replan status while the computation remains in a notebook.

11.2.4 Whole-chapter synthesis

A map is a time-stamped claim made in a coordinate frame for a purpose. Replanning responds in a disciplined way to a changed feasible set, provided the old plan, new evidence, rejected alternatives, and authority are all preserved.

11.2.5 Practice and extension

1. Draw the public functional structure of a cruise-missile system without sensitive parameters.
2. Transform a point between two named frames and interpret the result.
3. Compare navigation, guidance, and control outputs for one route-tracking scenario.
4. Define a replan trigger based on changed feasibility or cost.

5. Explain why averaging two maps cannot resolve different mission objectives.

Function	Basic question	Typical output
Navigation	Where am I, in which frame?	state estimate and uncertainty
Guidance	Where should I go next?	reference path or trajectory
Control	What bounded input reduces error?	command subject to limits

11.3 Three functions, one motion

Navigation estimates where the platform is. Guidance selects a desired path or reference. Control produces inputs that reduce deviation from the reference. Let reference state be x_k^r and tracking error be $e_k = x_k - x_k^r$. A simple feedback law is

$$u_k = K e_k,$$

where K is a gain or policy parameter. The equation has no value until the coordinate frame, dynamics, actuator limits, and uncertainty assumptions are stated.

11.4 Map version and mission version

Let a map claim be represented by

$$\mu = (\text{geometry, timestamp, source, resolution, purpose}).$$

Two maps with identical geometry but different purposes may recommend different paths. A perception map may emphasize obstacles, a mission map may encode objectives, and a safety map may encode constraints. It is incorrect to average their route suggestions if they disagree because the underlying goals differ.

11.5 Replanning as evidence revision

Let $J(P; \mu, G)$ be the cost of path P given map claim μ and mission objective G . Replanning is justified when a new observation changes either the feasible set or the cost model beyond a declared threshold:

$$\Delta J = |J(P; \mu_{k+1}, G) - J(P; \mu_k, G)| > \eta.$$

The threshold η is illustrative. The principle is that a route change requires a changed premise, not an unexplained preference.

Worked example

Three fictional maps propose three routes to the same field station. The first is current but low resolution; the second is detailed but stale; the third is current but uses an evacuation objective rather than a delivery objective. State which attributes must be reconciled before any route recommendation is evaluated. [Illustrative teaching value]

11.6 Reference frames and coordinate contracts

Navigation claims are incomplete without a reference frame. A position may be expressed relative to a local map, a platform body frame, a global frame, or a moving object. Let T_B^A denote a transformation from frame B to frame A . A chain of transformations is meaningful only when frame identities and timestamps are explicit.

Navigation estimates the current state in a reference frame. Guidance selects a desired reference trajectory or path. Control generates bounded inputs that reduce tracking error under a dynamic model. Mixing the three functions makes it difficult to locate responsibility when the motion is wrong.

11.7 Map metadata and purpose

Let a map claim be represented by

$$\mu = (\text{geometry}, \text{timestamp}, \text{source}, \text{resolution}, \text{purpose}, \text{frame}).$$

Two maps may have identical geometry and still recommend different paths because their purposes differ. A perception map may emphasize obstacles, a mission map may encode objectives, and a safety map may encode constraints. Their suggestions should not be averaged when the underlying goals are incompatible.

Metadata	Why it matters	Failure if omitted
Timestamp	establishes currency	stale geometry treated as current
Source	identifies observation path	circular or untrusted origin hidden
Resolution	limits detail	narrow obstacle disappears in coarsening
Purpose	defines cost and relevance	objectives mixed silently
Reference frame	makes coordinates comparable	correct numbers combined incorrectly
Validity	defines expiry or scope	old authority remains active

11.8 Tracking error and bounded correction

Let x_k^r be the reference state and $e_k = x_k - x_k^r$ the tracking error. A simple discrete model is

$$e_{k+1} = Ae_k + Bu_k + w_k, \quad u_k = Ke_k.$$

The feedback-system matrix $A + BK$ determines the behavior of the teaching model. If actuator limits are present, u_k must be saturated and the analysis must state what happens when the desired correction cannot be produced. A small nominal error does not guarantee safety when uncertainty and delay are large.

Worked example: three maps, one unresolved objective

Three fictional maps propose routes to the same field station. The first is current but low resolution; the second is detailed but stale; the third is current but optimized for evacuation rather than delivery. All three maps may be correct relative to their metadata. No route should be selected until currency, resolution, reference frame, and mission objective are reconciled. The conflict is not solved by averaging the paths. [Illustrative teaching value]

11.9 Replanning as evidence revision

Let $J(P; \mu, G)$ be the cost of path P under map claim μ and objective G . Replanning is justified when a new observation changes the feasible set or cost model beyond a declared threshold:

$$\Delta J = |J(P; \mu_{k+1}, G) - J(P; \mu_k, G)| > \eta.$$

The threshold η is illustrative. A hard constraint should trigger replanning even when ΔJ is small. The record should state which premise changed and why the old path

was rejected.

```
[title=Metadata-aware replanning]
```

```
estimate state in an identified reference frame
validate map age, source, resolution, purpose, and validity
compare current constraints with the candidate path
if a hard constraint fails or a premise is outside scope:
    enter safe state and request a new plan
else if cost or uncertainty change exceeds the declared trigger:
    generate alternatives and record rejection reasons
else:
    continue bounded tracking with feedback
record frame and plan
record state and result
```

11.10 Historical and methodological interpretation

The familiar plank-road idiom is a later summary of a historical tradition and should not be attributed directly to a text that does not use it. Its classroom value lies in the possibility that multiple descriptions can be locally true while their purposes differ. The engineering lesson is to preserve metadata and refuse silent frame changes.

11.11 Coordinate transformation exercise

Let a point have coordinates p^B in body frame B and let the platform pose relative to frame A be rotation R_B^A and translation t_B^A . Then

$$p^A = R_B^A p^B + t_B^A.$$

The equation is simple, but a valid result also needs frame labels, time, units, and uncertainty. A correct transformation with the wrong timestamp can still produce an invalid route decision.

11.12 Replanning and rejected alternatives

When a path changes, preserve the old path, the new observation, the changed metadata, the candidate alternatives, and the rejection reason. This record supports later learning. A successful new path does not prove that the old path was unreasonable under its original premises; it proves that the premises or objective changed enough to justify revision.

11.13 Historical notes and further reading

Do not attribute the familiar four-character idiom directly to a text that does not use it. The technical material concerns coordinate frames, estimation, path planning, and feedback; see [6, 7, 8].

11.14 Exercises

- 11.1 Write a five-field map metadata record μ for three maps used in a fictional logistics exercise.
- 11.2 Give an example where all three maps are correct but no route should be chosen until the mission objective is clarified.
- 11.3 Explain navigation, guidance, and control using a ground vehicle on a curved reference path.
- 11.4 Define a replanning trigger that uses both a map update and a hard constraint.
- 11.5 Why does an unexplained route change make the decision record incomplete even when the new path later succeeds?

Chapter summary

- Maps require metadata: source, time, resolution, and purpose.
- Navigation estimates, guidance chooses references, and control corrects motion.
- Replanning is an evidence-driven revision of a declared model, not an unrecorded change of mind.

Chapter 12

Ballistic Missiles: Strategic Systems and Decision Responsibility

Narrative Prelude — When the Alarm Lost Its Meaning. The story of beacon fires and lost trust is transmitted in several historical and literary versions. Its power lies in the damaged relationship between a signal, a receiver, and a response. Once a warning channel has been used without credible conditions, a later real warning encounters a system whose social calibration has already changed.

Source note. Details of the beacon-fire story vary across sources and should not be used as an unqualified historical record (H/R source note). The warning architecture, authority record, and capstone exercise are classroom examples (C). This chapter treats strategic stability, safety, and responsibility at a conceptual level only.

Learning objectives

1. explain why warning, confirmation, authorization, and response must be separate stages;
2. assign responsibility across technical and organizational functions;
3. identify failure modes created by short decision windows, stale data, and diffuse ownership;
4. construct a high-level safety case for a fictional strategic system review.

A safety argument begins with a requirement and ends with evidence. A requirement says what must be true; verification checks whether the requirement is satisfied; validation checks whether the requirement addresses the intended need. A safety case then connects the top-level claim to subclaims, assumptions, evidence, and challenges. Responsibility is not an extra paragraph added after the technical design: it identifies who defines the boundary, who accepts the evidence, who authorizes the transition, and who reviews the record. NASA's public systems-engineering handbook and NIST's public AI Risk Management

Framework both emphasize lifecycle evidence, risk treatment, and defined responsibility [16, 14].

12.1 Foundations: requirements, safety cases, and assigned decisions

The final chapter integrates the course's recurring distinctions: evidence is not a decision, a warning is not an order, and a technically plausible action is not necessarily authorized. A safety-critical system needs an argument that connects its requirements, evidence, uncertainty, boundaries, owners, and recovery states. This module gives a vocabulary for that argument, drawing on systems-engineering lifecycle practice and risk-management principles [16, 14].

12.1.1 Requirements, constraints, and claims

A requirement states what a system shall do under specified conditions. A constraint limits the designs or behaviors that are allowed. A claim is an assertion made about the system. These terms are related but not interchangeable. For example:

- Requirement: "The system shall preserve source, time, and configuration for every state transition."
- Constraint: "A transition shall not proceed when authority has expired."
- Claim: "The reviewed configuration preserves traceability under the stated scenario."

The requirement can be verified. The constraint defines a hard boundary. The claim must be supported by a structured argument and evidence. A broad claim with no explicit conditions is difficult to challenge and therefore difficult to trust.

A **safety case** is a structured, reviewable argument that a system is acceptably safe or appropriately bounded for a stated context. It connects a top-level claim to subclaims, evidence, assumptions, justifications, challenges, and named owners. A safety case does not prove that no failure is possible; it makes the basis and limits of acceptance inspectable.

12.1.2 Argument structure

An abstract argument can be represented as

$$C \xleftarrow{A,J} \{C_1, C_2, \dots, C_n\} \leftarrow \{E_1, E_2, \dots, E_m\},$$

where C is a top claim, C_i are supporting claims, E_i are evidence items, A are assumptions, and J are justifications explaining why the evidence supports the claims. A challenge is a test or condition that could weaken one of these arrows. The arrows matter as much as the nodes: a calibration report may support a measurement-integrity claim but not a claim about organizational authorization.

For a fictional monitoring system, a top claim might be: “Under its declared operating envelope, the system enters a safe state when evidence, communication, authority, or resource conditions are insufficient.” Supporting claims might cover input integrity, estimation validity, constraint enforcement, interface behavior, decision record, and withdrawal process. Each supporting claim needs evidence and an owner.

12.1.3 Responsibility matrices

Complex systems distribute work. A responsibility matrix prevents a post-event discovery that each participant assumed another role owned a critical decision. A common classroom notation is RACI: Responsible performs the work, Accountable accepts the decision, Consulted provides input, and Informed receives notice. The model should not be used to remove judgment; it should identify which judgment is required and when.

Function	Designer	Operator	Authority	Reviewer
Define boundary and requirements	R	C	A	C
Evaluate configuration evidence	R	C	C	A
Authorize a bounded transition	I	C	A	I
Maintain decision record	C	R	A	C
Review outcome and residual risk	C	R	A	R

The exact role names vary by organization. The design requirement is stable: the record must show who supplied evidence, who evaluated it, who had authority, what rule version applied, and who owns review of the outcome.

12.1.4 Decision records and auditability

Let a compact decision record be

$$\rho = (z, \hat{x}, P, \kappa, \mathcal{C}, \mathcal{A}, \omega, t, y),$$

where z is evidence, $\hat{x}$ the state estimate, P uncertainty, κ configuration, $\mathcal{C}$ checked constraints, $\mathcal{A}$ authority state, ω responsible role, t time, and y the observed outcome. This is a teaching schema, not a universal database. Its purpose is to ensure that a final command is never mistaken for a full explanation.

Auditability means a suitably authorized reviewer can reconstruct what was known, what was uncertain, what configuration and rules applied, which alternatives were considered, and why the system entered its recorded state. It also preserves dissent and unresolved evidence, because silence is not agreement.

12.1.5 Uncertainty, time, and safe states

The most responsible response to uncertainty is sometimes neither acceptance nor escalation. It can be a safe state that limits function, preserves recovery, requests review, and records the unmet condition. Let $\mathcal{P}$ be the premises required for a transition. If

$$\neg \bigwedge_{p \in \mathcal{P}} p,$$

then the system should not silently use the nominal transition as if all premises held. It should enter a declared alternative state. This condition may arise from uncertain estimates, expired authority, missing acknowledgment, insufficient energy, or a negative time margin.

Worked example: a safety-case response, not a pass/fail label

A fictional review finds a high-confidence classification, an absent message acknowledgment, a rule version that may be stale, and limited time. The responsible output is a structured record: classification evidence supports one limited claim; communication and authority claims are unresolved; the safe state is selected; current function is bounded; and named roles must obtain the missing evidence. A binary “pass” would hide the actual decision structure. [Illustrative teaching value]

12.1.6 Assurance cases must be challenged

An argument improves when it includes its own challenge questions. For measurement integrity: What if the clock is wrong, the sensor is uncalibrated, or the environment shifts? For model validity: What population and unknown cases are absent? For authority: What happens after expiry or a rule update? For recovery: Can recovery depend on the same channel or component that has failed? Challenges convert generic confidence into specific, testable doubts.

[title=Named-owner safety review]

state the top claim, operating envelope, and protected-state condition
 decompose the claim into evidence, model, constraint, authority, and recovery
 subclaims
 link every evidence item to source, time, configuration, uncertainty, and owner
 test challenge cases, including stale data, disagreement, loss, and version
 conflict
 identify supported, unresolved, and rejected claims separately
 select a bounded state and record authorization, dissent, outcome, and residual
 risk

- Treating a warning score as a direct order or authority token.
- Storing the action while discarding evidence, uncertainty, configuration, and rejected alternatives.
- Assigning a generic team responsibility instead of naming a role for each transition.
- Declaring a system safe because a single demonstration passed, without a claim structure and challenge cases.
- Designing a recovery state that depends on the same information or resource whose loss triggered it.

12.1.7 Foundation self-check

1. Separate a requirement, a constraint, and a safety claim.
2. Draw a top claim with two subclaims and at least one challenge to each.
3. Why is the final command alone an incomplete decision record?
4. Give one condition that should enter a safe state rather than continue nominal behavior.

The expected answer names evidence, assumptions, owners, and a bounded response. A safety case is credible only when a reviewer can identify what could disprove it.

12.1.8 Bridge to the loss of signal meaning

The beacon-fire story is treated as a narrative prompt about trust between signal and response, not as an unqualified technical specification. Its system lesson is that credibility depends on documented evidence, valid authority, confirmation, memory of

prior use, and the possibility of a safe response that avoids unnecessary escalation. The final chapter uses the safety-case structure to bring these requirements together.

12.2 Historical opening and source note

The final lecture uses “win before seeking battle” as a systems-engineering question: before an irreversible command, have the evidence record, configuration record, authorization record, and recovery argument already been established? The final button is treated as the end of a long decision process, not as the whole decision.

12.2.1 Foundational knowledge to master

At a public conceptual level, a ballistic missile is a guided system whose trajectory includes a powered portion followed by largely ballistic flight, with later guidance or terminal functions depending on the system. Students learn the broad flight-phase distinction—boost, midcourse, and terminal—and the relation among propulsion, inertial navigation, guidance, control, re-entry or terminal environment, command authority, and defense assessment. No operational performance values, real target procedures, or launch instructions are included.

The lecture also introduces terminal-guidance precision as a systems issue: uncertainty accumulated during earlier phases must be represented, later observations must be time-valid, and a terminal decision must obey declared constraints. Intelligent trends are discussed as onboard estimation, adaptive filtering, improved sensing, decision aids, communication robustness, and verification. The “spear and shield” discussion compares offense and defense as interacting information systems, not as a design manual.

12.2.2 Learning content and mathematical derivation

Let the available decision window be T_a and the sequential delays be detection, confirmation, authorization, and bounded response. The margin is

$$M = T_a - (T_d + T_c + T_u + T_r).$$

If $M < 0$, the system cannot meet the nominal process under its current assumptions; the correct design response is to change the process, reduce scope, or enter a safe state.

A safety case can be represented as a claim tree. Let C be the top claim, C_i sub-claims, and E_i evidence. A review is not the assertion $C = 1$; it is the argument

$$\{E_i, A_i, \text{challenges}_i\} \Rightarrow C \quad \text{under configuration } \kappa.$$

Students must state assumptions and identify which challenge would break each supporting arrow. A final command without its premises is an incomplete record.

12.2.3 Case studio: strategic authorization and safety-case simulator

The safe capstone is a fictional control-room review. The input contains a high-confidence but out-of-scope classification, a missing message acknowledgment, an old rule version, a negative time margin, and schedule pressure. Students do not choose or simulate a weapon action. They produce a safety case with supported claims, unresolved claims, a safe state, evidence owners, configuration, dissent, and recovery plan.

The final presentation may use a 嘉立创 EDA status board with LEDs for warning, confirmation, authorization, safe state, and audit complete. The board is an interface demonstrator only. The main assessment is the argument: can an independent reviewer reconstruct what was known, what was uncertain, and who accepted the remaining risk?

12.2.4 Whole-chapter synthesis

The final chapter returns to Chapter 1. A warning is not a fact, a fact is not an authorization, an authorization is not an outcome, and an outcome is not yet an explanation. A complete safety-critical-system record keeps the evidence, configuration, rule, owner, uncertainty, safe state, dissent, and residual risk together.

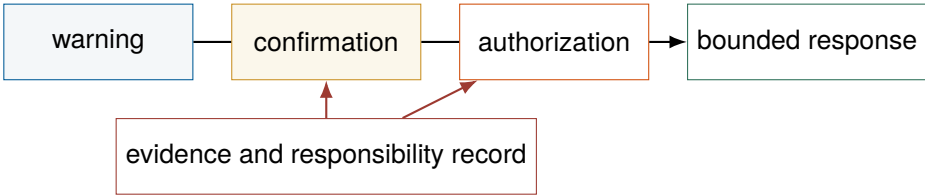
12.2.5 Practice and extension

1. Draw the public system architecture and flight-phase information needs of a ballistic missile.
2. Calculate decision margin under two review-chain delays.
3. Separate warning, confirmation, authorization, response, and outcome.
4. Build a top-level safety claim with four supporting claims and one challenge per claim.
5. Write the final-course responsibility statement for a fictional safety-critical system.

A claim is a statement that may be true or false. Evidence is material that supports or weakens the claim. A challenge is a condition or test that could show the claim is too broad, based on a wrong assumption, or unsupported by the available record.

12.3 A warning is not an order

Large strategic systems compress time but cannot erase uncertainty. A warning may be generated from sensing and estimation; confirmation checks source quality and competing hypotheses; authorization checks rule, authority, and responsibility; response is a separate state. Combining them into a single “alert” button hides the very distinctions that make the system governable.



12.4 Responsibility is not a single button

Let the complete decision record be

$$\rho = (z, \hat{x}, \kappa, \mathcal{C}, \mathcal{A}, \omega, t),$$

where z is the evidence, $\hat{x}$ the estimated state, κ the configuration, $\mathcal{C}$ the checked constraints, $\mathcal{A}$ the authority information, ω the responsible role, and t the time. If an organization keeps only the final command, it cannot later establish whether the command was supported, authorized, or even based on the same system version that was reviewed.

12.5 Safety cases

A safety case is a structured argument connecting a claim, evidence, assumptions, and challenges. For a fictional safety-critical evaluation, a top claim might be: “The system enters a safe state when confidence, communication, authority, or time conditions are insufficient.” Supporting claims would cover sensing integrity, model validation, constraint enforcement, human interface, logging, and withdrawal behavior. Every claim must identify evidence and an owner.

Worked example

Capstone exercise: a fictional review combines the course’s recurring failures: a high-confidence classification, a missing message acknowledgment, a conflicting rule version, and schedule pressure. The correct output is not a simple pass/fail label. It is a safety case: which claims are supported, which are unresolved, which safe state follows, and who must sign each part of the record? [Illustrative teaching value]

12.6 Warning, confirmation, authorization, and response

Large systems compress time but cannot erase uncertainty. A warning may be generated from sensing and estimation; confirmation checks source quality and competing hypotheses; authorization checks rule, authority, and responsibility; response is a separate state. Combining them into a single “alert” button hides the distinctions that make the system governable.

Decision responsibility is the documented relation among evidence, estimation, rule interpretation, authorization, action, outcome, and review. It identifies the technical function and the person or organization responsible at each stage.

Stage	Required evidence	Failure if collapsed
Warning	source, time, quality, uncertainty	alert treated as fact
Confirmation	independent check and alternatives	repetition treated as corroboration
Authorization	valid rule, authority, configuration	recommendation treated as permission
Response	bounded action and stop condition	action has no owner or recovery
Outcome	observed result and residual state	intent mistaken for execution

12.7 Safety cases and structured arguments

A safety case connects a claim, argument, evidence, assumptions, and challenges. A top claim for a fictional safety-critical evaluation might be: “The system enters a safe state when confidence, communication, authority, or time conditions are insufficient.” Supporting claims cover sensing integrity, model validity, constraint enforcement, human interface, logging, recovery, and lifecycle withdrawal.

For a claim C , let E_C be its evidence and A_C its assumptions. A responsible review does not write C as established merely because E_C exists. It asks whether E_C actually supports C under A_C , whether the assumptions are tested, and which challenge would invalidate the argument.

Worked example: a compressed decision window

A fictional review combines the course’s recurring failures: a high-confidence classification, a missing message acknowledgment, a conflicting rule version, and schedule pressure. A single pass/fail label hides the structure. A safety-case response lists supported claims, unresolved claims, the safe state that follows, the evidence still required, and the role responsible for each decision. [Illustrative teaching value]

12.8 Responsibility matrix

The same role should not be assumed to own every link. A responsibility matrix can assign functions to roles such as system designer, test engineer, operator, authority, maintainer, and independent reviewer. The letters below are classroom notation: R means performs the work, A means accepts accountability, C means consulted, and I means informed.

Function	Designer	Operator	Authority	Reviewer
Define system boundary	R	C	A	C
Validate model scope	R	C	C	A
Authorize current task	I	C	A	I
Record outcome	C	R	A	C
Verify end state	C	R	A	R

The matrix does not remove judgment. It prevents the organization from discovering after an incident that everyone believed someone else owned the decision.

12.9 Decision record and auditability

Let

$$\rho = (z, \hat{x}, \kappa, \mathcal{C}, \mathcal{A}, \omega, t, y),$$

where z is evidence, $\hat{x}$ the estimated state, κ configuration, $\mathcal{C}$ checked constraints, $\mathcal{A}$ authority information, ω responsible role, t time, and y outcome. The tuple is not a universal database schema. It is a reminder that a final command without its premises is not a complete explanation.

[title=High-consequence review loop]

receive warning and verify source, time, and configuration
maintain competing hypotheses and unresolved disagreement
check current rules, authority, time margin, and hard constraints

```
select a bounded state or request review
obtain explicit authorization where required
observe outcome and enter a safe state when premises fail
archive evidence, decision, dissent, and residual risk
```

12.10 Trust as a system variable

The beacon story shows that a signal has meaning only within a relationship. If a warning channel is repeatedly used without credible conditions, the receiver changes its response policy. This becomes a systems problem involving calibration, alert fatigue, authority, and organizational memory. Trust is an operational assumption that can be damaged, measured indirectly, and restored only through evidence.

Before a high-consequence system is accepted, ask: Can every warning be traced? Can a reviewer distinguish signal from order? Can the system refuse an out-of-scope request? Can authority expire? Can an independent person reconstruct what was known, what was uncertain, and why the system entered its final state?

12.11 Course synthesis exercise

Return to all twelve narratives and write one sentence for each in the form: “The story gives us an image of ____; the engineering object is ____; the limits of the evidence are ____.” Then assemble the sentences into one record of decisions and responsibilities. The aim is not to force history into a single theory. It is to show that the same discipline recurs: define the boundary, state the evidence, represent uncertainty, constrain action, preserve recovery, and assign responsibility.

12.12 Historical and methodological interpretation

The beacon-fire story varies across historical and literary versions. It prompts discussion of signal credibility and institutional memory, rather than serving as an unqualified historical record. The technical lesson is that speed must be paired with confirmation, authority, records, and safe states that avoid unnecessary escalation.

12.13 Challenges to a safety case

A safety case should be challenged by asking what would make each supporting claim false. For measurement integrity, challenge the timestamp, calibration, and common-mode assumptions. For authority validity, challenge expiry, rule version, and issuer identity. For recovery, challenge energy, communication, location, and human workload. A claim that has no credible challenge is often too vague to be useful.

Claim area	Example evidence	Challenge question
Sensing integrity	calibration and replay test	What if the environment shifts?
Model validity	held-out evaluation	What states are missing?
Constraint enforcement	fault-injection test	Can a software path bypass it?
Authority	signed and versioned record	What happens after expiry?
Recovery	scenario and stress tests	Can recovery depend on the failed channel?

12.14 Safety-case exercise

Give each student a different role: designer, operator, authority, maintainer, reviewer, or affected stakeholder. Present a fictional unresolved warning and ask each role to write what it knows, what it does not know, what it may authorize, and what record it requires. The class then compares how a single button hides multiple responsibilities.

12.15 Historical notes and further reading

The beacon story is useful because it makes trust part of signal quality, but sources must be checked version by version. For strategic and safety-critical systems, the primary technical lesson is not speed alone: it is the design of confirmation, authority, records, and safe states that avoid unnecessary escalation. See [13, 15] and the reading protocol in Appendix C.

12.16 Exercises

- 12.1 Draw a four-stage warning-to-response chain and name one distinct failure mode at each stage.
- 12.2 Define the record tuple ρ for a fictional laboratory safety review. Explain why every field is needed.

- 12.3** Write a top-level safety claim and four supporting claims for a fictional safety-critical autonomous system.
- 12.4** A manager requests a single “pass/fail” conclusion under unresolved evidence. Draft a responsible technical reply.
- 12.5** Return to Chapter 1. Explain how its functional boundary reappears in this chapter’s assignment of responsibility.

Chapter summary

- A warning, a confirmation, an authorization, and a response have different evidence requirements.
- Responsibility must be documented across technical and organizational layers.
- In safety-critical systems, safe states and honest uncertainty are system capabilities, not signs of weakness.

Chapter A

Mathematical Foundations

A.1 Vectors, matrices, and state descriptions

An engineering state is a compact description of the variables that matter for the next decision. For a ground vehicle, a teaching model may use

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \psi_k & v_k \end{bmatrix}^T,$$

where (x_k, y_k) is position, ψ_k is heading, and v_k is speed. A control input may be written as $\mathbf{u}_k = [a_k, \omega_k]^T$.

The notation does not claim that four variables capture a real platform. It makes the modeling boundary visible. A state that omits energy, communication, or uncertainty can produce a mathematically elegant but operationally incomplete answer.

A.2 Probability and conditional reasoning

For events A and B , conditional probability is $P(A \mid B) = P(A \cap B)/P(B)$ when $P(B) > 0$. Bayes' rule gives

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

The equation is useful only when the events, priors, measurements, and population are defined. A displayed confidence value without its reference population is not a complete argument.

A.3 Optimization and constraints

Many system decisions have the form

$$\min_{z \in \mathcal{F}} J(z), \quad \mathcal{F} = \{z : g_i(z) \leq 0, i = 1, \dots, m\}.$$

The feasible set $\mathcal{F}$ defines admissibility, rather than a penalty alone. If a decision is outside it, a lower numerical score cannot make it admissible. This distinction appears repeatedly in chapters on planning, thresholds, and authorization.

A.4 A reading rule for equations

For every equation in this book, ask four questions: What is the object being modeled? Which assumptions make the equation meaningful? Which variables are measured

and which are estimated? What decision is the equation allowed to inform, and what decision is outside its scope?

A.5 Norms, distances, and uncertainty

For a vector $x \in \mathbb{R}^n$, the Euclidean norm is

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}.$$

It describes geometric distance only after the coordinates have compatible units or have been normalized. A state error of one meter, one degree, and one unit of confidence cannot be added meaningfully without a declared scaling. Weighted norms can make the trade-off explicit:

$$\|x\|_W^2 = x^\top W x, \quad W \succeq 0.$$

The matrix W is a modeling decision. It should be documented because it changes what “close” means.

If a random vector x has mean μ and covariance P , then the covariance entries are

$$P_{ij} = \mathbb{E}[(x_i - \mu_i)(x_j - \mu_j)].$$

The diagonal describes marginal variance; the off-diagonal terms describe dependence. Ignoring the off-diagonal terms can make a fused estimate appear more certain than it is.

A.6 Linear algebra for interfaces

Matrices often represent transformations between coordinate systems or between physical states and observations. A matrix product is defined only when dimensions match, but dimensional compatibility is not semantic compatibility. Two vectors of the same length can still refer to different frames, timestamps, or units.

For a linear model $z = Hx$, the rows of H describe which state combinations are visible to the measurement. If two rows are nearly dependent, the measurements may add less independent information than their count suggests. The singular value decomposition

$$H = U \Sigma V^\top$$

helps identify weakly observed directions. In this course, the observation question is as important as the algebraic question: which state variable can the system actually distinguish under its sensors and conditions?

A.7 Conditional probability and likelihood ratios

Bayes' rule is often clearer in odds form:

$$\frac{P(H_1 | Z)}{P(H_0 | Z)} = \frac{P(Z | H_1)}{P(Z | H_0)} \frac{P(H_1)}{P(H_0)}.$$

The first ratio is a likelihood ratio and the second is a prior-odds term. A measurement changes belief by its likelihood ratio; it does not erase the base rate. This is why a classifier score without a reference population is not a complete probability statement.

For multiple measurements, conditional independence is an assumption:

$$P(z_1, z_2 | H) = P(z_1 | H)P(z_2 | H).$$

When sensors share a map, clock, or model, the assumption may fail. The correct response is to model dependence, add a conservative uncertainty, or request an independent observation.

A.8 Optimization and the feasible set

An optimization problem has three distinguishable parts: decision variables, objective, and constraints. Write

$$\min_{z \in \mathcal{F}} J(z), \quad \mathcal{F} = \{z : g_i(z) \leq 0, i = 1, \dots, m\}.$$

If $\mathcal{F} = \emptyset$, the correct conclusion is infeasibility. It is not correct to choose the least bad infeasible point without explicitly changing the policy.

For a constrained problem, a Lagrangian may be written

$$\mathcal{L}(z, \lambda) = J(z) + \sum_i \lambda_i g_i(z), \quad \lambda_i \geq 0.$$

Penalty terms can be useful for numerical search, but a penalty implementation must not be confused with an authorization rule. A hard boundary should be checked separately and should generate a protected response when violated.

A.9 Graphs and search

Let a graph be $G = (V, E)$, with nodes V representing states or regions and edges E representing allowed transitions. A path $P = (v_0, \dots, v_m)$ has cost

$$J(P) = \sum_{j=0}^{m-1} c(v_j, v_{j+1}).$$

Breadth-first search is appropriate for equal edge costs; Dijkstra's algorithm handles nonnegative weights; heuristic search adds an estimate of remaining cost. The method is secondary to the model: if a forbidden transition is included as an edge, an optimal search can return an unacceptable path.

A.10 Difference equations and stability intuition

A simple scalar feedback error model is

$$e_{k+1} = ae_k.$$

When $|a| < 1$, the error decays in the nominal model; when $|a| > 1$, it grows; when a is near -1 , oscillation may persist. Real systems add disturbance, delay, saturation, and uncertainty, so this is intuition rather than a certification. It is enough to explain why feedback, constraints, and test conditions must be stated together.

A.11 Practice problems

1. Normalize three capability measurements with one benefit and one burden attribute. Explain the chosen reference range.
2. Compute a posterior probability for a rare event and describe how the answer changes when the base rate changes.
3. Give a two-variable optimization problem whose unconstrained optimum violates a hard boundary. Draw the feasible set.
4. Construct a graph with one low-cost but inadmissible edge. Explain how a search algorithm should represent it.
5. For $e_{k+1} = 0.8e_k + w_k$, identify which assumptions would be required before calling the system stable.

Chapter B

Algorithmic Notation and Pseudocode

B.1 A minimal pseudocode convention

Algorithms in this book use explicit inputs, outputs, state updates, and stop conditions. A safe state is an ordinary state in the model, not an afterthought.

Generic evidence-aware loop

```
while mission_window_is_open:
    observation <- acquire_measurement()
    estimate <- update_state(observation)
    if evidence_is_out_of_scope(estimate):
        state <- REQUEST_REVIEW
    elif hard_constraints_fail(estimate):
        state <- SAFE_HOLD
    else:
        recommendation <- evaluate_options(estimate)
        state <- await_authorization(recommendation)
    record(observation, estimate, state, owner, version)
```

The pseudocode intentionally does not contain a real-world weapon action. Its purpose is to show that uncertainty, constraints, authorization, and records belong inside the loop.

B.2 Completeness, correctness, and explainability

An algorithm may be complete relative to a stated model and still be unsuitable when the model is wrong. In this book, a claim of completeness must always name the environment, sensing assumptions, state space, stopping condition, and failure behavior.

B.3 Algorithm contracts

Every algorithm should have a contract with four parts: preconditions, inputs, outputs, and postconditions. For example, a state update requires valid timestamps, a defined configuration, and a measurement within the declared sensor scope. Its output is not merely a number; it is a state estimate with uncertainty, age, and provenance.

Contract item	Example	Review question
Precondition	frames and timestamps are aligned	What checks reject invalid input?
Input	measurement z_k and prior $(\hat{x}, P)$	Which variables are observed?
Output	estimate, covariance, status	Does status preserve uncertainty?
Postcondition	record written and version stored	Can a reviewer reproduce the update?
Failure behavior	request review or safe state	Is failure bounded and owned?

B.4 Complexity and resource budgets

Let n be the number of states or samples. An algorithm with time complexity $O(n^2)$ may be acceptable for a small classroom graph but unacceptable when the update rate is fixed and the input grows. Complexity must be compared with memory, communication, energy, and latency budgets.

Two questions matter: “How fast is the algorithm?” and “What happens when the budget is exceeded?” A robust design specifies a reduced-scope mode, a deferred computation, a bounded approximation, or a safe state. Silent truncation changes the algorithm’s meaning.

B.5 Testing levels

Testing should proceed from small to large units:

1. unit test: one transformation or decision rule;
2. interface test: data type, timestamp, version, and failure signal;
3. integration test: multiple functions under a shared configuration;
4. scenario test: a sequence of observations and state transitions;
5. stress test: delay, missing data, conflicting sources, and resource limits;
6. audit test: reconstruct a selected outcome from stored records.

Passing a unit test does not establish scenario safety. The levels answer different questions and should be reported separately.

[title=Record-reconstruction test]

```
load a fictional scenario and its declared configuration
inject one known disturbance or missing field
run the algorithm and collect outputs plus records
check numerical result, state transition, timing, and failure behavior
attempt an independent reconstruction from the records alone
report passed checks, violated assumptions, and unresolved ambiguity
```

B.6 Explainability is a record property

An explanation is not necessarily a long paragraph generated after the fact. For the purposes of this book, an explanation is a concise relation among input evidence, model version, influential variables or constraints, selected option, rejected alternatives, and authority state. An explanation that cannot be linked to the actual configuration is a narrative, not an audit.

B.7 Algorithm review questions

1. What is the algorithm allowed to decide, and what is outside its contract?
2. What input condition causes it to refuse an answer?
3. Which variables determine the stop condition?
4. What is the worst-case delay or resource burden in the declared environment?
5. How can a reviewer distinguish a correct result from a lucky result?
6. Which record proves the exact configuration used?

Chapter C

Source Limits and Reading Protocol

C.1 Material labels

Label	Material type	Permitted use
H	Historical narrative	State the textual core; do not invent unmarked scene details.
P	Philosophical parable	Use as a conceptual analogy; do not present it as an engineering record.
R	Rhetorical or political story	Analyze persuasion and transmission; do not treat repetition as independent evidence.
L	Literary fiction	Discuss the literary construction; do not cite it as direct history.
C	Classroom example	Use example values and fictional cases; do not generalize them to real systems.

C.2 Source classes and citation discipline

The labels H, P, R, L, and C are not judgments about whether a source is “good” or “bad.” They tell the reader what kind of claim the material can support. A historical narrative can establish that a text tells a story; it cannot establish a sensor performance value. A classroom example can clarify a mathematical idea; it cannot be reported as field data.

Claim form	Suitable evidence	Common overreach
"The text says..."	primary text or reliable translation	treating later scene detail as primary fact
"The model assumes..."	equation and assumption list	presenting assumption as observation
"The test observed..."	protocol, data, and log	generalizing beyond test conditions
"The system may..."	authority rule and safety case	confusing recommendation with permission
"The result is reproducible..."	configuration and independent rerun	counting copied outputs as replications

C.3 The four-sentence source check

For every historical opening, state: what the text says; what the text does not say; why the class still uses it; and how the modern system problem differs. This protocol is not a decorative disclaimer. It is the first example of keeping a source record in a course about intelligent systems.

C.4 Reading a translation

When a classical text is read in translation, mark the translator, edition, section, and link. Separate direct quotation from paraphrase. When wording differs between translations, record the difference rather than silently choosing the version that best fits the lesson. A story can remain vivid while its source history is handled carefully.

C.5 A student evidence card

C.6 The four-sentence source check

For every historical opening, state: what the text says; what the text does not say; why the class still uses it; and how the modern system problem differs. This protocol is not a decorative disclaimer. It is the first example of keeping a decision record in a course about intelligent systems.

C.7 A student evidence card

Claim	What exactly is being claimed?
Source	Which text, dataset, log, or experiment supports it?
Interpretation	What inference is added by the reader or engineer?
Boundary	What cannot be concluded from the evidence?
Decision	Which action, if any, is actually authorized?

C.8 Evidence-card workflow

Complete the card in five passes. First, write the claim without persuasive adjectives. Second, attach the closest primary source. Third, list the transformations from source to current statement. Fourth, write at least one alternative explanation. Fifth, specify the permitted educational or engineering use. Exchange cards with a partner who is instructed to challenge scope rather than style.

Field	Prompt
Claim	What exactly is being claimed?
Source	Which text, dataset, log, or experiment supports it?
Transformation	What was translated, filtered, averaged, or inferred?
Alternative	What else could explain the observation?
Boundary	What cannot be concluded from the evidence?
Decision	Which action, if any, is authorized in the scenario?
Open question	What observation would most reduce uncertainty?

C.9 Review protocol for course projects

Before submitting a project, each group should identify one claim that is well-supported, one claim that is conditional, and one claim that must be withdrawn. The group should also name the configuration, the evidence owner, and the unresolved risk. This practice rewards intellectual honesty and produces better engineering decisions than a forced single conclusion.

Chapter D

Case Studio: Twelve Observation-to-Decision Exercises

This appendix turns the twelve narrative openings into compact engineering case studies. Each case is fictional and safe for classroom analysis. The purpose is to practice modeling, source limits, and responsibility rather than to reproduce a historical event or provide a field procedure.

D.1 Case 1: The changed trace

A monitoring team receives two observations of a marked route. The first report says that activity has decreased; the second reports that the map is missing a time stamp. The team must decide whether to update the route estimate, request a new observation, or enter a safe state.

1. Draw the observation, estimate, recommendation, authorization, and outcome layers.
2. List two alternative explanations for reduced activity.
3. Write the minimum record required for an audit.

D.2 Case 2: Three platforms, four tasks

Three fictional inspection platforms have different mobility, sensing, energy, and communication profiles. Four tasks are requested, but only three can be performed within the time window. The platform with the highest average score is also the least observable to the supervisor.

1. Build a capability matrix and a task-admissibility matrix.
2. Formulate a constrained allocation problem.
3. Explain which task remains unassigned and who owns the deferral.

D.3 Case 3: Waiting is a state

A link-quality estimate crosses a threshold for six seconds and then returns to normal. A second sensor reports a persistent timing error. The current state machine has only NORMAL and FAILED.

1. Add WAIT, VERIFY, and SAFE_HOLD states.
2. Define a guard, invariant, timer, and expiry action for each new state.
3. Identify a transition that should not be triggered by the link event alone.

D.4 Case 4: The open corridor

A ground platform follows a corridor in a semantic map. A new observation marks one region as unknown rather than occupied. The global path remains geometrically valid but has no recovery margin near the unknown region.

1. Separate geometry, traversability, uncertainty, and recovery layers.
2. Give two locally feasible alternatives and state their trade-offs.
3. Write a replanning record that explains why the original path was rejected.

D.5 Case 5: The confident label

A classifier assigns the same high score to three inputs. One is in-distribution, one is out-of-distribution, and one is outside the declared task scope.

1. Define a response for each input without changing the score.
2. State the calibration population and the unknown-input policy.
3. Explain why confidence does not equal authorization.

D.6 Case 6: The system that was turned off

A platform has stopped moving, but its local policy cache, stored credentials, and final location record have not been verified. The project manager declares the mission complete.

1. Define physical stop, policy revocation, and verified end state.
2. Construct an end-state checklist with evidence owners.
3. Identify one residual risk that survives the visible shutdown.

D.7 Case 7: The shrinking window

A laboratory monitor produces a moderate anomaly score. Manual review takes four minutes, while the estimated time to an irreversible condition is decreasing.

1. Define observe, verify, degrade, and safe states.

2. Write a time-margin inequality.
3. Explain when a lower-confidence protected response is rational.

D.8 Case 8: The unanswered message

A local system generates an important record, attempts transmission twice, and receives no acknowledgment. The queue is growing while energy reserve falls.

1. Draw the message-state chain.
2. Separate transmission evidence from delivery evidence.
3. Define a local policy that preserves a recovery margin.

D.9 Case 9: Three documents, one source

An engineering claim appears in a report, a slide deck, and a student handout. All three cite one preliminary note.

1. Draw the provenance graph.
2. Identify which claim can be retained and which claim must be narrowed.
3. Specify the missing independent evidence.

D.10 Case 10: Correlated agreement

Two sensors agree because both use the same stale map. A third sensor disagrees but has lower precision and an independent clock.

1. Write a measurement model for all three sensors.
2. State the correlation assumption needed for fusion.
3. Design an interface that preserves disagreement instead of hiding it.

D.11 Case 11: Three maps

One map is current and coarse, one is detailed and stale, and one is current but optimized for a different task.

1. Build the metadata tuple for each map.
2. Explain why route averaging is not a valid resolution of the conflict.
3. Define an evidence-driven replanning trigger.

D.12 Case 12: The warning chain

A warning is generated, a confirmation is delayed, an authorization is signed under an old rule version, and the outcome log contains only the final command.

- 1. Reconstruct the sequence of decisions and responsibilities.
- 2. Write a top-level safety claim and four supporting claims.
- 3. List the evidence that must be recovered before the review can close.

D.13 Instructor synthesis rubric

Each case can be assessed on five dimensions, each scored from 0 to 2: source limits, model clarity, uncertainty representation, constraint and recovery logic, and record reconstruction. A high score requires students to explain what the system should do, why the evidence permits that action, and who is responsible for the transition.

Dimension	Strong response	Weak response
Source limits	separates text, analogy, and construction	treats story as technical evidence
Model clarity	variables and assumptions are explicit	uses vague labels
Uncertainty	retains age, variance, or alternatives	converts all output to certainty
Recovery	defines safe states and expiry	describes only the nominal path
Record reconstruction	records owner, version, evidence, and outcome	records only final action

Chapter E

Study Guide, Laboratory Templates, and Review Sheets

E.1 A twelve-week reading rhythm

For each lecture, students should complete four passes. Before class, read the narrative source boundary and write one question. During class, annotate the system diagram and identify one assumption. After class, solve one mathematical exercise and one evidence exercise. At the end of the week, revise the original question in light of the model.

Pass	Student action	Evidence of learning
Before class	source and question	one-sentence reading card
During class	diagram and assumption	annotated architecture
After class	equation and case	worked calculation plus decision record
Weekly review	revise the question	short reflection with changed claim

E.2 Laboratory 1: system boundary map

Choose a safe monitoring or logistics scenario. Draw physical components, logical functions, human roles, communication paths, and record stores. For every arrow, write a data type and a failure indication. Submit the diagram with a one-page boundary statement.

E.3 Laboratory 2: allocation matrix

Create four fictional platforms and four tasks. Normalize three capabilities, define two hard constraints, and compute at least two feasible assignments. Explain the effect of changing one weight. The report must include the rejected alternative and the reason for rejection.

E.4 Laboratory 3: state-machine test

Design a state machine with normal, verify, degraded, protected, recovery, and terminate states. Create five event sequences, including a missing acknowledgment and

a timer expiry. For each sequence, state the expected state, record, and owner.

E.5 Laboratory 4: map and feedback

Construct a small grid map for a fictional ground platform. Mark occupied, unknown, traversable, and recovery regions. Compare a global corridor and a local detour. Explain which map update caused replanning and how feedback changes the next estimate.

E.6 Laboratory 5: calibration table

Make a table of score bins for a fictional classifier. For each bin, record mean score and observed correctness. Identify overconfidence, underconfidence, and an unknown-input policy. Do not report a single accuracy number without the population and conditions.

E.7 Laboratory 6: lifecycle gate review

Prepare a gate review from design to verified end state. For each gate, name the claim, evidence, assumption, decision authority, and unresolved risk. Include at least one change-management event and one withdrawal event.

E.8 Laboratory 7: threshold and time margin

Create a synthetic sequence of anomaly scores and a decreasing time margin. Define four states and thresholds. Explain why the same score can lead to different decisions at different times.

E.9 Laboratory 8: message audit

Simulate generated, queued, transmitted, acknowledged, timed-out, and archived messages. Compare the local queue with the receiver log. Identify a case in which the sender's record is true but the receiver's record is absent.

E.10 Laboratory 9: provenance graph

Collect five sources on an intelligent-system topic. Draw citation and derivation edges, mark shared origins, and write one claim that must be narrowed. The submission must distinguish source count from independent evidence count.

E.11 Laboratory 10: fusion note

Generate three one-dimensional measurements with different variances and one shared bias. Compute an estimate under an independence assumption, then write a second note that acknowledges correlation. Explain why the second note is more honest even if it looks less precise.

E.12 Laboratory 11: navigation metadata

Create three map records with different timestamps, frames, purposes, and resolutions. Produce one route recommendation and one refusal to recommend. The refusal should state which metadata conflict must be resolved.

E.13 Laboratory 12: safety case

Write a top-level claim for a fictional safety-critical system and decompose it into five supporting claims. For each support, list evidence, assumptions, challenge, owner, and safe response. End with a one-page responsibility matrix.

E.14 Final examination review sheet

Students should be able to answer the following without notes:

1. What is the difference between a measurement, an estimate, a recommendation, an authorization, and an outcome?
2. Why is a system boundary a technical and ethical decision?
3. How do hard constraints differ from penalty terms?
4. Why do state machines need timers and exit conditions?
5. Why does a high confidence score not authorize a response?
6. What does covariance contribute to an estimate?
7. How can a message be transmitted without being acknowledged?
8. Why does a repeated citation not necessarily add independent evidence?
9. What metadata makes two maps comparable?
10. How does a safety case connect evidence to responsibility?

E.15 Project report template

Every final project should contain: title and scope; source boundary; system boundary; assumptions; variables and equations; architecture diagram; state machine; one worked example; failure modes; verification plan; lifecycle and withdrawal plan; responsibility matrix; limitations; and references. The report should explicitly mark fictional values and should not include operational procedures for real weapons.

Chapter F

Glossary of Core Terms

Admissible action An action that satisfies the current environment, rule, authority, review, and safety constraints.

Association The process of deciding which state or track a measurement belongs to.

Authorization A bounded permission issued by a valid authority under stated conditions.

Calibration The relation between a score and the observed frequency of the stated event under declared conditions.

Capability profile A task-specific description of what a platform can sense, compute, communicate, move, and report, including limitations.

Feedback system A system arrangement in which action changes the environment and new observations feed back into estimation and decision.

Common-mode error An error shared by multiple sources because they depend on a common origin or condition.

Configuration The model, data, software, rules, interface, hardware, and versions that define a system instance.

Constraint A condition that limits the feasible set of decisions or trajectories.

Covariance A matrix describing variance and dependence among uncertain variables.

Source limits A statement of what a source supports, what it does not support, and how it may be used.

Event An occurrence that may enable a state transition.

Feasible set The set of decisions satisfying all declared constraints.

Guidance The function that selects a desired path, trajectory, or reference state.

Guard A predicate that must be true before a state transition is enabled.

Hard constraint A condition that cannot be traded away by a lower objective value.

Human authority The role or organization that owns a permission, boundary, or consequential decision.

Independent evidence Evidence whose observation path, processing, assumptions, and failure modes are sufficiently distinct for the claim under review.

Information age The time elapsed since a measurement, estimate, or authority statement was validly produced.

Invariant A condition that must remain true while a state is active.

Lifecycle gate A review point before a system changes from one stage to another.

Local planning Short-horizon planning using recent observations and platform constraints.

Map metadata Source, time, resolution, purpose, frame, and validity information attached to a map claim.

Observation A measurement or report produced by a source at a time and under a configuration.

Safe state A state with bounded permitted behavior used when normal mission logic is unsupported.

Recommendation A proposed option produced by a decision function; it is not necessarily authorized.

Replanning Revision of a path or option set after a premise, constraint, or uncertainty changes.

Residual risk Risk that remains after controls, including risk after visible mission completion.

State A condition with permitted behavior, forbidden behavior, entry evidence, exit evidence, and ownership.

State estimate An inferred description of variables relevant to the next decision, together with uncertainty and age.

System boundary The declared components, interfaces, authorities, assumptions, and records included in an analysis.

Threshold A declared boundary in an evidence statistic that changes a permitted state or initiates review.

Record reconstruction The ability to connect an outcome to evidence, transformation, model, constraints, authority, configuration, and owner.

Traversability A model-based estimate of whether a platform can move through a region while satisfying constraints.

Verification The process of checking whether a system or record satisfies a stated requirement or claim.

Chapter G

Extended Worked Examples

The examples in this appendix are deliberately small enough to calculate by hand. Their purpose is to connect equations to engineering judgment. All platforms, missions, values, and responses are fictional and suitable for classroom analysis.

G.1 Boundary completeness

Suppose a monitoring system is described by $z_k = h(x_k) + v_k$ and $x_{k+1} = f(x_k, u_k) + w_k$. A draft architecture lists only sensor, processor, and platform. The missing human authority and record server are not decorative additions. Without them, the model has no representation for authorization, intervention, or later reconstruction. The revised state is

$$X_k = (x_k, e_k, \ell_k, \kappa_k, a_k, r_k),$$

where e_k is energy, ℓ_k communication quality, κ_k configuration, a_k authority state, and r_k record status. The expanded state is less elegant but more honest about the decision.

G.2 Assignment with an inadmissible pairing

Let three platforms and three tasks have costs

$$C = \begin{bmatrix} 3 & 7 & 4 \\ 5 & 2 & 8 \\ 6 & 4 & 3 \end{bmatrix}.$$

The lowest-cost diagonal has cost 8. Now impose $b_{33} = 0$, meaning that platform 3 cannot perform task 3 under its current interface. The diagonal is removed from the feasible set. The next feasible permutation must be computed, and the record should state that the choice changed because admissibility changed, not because the original arithmetic was wrong.

G.3 A state-machine invariant

Let the protected-state invariant be $e_k \geq e_{\min}$ and $\ell_k \geq 0$. If e_k decreases by 0.6 units per step and starts at 4.0, the nominal invariant is violated after six steps if no recovery action changes the consumption. The example is simple, but it forces the designer to define what happens before the sixth step. A timer that waits until the invariant is already false is not a protective timer.

G.4 Grid planning and unknown cells

Consider a 5×5 grid with start at $(1, 1)$ and goal at $(5, 5)$. An unknown cell has cost $c_u = 8$, a known traversable cell has cost 1, and an occupied cell is forbidden. A shortest geometric path through the unknown cell can be cheaper than a detour only if the unknown cost is treated as a preference. If the unknown cell is a hard uncertainty boundary, it must be removed from the feasible set until verified. The distinction changes the graph, not just the edge weight.

G.5 Base-rate calculation

Let an event have base rate 0.02, sensitivity 0.90, and false-positive rate 0.10. Then

$$P(A | +) = \frac{0.90 \cdot 0.02}{0.90 \cdot 0.02 + 0.10 \cdot 0.98} \approx 0.155.$$

The detector is useful for screening but a positive result is not a complete conclusion. Students should write a response policy that requests a second, independent observation rather than turning the positive label into an authorization.

G.6 Lifecycle residual risk

Suppose residual risk is $R(t) = 0.8e^{-0.2t} + 0.1$ after a control is installed. The first term decays, but the asymptote 0.1 remains. A visible improvement therefore does not prove zero residual risk. An end-state review should record the asymptotic condition, the owner, and the evidence required to accept or transfer it.

G.7 Time margin

Let the estimated time to an irreversible condition be $t_c = 18$ minutes, and let review and response require $T_r = 5 + 2$ minutes under a nominal delay model. At current time $t = 10$, the nominal margin is $18 - 10 - 7 = 1$ minute. If delay has a two-minute uncertainty margin, a safe state should be considered before the nominal margin reaches zero. The lesson is that mean time is not the same as guaranteed time.

G.8 Queue evolution

Assume a local queue begins with $q_0 = 2$ records. Over four communication opportunities, generated records are $(1, 0, 2, 1)$ and acknowledged deliveries are $(0, 1, 1, 1)$. The queue sequence is

$$q_1 = 3, \quad q_2 = 2, \quad q_3 = 3, \quad q_4 = 3.$$

The final queue is not evidence that three records were received. It is evidence that three records remain under the chosen queue definition. A receiver log is required for delivery claims.

G.9 Provenance graph counting

Suppose a claim in document A is copied to B and C, while A cites one preliminary note P. The graph has four document nodes but only one primary observation path. If an independent experiment Q supports the same claim, the evidence graph gains a new path. The number of documents is not the same as the number of independent paths.

G.10 Correlated sensor fusion

Two measurements are $z_1 = 10$ and $z_2 = 12$, each with variance 1, but both share an unknown bias b . Under independence, the estimate would be 11 with variance 0.5. Under a shared bias, the variance cannot be reduced below the bias uncertainty. The correct engineering action is to model b , obtain an independent source, or report the estimate as conditional.

G.11 Tracking error

Let $e_{k+1} = 0.7e_k + 0.2$ with $e_0 = 1$. The first values are 1, 0.9, 0.83, and 0.781. The error approaches the fixed point $e^* = 0.2/(1 - 0.7) = 0.667$, not zero. A controller can reduce transient error while leaving a systematic bias. A useful test examines the first response together with steady-state error and model mismatch.

G.12 Safety-case decomposition

For the top claim “the system enters a safe state when evidence is insufficient,” supporting claims can address: measurement integrity, model scope, hard constraints, authority validity, and record completeness. Each supporting claim needs an evidence item and a challenge. For example, a test showing correct state entry under one link outage does not support the stronger claim that every communication failure is handled correctly. The claim must be narrowed or the evidence expanded.

G.13 Example interpretation sheet

For each calculation in this appendix, students should complete four lines: the numerical result; the assumptions; the decision the result may inform; and the decision

it may not authorize. This sheet is a compact habit that prevents equations from becoming detached from scope.

Chapter H

Annotated Reading Dossier

H.1 How to read the historical sources

The historical and literary openings in this textbook are not interchangeable. Read the primary or near-primary text first, then a translation or scholarly commentary, and only then the classroom retelling. Mark which sentences are direct evidence, which are interpretation, and which are scene-setting.

H.1.1 Sima Qian and the Maling narrative

The “Sunzi Wuqi Biography” in *Shiji* is the primary reading for the first chapter’s Maling opening. Students should identify the sequence of action, the visible traces, the decision made by Pang Juan, and the final encounter. The text does not provide modern sensor variables. The engineering reading therefore concerns evidence transformation and system boundaries.

H.1.2 Tian Ji’s horse race

The story is associated with accounts of Sun Bin and Tian Ji. Read the minimal narrative before accepting later horse-by-horse dialogue. The engineering question is constrained allocation: how does a reassignment change an aggregate result when resources are heterogeneous?

H.1.3 Cao Gui and the drumbeats

The relevant passage is in the tenth year of Duke Zhuang in *Zuo Zhuan*. Students should attend to timing, morale, observation, and the later inspection of tracks. The engineering interpretation is a state machine with temporal evidence, guards, and exit conditions.

H.1.4 Cook Ding

The “Nourishing Life” passage in *Zhuangzi* is a philosophical parable. It is useful for discussing structure, feedback, and the limits of forceful action. It is not a field report on perception or motion control.

H.1.5 The rescue of Zhao

The account of Qi, Wei, Zhao, and Sun Bin belongs to the historical tradition around the Warring States. The classroom use is to distinguish visible crisis from mission relevance and to formulate an admissible action set. Students must not convert the narrative into an operational recipe.

H.1.6 The old man and the horse

The frontier story in *Huainanzi* is a parable about uncertain consequences. Its role in the textbook is lifecycle thinking: an immediate outcome does not close a causal chain, and a system needs withdrawal and end-state verification.

H.1.7 Bian Que and Duke Huan

The story appears in *Han Feizi* and related traditions. Its rhetorical force comes from repeated warnings and delayed recognition. The engineering reading is about thresholds, time-to-action, false alarms, missed states, and irreversible regions.

H.1.8 Flood-control traditions

Flood-control materials are layered across myth, history, political theory, and later commentary. They are used here only as conceptual openings for autonomy under restricted communication. Technical claims must come from systems literature and tests.

H.1.9 Three people make a tiger

The expression is a rhetorical and political story about repetition and belief. Its teaching value is provenance analysis. Three reports that share one origin are not three independent observations.

H.1.10 Borrowed arrows

The borrowed-arrows episode belongs to the literary world of *Romance of the Three Kingdoms*. It should be labelled fiction. It opens discussion of obscured visibility, expectation, sensor correlation, and interface semantics.

H.1.11 The plank road and the hidden route

The familiar idiom is a later summary of a historical tradition associated with Han Xin. It is useful for discussing map metadata, incompatible purposes, coordinate frames,

and replanning. Students should avoid attributing the exact idiom to a source that does not contain it.

H.1.12 Beacon fires

The beacon-fire story has several historical and literary versions. Its classroom use is institutional trust: warnings, confirmation, authorization, response, and the memory of previous false alarms must be separated.

H.2 Technical reading route

For system architecture and mobile platforms, read a general autonomous-robot text such as Siegwart, Nourbakhsh, and Scaramuzza. For probabilistic estimation, read selected chapters of Thrun, Burgard, and Fox. For planning algorithms, use LaValle. For uncertainty and model calibration, compare Guo et al., Hendrycks and Gimpel, and Ovadia et al. For risk management, use NIST AI RMF and ISO/IEC 23894.

H.3 Reading notes template

For every technical source, record: problem definition, state and observation variables, assumptions, data or environment, algorithm, evaluation metric, failure cases, configuration, limitations, and relation to one chapter of this book. The last item prevents reading from becoming a list of names; students must explain why a source changes or sharpens their model.

H.4 Citation and adaptation note

This textbook is an independent educational work. The reference textbook supplied for comparison influenced organization and pedagogical density, not copied content. Classical works are cited as historical or philosophical sources. Classroom values are marked as illustrative. Technical claims should be checked against the primary sources listed in the bibliography.

Bibliography

Bibliography

- [1] Sima Qian. “Sunzi Wuqi Biography,” in *Shiji*. Chinese Text Project. <https://ctext.org/shiji/sun-zi-wu-qi-lie-zhuan/zhs>.
- [2] Zuo Qiuming. “The Tenth Year of Duke Zhuang,” in *Zuo Zhuan*. Chinese Text Project. <https://ctext.org/chun-qi-u-zuo-zhuan/zhuang-gong-10-nian/zhs>.
- [3] Zhuangzi. “Nourishing Life,” in *Zhuangzi*. Chinese Text Project. <https://ctext.org/zhuangzi/nourishing-the-principal-of-life/zhs>.
- [4] Han Feizi. “Explaining Lao,” in *Han Feizi*. Chinese Text Project. <https://ctext.org/hanfeizi/explaining-lao/zhs>.
- [5] Ralph D. Sawyer, trans. *Sun Bin: The Art of Warfare*. Boulder: Westview Press, 1995.
- [6] Steven M. LaValle. *Planning Algorithms*. Cambridge: Cambridge University Press, 2006.
- [7] Roland Siegwart, Illah Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.
- [8] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [9] Rudolf E. Kalman. “A New Approach to Linear Filtering and Prediction Problems.” *Journal of Basic Engineering*, 82(1), 1960.
- [10] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. “On Calibration of Modern Neural Networks.” In *Proceedings of ICML*, 2017.
- [11] Dan Hendrycks and Kevin Gimpel. “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks.” In *Proceedings of ICLR*, 2017.
- [12] Yaniv Ovadia et al. “Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift.” In *Proceedings of NeurIPS*, 2019.
- [13] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework 1.0*. NIST AI 100-1, 2023.
- [14] National Institute of Standards and Technology. “AI Risk Management Framework.” Public overview and resources. [public resource](#).

- [15] ISO/IEC. *ISO/IEC 23894:2023 Information Technology – Artificial Intelligence – Guidance on Risk Management*. Geneva: International Organization for Standardization, 2023.
- [16] National Aeronautics and Space Administration. *NASA Systems Engineering Handbook*. Public reference edition. [public resource](#).
- [17] Massachusetts Institute of Technology OpenCourseWare. *18.06SC Linear Algebra*. Public course materials. [public course page](#).
- [18] Massachusetts Institute of Technology OpenCourseWare. *6.041SC Probabilistic Systems Analysis and Applied Probability*. Public course materials. [public course page](#).
- [19] Massachusetts Institute of Technology OpenCourseWare. *2.12 Introduction to Robotics*. Public course materials. [public course page](#).
- [20] Steven M. LaValle. *Planning Algorithms*. Online edition. [public online edition](#).
- [21] Francesco Bullo and Stephen L. Smith. *Lectures on Robotic Planning and Kinematics*, version 0.94, 2025. Used as a structural reference for the present notes; the present book is an independent work.
- [22] Clemens Szyperski. *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Boston: Addison-Wesley, 2002.